

Multi-Verifier Keyed-Verification Anonymous Credentials

Abstract. Keyed-verification anonymous credentials (KVAC) enable privacy-preserving authentication and can be seen as the symmetric version of conventional anonymous credentials: issuance and verification of credentials requires a shared secret key. The core advantage of KVACs is that they can be realized without pairings, which still appear to be a significant bottleneck when it comes to real-world adoption. KVACs provide all the benefits from anonymous credentials, in particular multi-show unlinkability, but only work in the setting where the issuer and verifier are one singular entity, limiting the applications KVACs can be used in. In this work we extend the idea of keyed-verification credential to a setting where again *multiple verifiers* are supported, each sharing an individual secret key with the issuer. We formally introduce this as multi-verifier keyed-verification anonymous credentials (mKVACs). While users must now get verifier-specific credentials, each credential still provides multi-show unlinkability. In terms of security, mKVAC naturally strengthens the single-verifier variant, as it guarantees that corruption of any verifier does not impact unforgeability guarantees for other verifiers. The main challenge therein is to not trade this added flexibility for privacy, and hide the verifier’s identity in the credential issuance. We provide formal definitions of all desired security and privacy features and propose a provably secure and pairing-free construction. Along the way, we develop a new KVAC-like primitive that authenticates group elements and offers statistical privacy, solving the open problem of combining multi-verifier support and pairing-freeness. Finally, we demonstrate practicality of our protocol via implementation benchmarks.

1 Introduction

Anonymous credentials (ACs) [Cha85] let users prove certified attributes, such as age or membership, in an unlinkable manner and revealing only the minimal amount of information necessary for every authentication. Efficient protocols for such anonymous credentials are well-known in the academic community for more than 20 years. While initial schemes relied on RSA-based constructions [CL01, CL03, CL02], the most efficient ones all rely on pairing-based cryptography [BBS04, CL04, FHS19, TZ23, MBS⁺25]. Pairing-based systems are elegant and highly efficient. In particular they are performant enough to be used on high-throughput or resource-limited environments like mobile phones — at least in theory.

Unfortunately, pairing-friendly curves are not recognized by national standardization bodies, and thus also not widely supported in hardware and face

regulatory hurdles [SKSW22]. This hinders the real-world deployment of anonymous credentials, despite seeing a massive interest in these techniques lately. Particularly noteworthy is the EUDI wallet, a digital identity app currently developed by the EU and expected to be launched to 500 million EU citizens by the end of 2026 [Eur25]. The EUDI wallet must satisfy the security and privacy requirements laid out in the eIDAS 2.0 regulation [Eur24], which explicitly demands unlinkability and selective disclosure. While technically this can only be realized with anonymous credentials, the EU decided to revert to batch issuance of one-time ECDSA credentials due to the deployment challenges of pairing-based anonymous credential schemes [BBC⁺24, EU 25]. Apart from not satisfying the privacy features legally required, the batch issuance also comes with significant challenges in deployment and thus the interest in supporting solutions that support multi-show unlinkability remains high — but it must be compatible with secure hardware that is currently on the market, given the tight time-constraints.

KVACs: Anonymous Credentials w/o Pairings. Keyed-verification anonymous credentials (KVACs), introduced by Chase, Meiklejohn, and Zaverucha [CMZ14], offer a solution to realize multi-show unlinkable presentations *without* pairings. In this setting, the issuer and verifier share a secret key, which turns verification of credential into a keyed operation. This reliance on a shared secret key enables to easily turn pairing-based anonymous credential schemes into pairing-free variants [Orr25]. Essentially, they replace the signatures with algebraic message authentication codes (MACs), with the most prominent constructions being CMZ [CMZ14] and BBS [BBS04] MACs. As KVACs can be implemented over standard elliptic curves, they have already been used in practical systems such as Signal’s private group messaging [CPZ20] and Tor’s bridge distribution [TG23].

More recently, KVACs also see strong interest from major tech providers, such as Google and Apple, aiming to improve their systems that currently rely on Privacy Pass [DGS⁺18, HIP⁺24]. Privacy Pass is a protocol suite standardized by the IETF that enables privacy-preserving rate limiting and is currently built from either OPRFs or blind RSA signatures, and has seen massive adoption in real-world applications. Both instantiations rely on batch issuance of many one-time tokens, that users can later redeem in an unlinkable manner. Similar as in the EUDI context, securely realizing batch-issuance for millions of users is a significant deployment challenge. Therefore, both Google and Apple have proposed KVAC-based solutions for so-called Anonymous Rate-Limited Credentials (ARC), that enable multi-show unlinkable presentations from a single credential. Both ARC proposals already undergo standardization in the IETF [YW25, Fas25].

Clearly, the reliance of a shared secret key between the verifier and issuer significantly limits the scenarios where KVACs can be deployed, and makes them unsuitable in open settings such as eID systems. In fact, even in closed enterprise environments such as the Google and Apple deployments, relying on a secret key that needs to be highly replicated among all verifying entities does not seem ideal from a security perspective. The high-availability needed for verification puts the

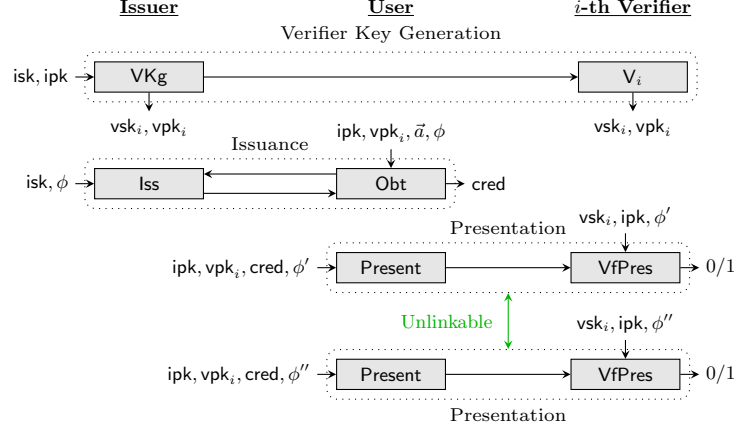


Fig. 1. Illustration of our mKVAC setting, supporting multiple verifiers and enabling multi-show unlinkable credential presentation.

secret key at risk, which provides the foundation for the unforgeability of the entire authentication system.

KVACs for Multiple Verifiers. The fundamental drawback of KVACs is that the issuer and the verifier need to share a secret key. This raises the question of how to handle settings with *multiple* verifiers, e.g., in an eID scheme where one government agency issues credentials but different authorities and online services must verify them. A naïve approach would be for all verifiers to share the issuer’s secret key. This is clearly insecure as the compromise of a single, less protected verifier would allow forging arbitrary credentials. A more reasonable alternative is for the issuer to maintain a separate KVAC key for each verifier. This way, verifiers cannot forge credentials for each other, and a leaked verifier key only impacts that one verifier. This makes credential issuance verifier-specific, i.e. whenever the user encounters a new verifier, they need to obtain a new credential specifically for this verifier. This immediately destroys a fundamental privacy property of anonymous credentials: the issuer learns the services the users wants to access, essentially defeating much of the purpose of using anonymous credentials in the first place.

1.1 Our Contributions

In this paper, we push the boundaries of anonymous credentials that can be realized without pairings by introducing *multi-verifier keyed-verification anonymous credentials* (mKVACs), essentially extending KVACs to the multi-verifier setting. Our goal is to preserve the multi-show unlinkability property of KVACs while avoiding the privacy issues associated with verifier-specific issuance outlined above. We formally define mKVAC and its desired security and privacy

guarantees. We then give a pairing-free construction whose tokens are compatible with the popular CMZ scheme [CMZ14] (meaning that most extensions to and applications of CMZ are compatible with our mKVAC scheme). In more detail we provide the following contributions:

Formal Model for mKVACs. We start by formally defining this new type of keyed-verification credentials, supporting multiple verifiers. The overall mKVAC setting is illustrated in Figure 1. In mKVAC the issuer still maintains a single short secret key isk . With it, the issuer can derive many verifier key pairs, (vpk_i, vsk_i) , and we imagine verifiers registering with the issuer to obtain their key. This registration process for verifiers, i.e., relying parties, is part of the eIDAS 2.0 specifications [Eur24, Article 5b], and is also a common pattern in legacy protocols such as OpenID Connect. The user can obtain a verifier-specific credential from the issuer, certifying her attributes along the verifier public key vpk_i . Importantly, the verifier key vpk_i must remain hidden towards the issuer. The user can then use such a verifier-specific credential to prove a predicate over her attested attributes to the verifier associated with vpk_i , who uses its secret key vsk_i to verify the presentation. This process can be safely repeated multiple times using the same credential, and does not require any further interaction with the issuer.

More precisely, we formalize the following security and privacy properties for mKVACs: **Unforgeability** covering the expected unforgeability of attributes and presentation. Crucially, compromise of any verifier key vsk_i does not impact the unforgeability guarantees for any verifier $i' \neq i$. **Presentation anonymity** ensuring the standard unlinkability and privacy guarantees expected from anonymous credentials, in particular *multi-show unlinkability*. **Issuance anonymity** guarantees that the issuer does not learn anything about the verifier the user requests a credential for (and hides additional attributes, as our scheme supports partially-blind issuance too). Further, we leverage the fact that we are in the keyed-verification setting to express an additional property that is desirable in the context of identity systems but usually considered out-of-scope: **Deniability**, ensuring that a verifier cannot provide convincing evidence to a third party that a given presentation was produced by using a legitimate credential.

Pairing-Free Constructions. We present two practical mKVAC constructions, offering different trade-offs between communication costs and privacy guarantees. We formally prove the security of our constructions under standard cryptographic assumptions. We demonstrate that they are practical and efficient for real-world applications, through a prototype implementation. For a credential with 8 attributes, presenting and verification can be run in less than 1.3ms for each party, and generating proofs of size 1KB.

Our constructions are given in a semi-generic manner and rely on an additional building block we formally introduce first and we believe to be of independent interest: Symmetric anonymous group element authenticators (SAGA), an algebraic MAC on vectors of group elements with a special KVAC-style presentation mechanism. This will be used to authenticate the verifiers' public keys,

which the user can then use to blindly obtain credentials on a previously authenticated $\mathbf{vpk}$. We formalize this building block and provide two instantiations. In the following, we give a high-level idea of our constructions before presenting our formal $\mathbf{mKVAC}$ and $\mathbf{SAGA}$ models, as well as concrete instantiations for both.

1.2 Technical Outline

One important constraint for our construction is that the issuer should not have to do work linear in the number of verifiers every time a credential is issued. This is because issuers may be restricted in hardware (e.g., HSMs) and we want to support large numbers of verifiers. Without additional assumptions (such as non-colluding servers), this rules out approaches based on private information retrieval or similar techniques. It is a priori unclear how to construct $\mathbf{mKVAC}$. There are approaches for *hiding the issuer* of a credential from a verifier (e.g., [BEK⁺21, ST24]), but these are inapplicable to our task, which is to *hide the verifier* from the issuer and which has not been tackled before. We now present the high-level blueprint of our $\mathbf{mKVAC}$ construction and explain the challenges we had to overcome.

High-Level Idea. Our approach is to build $\mathbf{mKVAC}$ by combining two KVAC mechanisms in a nested way. One layer is used to authenticate the verifier’s public key, and the other is used to issue credentials bound to that authenticated key. Instead of having the issuer directly store a separate secret key for every verifier, each verifier’s public key contains an encrypted form of its signing key. The issuer holds the matching decryption key and can recover just enough information during issuance to produce a valid credential, without learning which verifier is involved. In practice, the user sends a blinded version of the verifier’s public key to the issuer, the issuer applies its decryption key to obtain the necessary signing component, and then completes issuance of a blinded credential. This design keeps the issuer’s role lightweight, while each verifier still holds the secret needed to check credentials.

Challenges to Overcome. However, we first need to address several challenges with this approach. First, hiding the signing key in the verifier’s public key (in some randomizable/blindable form) seems to imply that the signing key must be a group element. This informs our choice of using CMZ tokens [CMZ14] for our $\mathbf{mKVAC}$, as its signing key is a group element. For this reason, we refer to this variant as $\mathbf{mKVAC}_{\text{CMZ}}$.

Second, the CMZ issuance protocol involves the user computing a Pedersen commitment C_a to their desired attributes $a_1, \dots, a_n$ on a basis $(X_1, \dots, X_n) \in \mathbb{G}^n$, derived from the verifier’s secret key. Crucially, the integrity of this basis must be protected. If, for example, the user were to use the basis $(2X_1, \dots, 2X_n)$ for the commitment, they can effectively receive a credential for attributes $2a_1, \dots, 2a_n$, which may not satisfy the issuance policy. This leads to a need to authenticate the basis (alongside the other parts of the verifier’s public key) and to prove in zero-knowledge that C_a is a valid commitment on *hidden* attributes

$\vec{a}$ on a *hidden* basis $\vec{X}$ that is *authenticated*. Such statements (where both the basis and the message of a Pedersen commitment are hidden) are notoriously difficult to prove using efficient Sigma protocols.

Need for a new primitive: SAGA. We solve this problem by introducing *symmetric anonymous group element authenticators* (SAGA), an algebraic MAC on vectors of group elements with a special KVC-style presentation mechanism. SAGA is used to authenticate the verifier’s commitment basis $\vec{X}$. Its presentation mechanism hides $\vec{X}$ in a way carefully designed to enable efficient Sigma protocol proofs such as the commitment proof for C_a above. However, constructing SAGA presents another challenge: while there are MACs on group elements with a suitable presentation mechanism to instantiate SAGA (for example the CPZ MAC [CPZ20], cf. Appendix L), their privacy property can be broken by future quantum computers. Surprisingly, there are no MACs on group elements with a statistically private presentation mechanism in the literature. We design such a MAC, leading to our BBSSAGA construction (the form of the MAC is reminiscent of BBS signatures [BBS04, TZ23]). We prove it secure in the generic group model (GGM) and its presentation mechanism is statistically secure. BBSSAGA may be of independent interest. It enables our mKVC construction to remain secure against attackers that harvest today’s traffic to attack its privacy using future quantum computers.

Finally, we construct the issuance protocol of mKVC_{CMZ} by using SAGA to ensure that the user is using a valid verifier commitment basis for the CMZ issuance protocol while statistically hiding the verifier’s public key. To make this construction efficient, we solve several technical and optimization problems. See Section 5 for details on our final construction.

1.3 Related Work

Our mKVC scheme is positioned somewhere in between KVCs and publicly-verifiable anonymous credentials. They preserve the pairing-freeness of KVCs, but cater for multiple (yet registered) verifiers, getting closer to an open setting.

Another noteworthy recent work that aims at providing pairing-free anonymous credentials are *server-aided anonymous credentials* (SAACs) [CAHLT25]. Their approach also implicitly relies on KVCs but is entirely orthogonal to our work: they aim at public-verifiability, whereas we are still in a privately-verifiable (but multi-verifier) setting. SAAC achieves public verifiability by sacrificing multi-show unlinkability. That is, while the user is in possession of a KVC credential, she also needs to get a helper token from the issuer for every unlinkable presentation she wants to derive from that credential. This token essentially transforms the private-verifiable credential into a publicly verifiable presentation with the help of an always-online helper server. Thus, in terms of public verifiability, SAACs are clearly more powerful than our mKVCs. However, SAAC inherits the same batch issuance challenges currently faced in the EUDI context with batch-issued ECDSA credentials and Privacy Pass. In fact, seeing that companies such as Google and Apple that deployed Privacy Pass

Table 1. Comparison of Anonymous Credential and Related Schemes. The number of attributes is n . Credential size excludes storage of attributes. $\mathbb{G}$: size of group elements. $\mathbb{Z}_p$: size of scalars. All security analyses assume the ROM. *: Showing requires helper server interaction which can be performed offline and batched. †: Multi-Verifier.

Scheme	Public vrfy	Multi-show	Pairing-free	Cred. size	Presentation size	Assumption	Anon-imity
Traditional Anonymous Credentials							
CL01 [CL01, CL03]	✓	✓	✓	$O(1)$	$O(n)$	Strong RSA	Comp.
ACL [BL13, KLR23]	✓	×	✓	$2\mathbb{G} + 6\mathbb{Z}_p$	$2\mathbb{G} + (n + 8)\mathbb{Z}_p$	AGM+DL	Stat.
BBS [BBS04, TZ23]	✓	✓	×	$1\mathbb{G}_1 + \mathbb{Z}_p$	$2\mathbb{G}_1 + (n + 3)\mathbb{Z}_p$	q-SDH	Stat.
Brands [Bra95, PZ13]	✓	×	✓	$O(n)$	$O(n)$	DL	Comp.
SNARK-based							
LongFellow [Fs24]	✓	✓	✓	$O(n)$	$O(\text{polylog}(n))$	Sum-check	Stat.
Semaphore [Eth19]	✓	✓	×	$O(\log n)$	$O(1)$	SNARKs	Comp.
KVAC							
CMZ14 [CMZ14]	×	✓	✓	$2\mathbb{G}$	$(n + 2)\mathbb{G} + (2n + 2)\mathbb{Z}_p$	GGM	DDH
DGSTV18 [DGS ⁺ 18]	×	✓	✓	$2\lambda + \mathbb{G}$	4λ	gap-OMCDH	Stat.
BBDT16 [BBDT16]	×	✓	✓	$2\mathbb{G} + 2\mathbb{Z}_p$	$3\mathbb{G} + (n + 7)\mathbb{Z}_p$	Gap-q-SDH	Stat.
CDDH19 [CDDH19]	×	✓	✓	$(n + 1)\mathbb{G}$	$2\mathbb{G} + (n + 1)\mathbb{Z}_p$	n -SCDHI	Stat.
CPZ20 [CPZ20]	×	✓	✓	$2\mathbb{G} + \mathbb{Z}_p$	$(n + 3)\mathbb{G} + 4\mathbb{Z}_p$	GGM	DDH
μ CMZ [Orr25]	×	✓	✓	$2\mathbb{G}$	$(n + 2)\mathbb{G} + (2n + 2)\mathbb{Z}_p$	AGM+3-DL	Stat.
μ BBS [Orr25]	×	✓	✓	$1\mathbb{G} + \mathbb{Z}_p$	$2\mathbb{G} + (n + 4)\mathbb{Z}_p$	AGM+q-DL	Stat.
MBS+25 [MBS ⁺ 25]	×	✓	✓	$(n + 2)\mathbb{G}$	$2\mathbb{G}$	GGM	Stat.
SAAC							
SAAC _{BBS} [CAHLT25]	✓	×	✓	$1\mathbb{G} + 2\mathbb{Z}_p$	$3\mathbb{G} + (n + 8)\mathbb{Z}_p$	Gap-q-SDH	Stat.
SAAC _{DDH} [CAHLT25]	✓	×	✓	$4\mathbb{G}$	$(n + 6)\mathbb{G} + (4n + 11)\mathbb{Z}_p$	DDH	DDH
Multi-Verifier KVAC (This Work)							
mKVAC _{CMZ}	✓ [†]	✓	✓	$2\mathbb{G}$	$(n + 3)\mathbb{G} + (2n + 4)\mathbb{Z}_p$	GGM	Stat.

at scale, are now replacing their batch-issuance based implementation that was publicly-verifiable and had eternal privacy through a KVAC-based scheme with DL-based privacy, demonstrates the need for solutions that have multi-show unlinkability built in — which is what our solution provides.

Table 1 provides a comprehensive comparison of various anonymous credential schemes and related systems. We discuss additional related work and give a broader overview in Appendix A.

2 Preliminaries

Notation. We introduce some notation that we use throughout the paper. We notate a vector as $\vec{a}$ and the i th element of the vector as a_i . Given $\vec{a}, \vec{b} \in \mathbb{Z}_p^n$, $\vec{X}, \vec{Y} \in \mathbb{G}^n$, and $Z \in \mathbb{G}$: $\vec{a} \cdot \vec{b}$ and $\vec{a} \circ \vec{b}$ gives the inner product and the entrywise product of the two vectors, respectively. Similarly $\vec{a} \cdot \vec{X} := \sum_{i \in [n]} a_i X_i$, $\vec{a} \circ \vec{X} :=$

$(a_i X_i)_{i \in [n]}$, and $\vec{a}Z := (a_i Z)_{i \in [n]}$. In our game descriptions, **require** x is a shortcut for **if** $\neg x$: **return** $\perp$.

Groups. We rely on pairing-free prime order groups with the description $(\mathbb{G}, p, G)$ such that $\langle G \rangle = \mathbb{G}$ and $|\mathbb{G}| = p$ throughout the paper. We describe a group generator $\text{GGen}(\lambda)$ that outputs a group description with $|p| > 2\lambda$. Throughout the paper we are going to rely on the two algebraic assumptions on groups which we sketch below and formally define in Appendix B: (1) the DDH Assumption: Given $(\mathbb{G}, p, G, A := aG, B := bG, C)$ for $a, b \xleftarrow{\$} \mathbb{Z}_p$, it is hard for an adversary to distinguish whether $C = abG$ or $C \xleftarrow{\$} \mathbb{G}$. (2) n -DL Given $(\mathbb{G}, p, G, (A_i := (a^i)G)_{i \in [n]})$ for $a \xleftarrow{\$} \mathbb{Z}_p$, it is hard for an adversary to output a .

NIZK PoK. Below we describe our syntax for NIZK proofs-of-knowledge schemes $(\text{NIP}_{\mathcal{R}})$ and sketch the properties of them.

Definition 1. A $\text{NIP}_{\mathcal{R}}$ scheme for a relation $\mathcal{R}$ is a tuple of algorithms s.t.:

$\text{Setup}(\lambda) \rightarrow \text{pp}_{\text{NIP}}$: Outputs the public parameters for input security parameter.
 $\text{Prove}(\mathfrak{x}, \mathfrak{w}, \text{ctx}_{\text{NIP}}) \rightarrow \pi$: Given $(\mathfrak{x}, \mathfrak{w}) \in \mathcal{R}$, outputs a proof π for the context ctx_{NIP} .
 $\text{Vf}(\mathfrak{x}, \text{ctx}_{\text{NIP}}, \pi) \rightarrow b \in \{\text{true}, \text{false}\}$: It verifies the proof π against the statement $\mathfrak{x}$ for the context ctx_{NIP} .

Below we give the informal descriptions of the NIP properties that we rely on, and present their formal definitions in Appendix B.

Completeness. $\text{NIP}_{\mathcal{R}}$ has completeness if for all $\text{pp}_{\text{NIP}} \leftarrow \text{Setup}(\lambda)$, $(\mathfrak{x}, \mathfrak{w}) \in \mathcal{R}$, $\text{ctx}_{\text{NIP}} \in \{0, 1\}^*$, and $\pi \leftarrow \text{Prove}(\mathfrak{x}, \mathfrak{w})$, $\Pr[\text{Vf}(\mathfrak{x}, \text{ctx}_{\text{NIP}}, \pi) = \text{true}] = 1$.

Zero-knowledge. $\text{NIP}_{\mathcal{R}}$ is zero-knowledge if there exists a PPT simulator $S := (S.\text{Setup}, S.\text{Prove})$, s.t. for all adversaries, it is hard to distinguish honestly generated pp_{NIP} and π from the simulated ones through $S.\text{Setup}$ and $S.\text{Prove}$.

Soundness. $\text{NIP}_{\mathcal{R}}$ is sound if it is hard for PPT adversaries to forge a proof π on $(\mathfrak{x}, \text{ctx}_{\text{NIP}})$ such that $\text{Vf}(\mathfrak{x}, \text{ctx}_{\text{NIP}}, \pi) = \text{true}$, yet *there is no* valid witness $\mathfrak{w}$ such that $(\mathfrak{x}, \mathfrak{w}) \in \mathcal{R}$.

Extractability. $\text{NIP}_{\mathcal{R}}$ is extractable if there exists an extractor Extract which can extract a valid witness from all adversarially created verifying proofs. Extractability property can be achieved with a *rewinding-based* extractor requiring access to the adversary's random coins, or a *straightline* extractor.

Simulation-extractability. $\text{NIP}_{\mathcal{R}}$ is simulation-extractable if the extractability property holds even when the adversary can see simulated proofs.

Throughout the paper, we give the relations that we wish to be proven without immediately instantiating them. We describe the corresponding Sigma protocols in Appendix C and use either Fiat-Shamir [FS87] or Fischlin [Fis05, Ks22, LR22] compiled NIP schemes in our instantiations.

$\text{CMZ.Setup}(\lambda, n)$	$\text{CMZ.Sign}(\text{sk}, \vec{m})$	$\text{CMZ.Vf}(\text{sk}, \sigma, \vec{m})$
$(\mathbb{G}, p, G) \leftarrow \text{GGen}(\lambda)$	parse sk as $(x_0, \vec{x})$	parse σ as (U, V)
return $\text{pp} := (\mathbb{G}, p, G)$	$U \xleftarrow{\$} \mathbb{G}$	parse sk as $(x_0, \vec{x})$
$\text{CMZ.Kg}(\text{pp})$	$V := (x_0 + \vec{x} \cdot \vec{m})U$	return $\begin{pmatrix} V = (x_0 + \vec{x} \cdot \vec{m})U \\ \wedge U \neq 0 \end{pmatrix}$
$x_0 \xleftarrow{\$} \mathbb{Z}_p, \vec{x} \xleftarrow{\$} \mathbb{Z}_p^n$	return $\sigma := (U, V)$	
return $(\text{sk} := (x_0, \vec{x}), \text{pk} := \vec{x}G)$		

Fig. 2. Description of CMZ MAC [CMZ14].

MAC Definition and CMZ MAC. A $\text{MAC} := (\text{Setup}, \text{Kg}, \text{Sign}, \text{Vf})$ scheme is a tuple of algorithms such that $\text{Setup}(\lambda) \rightarrow \text{pp}$ outputs the public parameters, $\text{Kg}(\text{pp}) \rightarrow (\text{sk}, \text{pk})$ outputs a key pair, $\text{Sign}(\text{sk}, m) \rightarrow \sigma$ outputs a tag σ on message m , and $\text{Vf}(\text{sk}, \sigma, m)$ verifies whether σ is a valid tag on message m . We recap the CMZ MAC scheme by Chase et al. [CMZ14] in Figure 2 and its unforgeability game in Section B.2.

3 Definition of mKVAC

An mKVAC system has three main entities: issuer, users and verifiers. In contrast to classic keyed-verification (KVAC) schemes, where the issuer and verifier are the same entity sharing a single secret key, our mKVAC version supports *multiple* verifiers. That is, verification is still a keyed operation, but now takes a verifier-specific secret key as input. This enables a more distributed setting than classic KVAC. Issuance is still done by the single issuer, and users must acquire credentials for the targeted verifiers. The challenge therein is to hide the verifier's identity from the issuer during issuance. This is what we capture through the notation of *issuance anonymity*, that complements the standard notions of *unforgeability* and *presentation anonymity*. As mKVAC targets a more open and distributed setting than classic KVACs, but still works in a keyed-verification setting, we further leverage this keyed aspect to demand *deniability* of the presented information. We begin by specifying the syntax of mKVAC schemes, and then formalize these desired security guarantees through game-based definitions.

3.1 Syntax

We start by explaining the syntax of mKVAC along its three main phases: setup and key generation, issuance of credentials and presentations, followed by the formal definition of all algorithms.

Setup and Keys. The **Setup** algorithm initializes the public parameters, which also determine the set of allowed predicates ϕ used throughout the scheme. The issuer generates its key pair (isk, ipk) by running **IKg**. For each verifier, the issuer additionally generates a verifier key pair (vsk, vpk) by running **VKg** on isk . Recall that we are still in a keyed-verification setting, i.e., such a dependency from the issuer's *secret* key is needed here.

(Partially-blind) Issuance. Credential issuance is a two-move protocol between the issuer and a user, captured through the algorithms Iss run by the issuer, and $\text{Obt}_1, \text{Obt}_2$ run by the user. The user initiates the protocol on input the issuer and verifier public keys ipk, vpk , the attribute vector $\vec{a}$, and a predicate ϕ which is satisfiable by $\vec{a}$. The issuer's inputs are its secret key isk and the predicate ϕ but not the complete attribute vector $\vec{a}$ and not the vpk . Upon successful completion of issuance, the user obtains a valid credential cred for $\vec{a}$ and vpk .

Note that our protocol does not only hide the verifier's identity vpk from the issuer, but also supports partially blind issuance. The issuer is provided only with a predicate ϕ that is satisfied by the user's attributes, instead of the attributes directly (this aspect is similar to [Orr25, CAHLT25]). This abstraction enables a wide range of applications and extensions. For instance, our scheme naturally supports pseudonyms, where the user obtains a credential on a secret value that later seeds a PRF, without disclosing this key to the issuer. Likewise, it enables blind attribute transfer, allowing a user to prove possession of attributes from an existing credential and obtain a new credential on the same attributes without revealing them. This is particularly useful when transferring credentials to new verifiers or updating them for existing attributes [BBDE19]. We can imagine users holding a source credential on attributes $\vec{a}$, where the issuer is the verifier. When requesting a credential for a new verifier, the user would transfer the attributes $\vec{a}$ to the new credential without revealing them to the issuer. This enables even more flexible security: the issuer is fully blind and does not learn anything about this reissuance process (neither what verifier the user wants to use nor what attributes the user has).

Presentation. To create a credential presentation pres , the user runs Present on a predicate ϕ and a presentation context ctx , that it wants to share with the verifier. This algorithm additionally takes the public keys ipk, vpk of the issuer and targeted verifier, as well as a credential cred that is valid for ϕ and vpk as input. The presentation context ctx can contain a nonce, or any other information the presentation shall be bound to, preventing replay attacks. Finally, a verifier runs VfPres using its secret key vsk to check the validity of a presentation pres on a predicate ϕ for a presentation context ctx .

Definition 2 (Multi-Verifier KVAC). *An mKVAC scheme is defined through the following algorithms:*

- $\text{Setup}(\lambda, n) \rightarrow \text{pp}$: For a security parameter λ and the attribute vector size n , it outputs the public parameters pp to issue credentials on attribute vectors from $\mathbb{A}^n$. We assume that pp fixes the set of allowed predicates $\phi : \mathbb{A}^n \rightarrow \{\text{true}, \text{false}\}$ during the issuance and presentation. We only make pp an explicit input to IKg and treat it as an implicit input to all other algorithms.
- $\text{IKg}(\text{pp}) \rightarrow (\text{isk}, \text{ipk})$: Outputs an issuer key pair (isk, ipk) .
- $\text{VKg}(\text{isk}) \rightarrow (\text{vsk}, \text{vpk})$: Given isk , it outputs a verifier key pair (vsk, vpk) .
- $\text{Obt}_1(\text{ipk}, \text{vpk}, \vec{a}, \phi) \rightarrow (\text{creq}, \text{st})$: For an issuer public key ipk , a verifier public key vpk , vector of attributes $\vec{a} \in \mathbb{A}^n$, and a credential predicate ϕ , it outputs a credential request creq and issuance state st .

- $\text{Iss}(\text{isk}, \phi, \text{creq}) \rightarrow \text{bcred} / \perp$: For an issuer secret key isk , a credential request creq , and a predicate ϕ , it outputs a blinded credential bcred if the issuance is successful, and $\perp$ otherwise.
- $\text{Obt}_2(\text{st}, \text{bcred}) \rightarrow \text{cred} / \perp$: Given an issuance state st and a blind credential bcred , it outputs the finalized credential cred if the issuance is successful, and $\perp$ otherwise.
- $\text{Present}(\text{ipk}, \text{vpk}, \text{cred}, \text{ctx}, \phi) \rightarrow \text{pres}$: For an issuer and verifier public keys ipk and vpk , a credential cred , a presentation context ctx , and a credential predicate ϕ , it outputs a credential presentation pres .
- $\text{VfPres}(\text{vsk}, \text{ipk}, \text{pres}, \text{ctx}, \phi) \rightarrow b \in \{\text{true}, \text{false}\}$: For a verifier secret key vsk , an issuer public key ipk , a presentation pres , a presentation context ctx , and a credential predicate ϕ , it verifies the credential.

The correctness definition is given in Appendix F for completeness.

3.2 Unforgeability

Unforgeability requires that the adversary must not be able to present predicates for which they do not have an underlying credential. While the issuer is assumed to be honest for unforgeability, we aim at more granularity for the verifiers. Namely, our mKVAC supports *multiple* verifiers where all but one may be compromised. The adversary's goal is to forge a valid presentation for the remaining honest verifier. Our full definition is given in Figure 3. The game is parameterized with an extractor $\mathbf{E}$, which we use to extract hidden attributes $\vec{a}$ during issuance (similar to [Orr25] for single-verifier KVACs). The adversary receives the public keys $\text{ipk}^*, \text{vpk}^*$ of the honest issuer and verifier as input, and can interact with all honest parties through a number of oracles before outputting its alleged forgery. To model the aforementioned corruption of other verifiers, the adversary can request verifier key pairs (vsk, vpk) through the oracle $\mathcal{OVKg}$. For the honest challenge verifier, the adversary is granted the oracle $\mathcal{OVfPres}$ that models keyed-verification with the honest vsk^* .

The adversary can obtain credentials and presentations through a number of oracles. When acting in the role of corrupt users, they can request credentials on blinded inputs through the $\mathcal{OCCIssue}$ oracle. We obtain the attributes $\vec{a}$ from the blinded request creq using the extractor $\mathbf{E}$. The game adds the extracted $\vec{a}$ to $\mathcal{CC}$, the set of corrupted credentials. This bookkeeping helps define trivial wins. Honest users are simulated through two oracles. First, $\mathcal{OHCIssue}$ simulates the issuance of credentials for attributes $\vec{a}$ and a verifier vpk of the adversary's choice. The oracle internally runs the issuance, and, upon successful completion, stores the credential along with $\vec{a}$ and vpk . Unlike corrupt users, the adversary does not learn the issued credential but instead can ask for presentations thereof through the $\mathcal{OHCSHOW}$ oracle. The adversary can request presentations for predicates and contexts of their choice, and presentations for the challenge vpk^* are stored for bookkeeping of trivial wins in $\mathcal{HP}$.

Eventually, the adversary ends by outputting a forgery $(\text{pres}^*, \text{ctx}^*, \phi^*)$, and wins if it is valid under vsk^* and non-trivial. The tuple $\vec{a}^*, \text{ctx}^*, \phi^*$ is considered

$\frac{\text{Exp}_{\mathcal{A}, \text{mKVAC}, \mathbf{E}}^{\text{Unf}}(\lambda)}{c := 0, \text{pp} \leftarrow \text{Setup}(\lambda, n)$ $(\text{isk}^*, \text{ipk}^*) \leftarrow \text{IKg}(\text{pp})$ $(\text{vsk}^*, \text{vpk}^*) \leftarrow \text{VKg}(\text{isk}^*)$ $(\text{pres}^*, \text{ctx}^*, \phi^*) \leftarrow \mathcal{A}^{\mathcal{O}}(\text{pp}, \text{ipk}^*, \text{vpk}^*)$ $\text{return} \left(\begin{array}{l} \text{VfPres}(\text{vsk}^*, \text{ipk}^*, \text{pres}^*, \text{ctx}^*, \phi^*) \\ \wedge (\text{ctx}^*, \phi^*) \notin \mathcal{HP} \\ \wedge (\forall \vec{a}^* \in \mathcal{CC} : \neg \phi^*(\vec{a}^*)) \end{array} \right)$ $\frac{\mathcal{OHCShow}(i, \text{ctx}, \phi)}{\text{parse } \mathcal{HC}[i] \text{ as } (\text{vpk}, \text{cred}, \vec{a})}$ $\text{require } \phi(\vec{a}) = \text{true}$ $\text{if } \text{vpk}^{(i)} = \text{vpk}^* : \mathcal{HP} := \mathcal{HP} \cup \{(\text{ctx}, \phi)\}$ $\text{pres} \leftarrow \text{Present}(\text{ipk}^*, \text{vpk}, \text{cred}, \text{ctx}, \phi)$ return pres $\frac{\mathcal{OVfPres}(\text{pres}, \text{ctx}, \phi)}{\text{return VfPres}(\text{vsk}^*, \text{ipk}^*, \text{pres}, \text{ctx}, \phi)}$	$\frac{\mathcal{OVKg}()}{}(\text{vsk}, \text{vpk}) \leftarrow \text{VKg}(\text{isk}^*)$ $\text{return } (\text{vsk}, \text{vpk})$ $\frac{\mathcal{OHCIssue}(\text{vpk}, \vec{a}, \phi)}{} \text{require } \phi(\vec{a}) = \text{true}$ $(\text{creq}, \text{st}) \leftarrow \text{Obt}_1(\text{ipk}^*, \text{vpk}, \vec{a}, \phi)$ $\text{bcred} \leftarrow \text{Iss}(\text{isk}^*, \phi, \text{creq})$ $\text{cred} \leftarrow \text{Obt}_2(\text{st}, \text{bcred})$ $\text{require cred} \neq \perp$ $c := c + 1, \mathcal{HC}[c] := (\text{vpk}, \text{cred}, \vec{a})$ $\text{return } c$ $\frac{\mathcal{OCCIssue}(\text{creq}, \phi)}{} \text{bcred} \leftarrow \text{Iss}(\text{isk}^*, \phi, \text{creq})$ $\text{require bcred} \neq \perp$ $\vec{a} \leftarrow \text{E}(\text{creq}, \phi, Q_{\text{RO}})$ $\text{if } \phi(\vec{a}) : \mathcal{CC} := \mathcal{CC} \cup \{\vec{a}\}$ return bcred
---	---

Fig. 3. Presentation unforgeability game for mKVAC.

to be non-trivial if the following holds: (1) No honest presentation on ϕ^* and context ctx^* exists in $\mathcal{HP}$ (the adversary cannot trivially win by outputting an honest user's presentation). (2) None of the attributes $\vec{a}^*$ issued to the adversary (according to the extractor $\mathbf{E}$) fulfills the presentation predicate ϕ^* (the adversary cannot win by presenting a credential it was issued).

Definition 3. An mKVAC scheme is unforgeable for an extractor $\mathbf{E}$ if for all PPT adversaries $\mathcal{A}$, $\text{Adv}_{\text{mKVAC}, \mathcal{A}, \mathbf{E}}^{\text{Unf}}(\lambda) := \Pr[\text{Exp}_{\mathcal{A}, \text{mKVAC}, \mathbf{E}}^{\text{Unf}}(\lambda) = \text{true}] \leq \text{negl}(\lambda)$.

Remarks on Extractor and ROM. Our definition is explicitly given in the ROM. We assume all algorithms and the adversary have access to a random oracle as in Figure 15 in Appendix B, and do not restate details here. The extractors take as input the set Q_{RO} of all random oracle queries made thus far to align with straight-line extractors obtained via the Fischlin transform.

We further stress that extractor-based unforgeability is necessary for a meaningful security notion under blind issuance, as was previously observed in other works [Orr25, CAHLT25, CKL⁺14].

3.3 Presentation Anonymity

The core privacy property of anonymous credentials ensures that presentations do not reveal any information beyond their corresponding predicate and designated verifier. We model presentation anonymity via an indistinguishability

$\text{Exp}_{\mathcal{A}, \text{mKVAC}}^{\text{PAnon}}(\lambda)$	$\mathcal{OLoR}(c^{(0)}, c^{(1)}, \phi)$
$b \leftarrow \{0, 1\}, c := 0$	require $\mathcal{HC}[c^{(0)}] \neq \perp \wedge \mathcal{HC}[c^{(1)}] \neq \perp$
$\text{pp} \leftarrow \text{Setup}(\lambda, n)$	for $i \in \{0, 1\}$: parse $\mathcal{HC}[c^{(i)}]$ as $(\text{cred}^{(i)}, \vec{a}^{(i)}, \text{vpk}^{(i)})$
$\text{ipk} \leftarrow \mathcal{A}(\text{pp}), b' \leftarrow \mathcal{A}^{\mathcal{O}}()$	require $\text{vpk}^{(0)} = \text{vpk}^{(1)} \wedge \phi(\vec{a}^{(0)}) \wedge \phi(\vec{a}^{(1)})$
return $b = b'$	return $\text{pres}^{(b)} \leftarrow \text{Present}(\text{ipk}, \text{vpk}^{(b)}, \text{cred}^{(b)}, \phi)$
$\mathcal{OObt}_1(\text{vpk}, \vec{a}, \phi)$	$\mathcal{OObt}_2(c', \text{bcred})$
require $\phi(\vec{a}) = \text{true}$	require $c' \leq c \wedge \mathcal{HC}[c'] = \perp$
$c := c + 1$	parse $\mathcal{HR}[c']$ as $(\text{st}, \vec{a}, \text{vpk})$
$(\text{creq}, \text{st}) \leftarrow \text{Obt}_1(\text{ipk}, \text{vpk}, \vec{a}, \phi)$	$\text{cred} \leftarrow \text{Obt}_2(\text{st}, \text{bcred})$
$\mathcal{HR}[c] := (\text{st}, \vec{a}, \text{vpk})$	require $\text{cred} \neq \perp$
return (c, creq)	$\mathcal{HC}[c'] := (\text{cred}, \vec{a}, \text{vpk})$
	return cred

Fig. 4. Presentation Anonymity Game for mKVAC.

game in Figure 4. The issuer and all verifiers are corrupt here, and the challenger represents honest users. The adversary can trigger honest users to obtain credentials for attributes of $\mathcal{A}$'s choice, through the oracles $\mathcal{OObt}_1$ and $\mathcal{OObt}_2$. Note that $\mathcal{OObt}_2$ reveals the resulting credential cred to the adversary, so presentation anonymity does not rely on credential secrecy. The challenge itself is provided by the left-or-right oracle $\mathcal{OLoR}$: given two credential identifiers $c^{(0)}$ and $c^{(1)}$, and a predicate ϕ , the oracle returns the presentation $\text{pres}^{(b)}$ derived from $c^{(b)}$. Trivial wins are excluded by checking that both credentials satisfy ϕ and they are issued for the same vpk . At the end of the game, the adversary outputs a guess for b and wins if the guess is correct.

Definition 4. An mKVAC scheme is *presentation anonymous* if for all PPT adversaries $\mathcal{A}$, $\text{Adv}_{\mathcal{A}, \text{mKVAC}}^{\text{PAnon}}(\lambda) := \left| \Pr \left[\text{Exp}_{\mathcal{A}, \text{mKVAC}}^{\text{PAnon}}(\lambda) = \text{true} \right] - 1/2 \right| \leq \text{negl}(\lambda)$.

Remarks. Our model provides the adversary with strong powers throughout the game, in particular it allows the adversary to choose the issuer public key (whereas e.g., Orru and Chase et al. [Orr25, CMZ14] assume honest key generation), and also explicitly reveals the generated credentials to $\mathcal{A}$, ensuring that users' privacy does not require secrecy of their credentials. This is why we split the anonymity definitions into two: as we give the issued credentials in clear to strengthen the presentation anonymity property, we cannot challenge the adversary on the indistinguishability of the credential requests anymore. Thus, we analyze issuance anonymity as a separate property, defined in the next section. We compare these choices with related work in Appendix E.

3.4 Issuance Anonymity

Our second privacy notion targets the issuance phase. Beyond the blind issuance aspect, ensuring that malicious issuers do not learn any information about $\vec{a}$

$\text{Exp}_{\mathcal{A}, \text{mKVAC}}^{\text{IAnon}}(\lambda)$ $\text{pp} \leftarrow \text{Setup}(\lambda, n)$ $(\text{ipk}, \phi, (\text{vpk}^{(i)}, \vec{a}^{(i)})_{i \in \{0,1\}}) \leftarrow \mathcal{A}(\text{pp})$ $b \xleftarrow{\$} \{0, 1\}$ for $i \in \{0, 1\}$: $(\text{creq}^{(i)}, \text{st}^{(i)}) \leftarrow \text{Obt}_1(\text{ipk}, \text{vpk}^{(i)}, \vec{a}^{(i)}, \phi)$ if $\neg \phi(\vec{a}^{(0)}) \vee \neg \phi(\vec{a}^{(1)}) \vee \perp \in \{\text{creq}^{(0)}, \text{creq}^{(1)}\}$: $b' \xleftarrow{\$} \{0, 1\}$ else : $(\text{bcred}^{(b)}, \text{bcred}^{(1-b)}) \leftarrow \mathcal{A}(\text{creq}^{(b)}, \text{creq}^{(1-b)})$ for $i \in \{0, 1\}$: $\text{cred}^{(i)} \leftarrow \text{Obt}_2(\text{st}^{(i)}, \text{bcred}^{(i)})$ if $\perp \in \{\text{cred}^{(0)}, \text{cred}^{(1)}\}$: $b' \leftarrow \mathcal{A}(\perp, \perp)$ else : $b' \leftarrow \mathcal{A}(\text{cred}^{(0)}, \text{cred}^{(1)})$ return $b = b'$

Fig. 5. Issuance Anonymity Game for mKVAC.

beyond what is revealed through ϕ , it also ensures that the verifier's identity vpk remains hidden. The latter is important to not trade our increased flexibility of supporting multiple verifiers with reduced privacy for the user. In fact, without such anonymity guarantees, supporting multiple verifiers would be trivial: the issuer could simply use individual key pairs per verifier.

Figure 5 presents our indistinguishability-based issuance anonymity game where the issuers and verifiers are fully malicious. The adversary outputs an issuer public key ipk , a predicate ϕ , and two tuples $(\text{vpk}^{(i)}, \vec{a}^{(i)})$ for $i \in \{0, 1\}$. The challenger computes the corresponding credential requests creq values and runs the issuance protocol on the two inputs in a random order determined by a challenge bit b . The resulting credentials are returned to the adversary in the correct order. The adversary wins if it successfully guesses b , implying that it has linked credential requests to their associated attributes or verifier keys.

To prevent trivial wins, both attribute vectors must satisfy the predicate ϕ , and neither credential requests nor credentials may fail. Note that in our mKVAC setting, credential requests may fail if a verifier public key is malformed. We prevent the trivial wins arising from invalid attribute vectors or credential requests by simulating the adversary's guess on the challenge bit as a random guess. Furthermore, trivial wins due to invalid credentials are mitigated by not sending any of the credentials to the adversary if any of the credentials is invalid.

Definition 5. An mKVAC scheme is issuance anonymous if for all PPT adversaries $\mathcal{A}$, $\text{Adv}_{\mathcal{A}, \text{mKVAC}}^{\text{IAnon}}(\lambda) := \left| \Pr \left[\text{Exp}_{\mathcal{A}, \text{mKVAC}}^{\text{IAnon}}(\lambda) = \text{true} \right] - 1/2 \right| \leq \text{negl}(\lambda)$.

Remarks. Our explicit notion of issuance anonymity is stronger than previous models. Interestingly, prior works [Orr25, CAHLT25, CKL⁺14] model this property within their presentation anonymity game, typically through simulator-based definitions. In those definitions, a simulator interacts with the adversary

$\text{Exp}_{\mathcal{A}, \text{mKVAC}, \text{DenS}}^{\text{Den}}(\lambda)$	$\mathcal{O}\text{Chall}(\text{ctx}, \phi)$
$b \xleftarrow{\$} \{0, 1\}, \text{pp} \leftarrow \text{Setup}(\lambda, n)$	require $\phi(\vec{a}) = \text{true}$
$(\text{isk}, \text{ipk}) \leftarrow \text{IKg}(\text{pp}), (\text{vsk}, \text{vpk}) \leftarrow \text{VKg}(\text{isk})$	if $b = 0$:
$(\vec{a}, \phi) \leftarrow \mathcal{A}(\text{isk}, \text{ipk}, \text{vsk}, \text{vpk})$	$\text{pres} \leftarrow \text{Present}(\text{ipk}, \text{vpk}, \text{cred}, \text{ctx}, \phi)$
$(\text{creq}, \text{st}) \leftarrow \text{Obt}_1(\text{ipk}, \text{vpk}, \vec{a}, \phi)$	if $b = 1$:
$\text{bcred} \leftarrow \mathcal{A}(\text{creq})$	$\text{pres} \leftarrow \text{DenS}(\text{ipk}, \text{vsk}, \text{ctx}, \phi)$
$\text{cred} \leftarrow \text{Obt}_2(\text{st}, \text{bcred})$	return pres
if $\text{cred} = \perp \vee \neg \phi(\vec{a})$: $b' \xleftarrow{\$} \{0, 1\}$	
else : $b' \leftarrow \mathcal{A}^{\mathcal{O}\text{Chall}}(\text{cred})$	
return $b = b'$	

Fig. 6. Deniability Game for mKVAC.

using only public inputs — the issuer key and the predicate — but does not output a valid credential. Thus, in contrast to our definition, issuance anonymity therein is ensured only as long as credentials remain uncompromised.

3.5 Deniability

Our final security property is deniability, ensuring that a malicious verifier cannot convince a third party that a presentation originates from a legitimate credential held by a user. Deniability is especially desirable in identity schemes, where certified attribute presentations could otherwise enable data markets. In such markets, verifiers might sell the attested information they receive. We leverage the keyed-verification setting, which allows us to express that any presentation could be generated either via a real credential or simply by the verifier itself.

Our formal game, given in Figure 6, captures the deniability aspect through an indistinguishability argument: an honestly generated presentation from a proper credential cred is indistinguishable from a simulated one. The simulator DenS only gets the public presentation information ϕ, ctx as well as the verifier's secret key vsk as inputs. The adversary is allowed to query the challenge oracle multiple times, which either returns a proper presentation (if $b = 0$) based on an honestly generated cred or a simulated presentation via DenS ($b = 1$). The task of the adversary is to determine the randomly chosen challenge bit b . We again give the adversary strong power: $\mathcal{A}$ knows both isk, vsk , can choose the users' attributes $\vec{a}$ and predicate ϕ , and also learns the issued credential cred .

Definition 6. An mKVAC is deniable for a simulator DenS if for all PPT adversaries $\mathcal{A}$, $\text{Adv}_{\mathcal{A}, \text{mKVAC}, \text{DenS}}^{\text{Den}}(\lambda) := \left| \Pr \left[\text{Exp}_{\mathcal{A}, \text{mKVAC}, \text{DenS}}^{\text{Den}}(\lambda) = \text{true} \right] - 1/2 \right| \leq \text{negl}(\lambda)$.

4 Symmetric Anonymous Group Element Authenticator

As discussed in Section 1.2, our $\text{mKVAC}_{\text{CMZ}}$ construction relies on a *symmetric anonymous group element authenticator* scheme (SAGA) to ensure the integrity

of $\text{mKVAC}_{\text{CMZ}}$ verifier keys when issuing a credential. We can think of SAGA as a KVAC-like primitive that authenticates group elements, tailored towards efficient use within our $\text{mKVAC}_{\text{CMZ}}$ construction later (Section 5). For this, the syntax of SAGA requires SAGA presentations to be in a specific form, which our $\text{mKVAC}_{\text{CMZ}}$ relies on.

Assume the SAGA issuer has generated a key pair and handed a MAC tag τ on a message vector $\vec{M} \in \mathbb{G}^\ell$ to the user. With this set up, the user can present the tag on $\vec{M}$ to the issuer (acting as a verifier) in a KVAC-like manner. The special form of this presentation required for SAGA is as follows.

- The user reveals a blinded version $C_j := M_j + \xi_j G_j$ of messages M_j to the verifier, where $\xi_j \in \mathbb{Z}_p$ are secret random blinding terms and $G_j \in \mathbb{G}$ are fixed public elements.
- The user proves (in ZK) a statement PresWitVf about the blinding terms ξ_j (and that some policy on $M_j = C_j - \xi_j G_j$ holds for these ξ_j).
- The verifier executes a secret key-dependent check PresKeyedVf on $\vec{C}$.

The first bullet point is crucial for our use of SAGA within $\text{mKVAC}_{\text{CMZ}}$: it allows us to treat the $C_j = M_j + \xi_j G_j$ values as a commitment basis in the ZK proof used in the $\text{mKVAC}_{\text{CMZ}}$ issuance protocol (where $\vec{M}$ is the authenticated intended commitment basis and the $\xi_j G_j$ blinding terms are shown to be benign error terms).

We note that while this SAGA presentation structure is tailored for use within our $\text{mKVAC}_{\text{CMZ}}$ construction, we expect it to expose just enough structure to enable highly optimized use in many other types of constructions.

Definition 7 (Symmetric anonymous group element authenticator).

A symmetric anonymous group element authenticator scheme SAGA for group generator GGen is specified by a tuple of PPT algorithms listed below:

- $\text{Setup}(\lambda, \ell, \mathbb{G}, p, G) \rightarrow (\text{pp}_{\text{SAGA}}, \vec{G} \in \mathbb{G}^\ell, \vec{td} \in \mathbb{Z}_p^\ell)$: On input a security parameter λ in its unary representation and a message length parameter ℓ , outputs public parameters pp_{SAGA} , which are assumed as implicit inputs to the following algorithms. It also outputs (secret) discrete logarithms td_j and corresponding bases $G_j = td_j G$.
- $\text{Kg}(\text{pp}_{\text{SAGA}}) \rightarrow (\text{sk}_{\text{SAGA}}, \text{pk}_{\text{SAGA}})$: On input the public parameters pp_{SAGA} , outputs a secret key sk_{SAGA} and a corresponding public key pk_{SAGA} .
- $\text{MAC}(\text{sk}_{\text{SAGA}}, \vec{M} \in \mathbb{G}^\ell) \rightarrow \tau$: On input a secret key sk_{SAGA} and message vector $\vec{M} \in \mathbb{G}^\ell$, outputs tag τ .
- $\text{MACVerify}(\text{sk}_{\text{SAGA}}, \tau, \vec{M}) \rightarrow b \in \{\text{true}, \text{false}\}$: A deterministic verification algorithm that checks validity of τ on $\vec{M}$ with respect to sk_{SAGA} .
- $\text{Present}(\text{pk}_{\text{SAGA}}, \tau, \vec{M}) \rightarrow (\text{pres}_{\text{SAGA}}, \vec{C} \in \mathbb{G}^\ell, \vec{\xi} \in \mathbb{Z}_p^\ell, \text{wit}_{\text{SAGA}}) / \perp$: On input a public key pk_{SAGA} , a tag τ , and messages $\vec{M}$, it either outputs $\perp$ if the tag is invalid, or else produces public presentation data $\text{pres}_{\text{SAGA}}$, public blinded messages $C_j := M_j + \xi_j G_j$, and private witness data wit_{SAGA} needed for later verification.

$\text{Exp}_{\text{SAGA}, \mathcal{A}}^{\text{UF-CoMA}}(\lambda)$	$\mathcal{OMAC}(\vec{\alpha} \in \mathbb{Z}_p^\ell)$
$\mathcal{Q} := \emptyset, (\mathbb{G}, p, G) \leftarrow \text{GGen}(\lambda)$	$\vec{M} := \vec{\alpha} \cdot G$
$(\text{pp}_{\text{SAGA}}, \vec{G}, \vec{td}) \leftarrow \text{Setup}(\lambda, \ell, \mathbb{G}, p, G)$	$\mathcal{Q} \leftarrow \mathcal{Q} \cup \{\vec{M}\}$
$(\text{sk}_{\text{SAGA}}, \text{pk}_{\text{SAGA}}) \leftarrow \text{Kg}(\text{pp}_{\text{SAGA}})$	return $\text{MAC}(\text{sk}_{\text{SAGA}}, \vec{M})$
$(\text{pres}_{\text{SAGA}}, \text{wit}_{\text{SAGA}}, \vec{C}, \vec{\xi}) \leftarrow \mathcal{A}^{\mathcal{O}}(\text{pp}_{\text{SAGA}}, \text{pk}_{\text{SAGA}})$	$\mathcal{OPresKeyedVf}(\text{pres}'_{\text{SAGA}}, \text{wit}'_{\text{SAGA}}, \vec{C}', \vec{\xi})$
$\vec{M} := \vec{C} - \vec{\xi} \circ \vec{G}$	require $\text{PresWitVf}^{\text{pres}_{\text{SAGA}}, \vec{C}}(\text{wit}_{\text{SAGA}}, \vec{\xi})$
return $\left(\begin{array}{l} \text{PresKeyedVf}(\text{sk}_{\text{SAGA}}, \text{pres}_{\text{SAGA}}, \vec{C}) \\ \wedge \text{PresWitVf}^{\text{pres}_{\text{SAGA}}, \vec{C}}(\text{wit}_{\text{SAGA}}, \vec{\xi}) \\ \wedge \vec{M} \notin \mathcal{Q} \end{array} \right)$	return $\text{PresKeyedVf}(\text{sk}_{\text{SAGA}}, \text{pres}'_{\text{SAGA}}, \vec{C}')$

Fig. 7. Presentation unforgeability game for SAGA. The adversary has access to $\mathcal{OMAC}$ and $\mathcal{OPresKeyedVf}$ oracles.

$\text{PresWitVf}^{\text{pres}_{\text{SAGA}}, \vec{C}}(\text{wit}_{\text{SAGA}}, \vec{\xi}) \rightarrow b \in \{\text{true}, \text{false}\}$: A deterministic predicate, parameterized by the public values $(\text{pres}_{\text{SAGA}}, \vec{C})$, that determines the validity of the private values $(\text{wit}_{\text{SAGA}}, \vec{\xi})$ of a presentation.

$\text{PresKeyedVf}(\text{sk}_{\text{SAGA}}, \text{pres}_{\text{SAGA}}, \vec{C}) \rightarrow b \in \{\text{true}, \text{false}\}$: Given the secret key sk_{SAGA} , checks that the public presentation values $(\text{pres}_{\text{SAGA}}, \vec{C})$ are valid.

We provide the correctness definition of SAGA in Appendix H. Note that for the sake of our $\text{mKVAC}_{\text{CMZ}}$ construction, we prescribe a specific structure for SAGA messages ($\vec{M} \in \mathbb{G}^\ell$) and their blinded presentation versions ($\vec{C} = \vec{M} + \vec{\xi} \circ \vec{G} \in \mathbb{G}^\ell$ for $\vec{\xi} \in \mathbb{Z}_p^\ell$). However, we do not prescribe distributions for $\vec{\xi}$ or $\vec{G}$, or any specific forms for sk_{SAGA} , pk_{SAGA} , τ , $\text{pres}_{\text{SAGA}}$, etc. These are up to the specific SAGA instantiation to specify.

In practice, to use a SAGA as a KVAC, the presentation phase works as follows: The user runs **Present** to obtain $\text{pres}_{\text{SAGA}}, \vec{C}, \vec{\xi}, \text{wit}_{\text{SAGA}}$. The user sends $\text{pres}_{\text{SAGA}}, \vec{C}$ to the issuer/verifier and keeps $\vec{\xi}, \text{wit}_{\text{SAGA}}$ secret. The verifier checks that $\text{PresKeyedVf}(\text{sk}_{\text{SAGA}}, \text{pres}_{\text{SAGA}}, \vec{C})$ holds. The user proves in zero-knowledge that $\text{PresWitVf}^{\text{pres}_{\text{SAGA}}, \vec{C}}(\text{wit}_{\text{SAGA}}, \vec{\xi})$ holds, without revealing $\text{wit}_{\text{SAGA}}, \vec{\xi}$. The process above only proves that the user is in possession of a tag τ on *completely hidden* messages $\vec{M}$. To prove some (authentication) policy about $\vec{M}$, the user adds statements about the hidden unblinded messages $\vec{M} = \vec{C} - \vec{\xi} \circ \vec{G}$ (for the same $\vec{\xi}$ that succeeded that PresWitVf check) to the zero-knowledge proof.

4.1 Security Properties

We now formally define the security properties required of a SAGA scheme. Intuitively, we expect (1) unforgeability: it should be hard to produce a valid presentation (public and secret parts) for messages that have not been signed by the issuer, and (2) privacy: one cannot distinguish the presentations (the public parts) of two different tags.

$\text{Exp}_{\text{SAGA}, \mathcal{A}}^{\text{PRIV-}b}(\lambda) \text{ for } b \in \{0, 1\}$
$(\mathbb{G}, p, G) \leftarrow [\text{GGen}(\lambda)], (\text{pp}_{\text{SAGA}}, \vec{G}, \vec{td}) \leftarrow \text{Setup}(\lambda, \ell, \mathbb{G}, p, G)$
$(\text{pk}_{\text{SAGA}}, \tau^{(0)}, \vec{M}^{(0)}, \tau^{(1)}, \vec{M}^{(1)}) \leftarrow \mathcal{A}(\text{pp}_{\text{SAGA}})$
for $i \in \{0, 1\}$: $(\text{pres}_{\text{SAGA}}^{(i)}, \vec{C}^{(i)}, \vec{\xi}^{(i)}, \text{wit}_{\text{SAGA}}^{(i)}) \leftarrow \text{Present}(\text{pk}_{\text{SAGA}}, \tau^{(i)}, \vec{M}^{(i)})$
if $\text{pres}_{\text{SAGA}}^{(0)} = \perp \vee \text{pres}_{\text{SAGA}}^{(1)} = \perp$: return 0 // Rule out trivial win
return $\mathcal{A}(\text{pres}_{\text{SAGA}}^{(b)}, \vec{C}^{(b)})$

Fig. 8. Privacy game for SAGA.

Unforgeability. We define unforgeability under chosen open message attacks (UF-CoMA), given in Figure 7, where “open message” means that the discrete logarithms of messages queried to $\mathcal{OMAC}$ must be known [FG18]. The adversary wins if it can produce a valid presentation (public and secret parts) such that the unblinded messages $M_j := C_j - \xi_j G_j$ have not been queried to $\mathcal{OMAC}$ before. Looking ahead, we only consider open message attacks as this will be sufficient to ensure the security of our mKVAC construction in Section 5.

Definition 8 (SAGA unforgeability under Chosen open Message Attack). A SAGA scheme satisfies (presentation) unforgeability under chosen open message attack (UF-CoMA) if for all PPT adversaries $\mathcal{A}$, the advantage

$$\text{Adv}_{\text{SAGA}, \mathcal{A}}^{\text{UF-CoMA}}(\lambda) := \Pr[\text{Exp}_{\text{SAGA}, \mathcal{A}}^{\text{UF-CoMA}}(\lambda) = \text{true}] \leq \text{negl}(\lambda).$$

Privacy. The adversary plays the role of a malicious issuer/verifier in the privacy game, presented in Figure 8. The adversary outputs two message vectors and corresponding tags and is tasked to guess which tag the public parts ($\text{pres}_{\text{SAGA}}, \vec{C}$) of the challenge presentation originates from. To prevent trivial wins, the game enforces that both $\text{pres}_{\text{SAGA}}^{(b)}$ values must be valid ones — not $\perp$.

Definition 9 (Privacy). A SAGA scheme satisfies privacy if for all probabilistic polynomial-time adversaries $\mathcal{A}$, the advantage

$$\text{Adv}_{\text{SAGA}, \mathcal{A}}^{\text{priv}}(\lambda) := \left| \Pr[\text{Exp}_{\text{SAGA}, \mathcal{A}}^{\text{PRIV-0}}(\lambda) = 1] - \Pr[\text{Exp}_{\text{SAGA}, \mathcal{A}}^{\text{PRIV-1}}(\lambda) = 1] \right| \leq \text{negl}(\lambda).$$

Note that Definition 9 only guarantees privacy with respect to public presentation data $\text{pres}_{\text{SAGA}}, \vec{C}$, not with respect to the *secret* presentation data $\text{wit}_{\text{SAGA}}, \vec{\xi}$. Indeed, $\vec{C}, \vec{\xi}$ together fully reveal $\vec{M}$. The user should keep the secret presentation data hidden, e.g., in zero-knowledge.

4.2 Our SAGA Instantiations

We propose two instantiations of our SAGA. The first follows CPZ KVACs [CPZ20], which provide MACs on group elements together with a KVAC-style protocol to present these MACs in a privacy-preserving manner. This presentation protocol can be decomposed into the two parts required by our SAGA definition in a natural way. We describe this in Appendix L. Note that in the

CPZ instantiation, the values $C_j := M_j + \xi G_j$ use the same blinding factor ξ for all messages M_j . This makes C_j perfectly binding and computationally hiding, assuming DDH is hard. As a consequence, if a future quantum computer breaks the discrete logarithms of G_j , then all past **SAGA** presentations retroactively lose their privacy. For this reason, we introduce a second **SAGA** instantiation, **BBSSAGA**, which we focus in this paper on due to its better privacy properties. This instantiation is inspired by BBS signatures [BBS04] and its presentations stay retroactively private even if discrete logarithms are broken in the future. In particular, the $C_j := M_j + \xi_j G_j$ are perfectly hiding.

BBS signatures have the message space $\mathbb{Z}_p$ originally. More specifically their signatures on a message $m_1 \in \mathbb{Z}_p$ are in the form of $\sigma := (A := (x + d)^{-1}(G_0 + m_1 Y_1), d)$ for a key pair ($\text{sk} := x, \text{pk} := (X := xG, Y_1)$) and $d \xleftarrow{\$} \mathbb{Z}_p$. The original signatures scheme relies on pairing-friendly curves to obtain public verifiability, but BBS MACs on pairing-free prime-order groups have already been proposed before [BBDT16, Orr25], but only for scalar messages $m_i \in \mathbb{Z}_p$. Our **BBSSAGA** scheme is adapted to support group element messages $M_i \in \mathbb{G}$, which turns out to be a nontrivial change in the security proofs, even restricting ourselves to unforgeability under open-message attacks (cf. Definition 8). Statistical privacy is achieved similarly to prior BBS-based signatures and MACs.

BBSSAGA Construction. We describe our **BBSSAGA** scheme in Figure 9. A **BBSSAGA** secret key consists of the MAC key x as well as an additional scalar y_j for each message in the message vector. The additional scalars are used to sign and verify tags on group elements. The public key of **BBSSAGA** contains $X := xG$ and $Y_j := y_j G_j$ terms to allow efficient presentation of **BBSSAGA** tags. In contrast to BBS MAC public keys, the Y_j terms are computed in distinct bases which is crucial for the unforgeability of **BBSSAGA** (see Appendix I).

A **BBSSAGA** tag computes A term by scalar-multiplying the commitment-like $G_0 + \vec{y} \cdot \vec{M}$ term with $(x + d)^{-1}$. Additional to (A, d) , a **BBSSAGA** tag contains a proof π_{BBSSAGA} of well-formedness of (A, d) according to $\mathcal{R}_{\text{BBSSAGA}}$ below. The proof of well-formedness ensures the privacy of presentations against malicious signers.

$$\mathcal{R}_{\text{BBSSAGA}} := \left\{ (x, \vec{y}) : X = xG \wedge \vec{Y} = \vec{y} \circ \vec{G} \wedge (x + d)A = G_0 + \vec{y} \cdot \vec{M} \right\}$$

Note that if the same tag is presented multiple times, re-verification of π_{BBSSAGA} is unnecessary: once validated, the core tag (A, d) can be stored and reused. For instance, in **mKVAC**, when a user obtains a second credential from the same verifier or updates an existing one, re-verifying this proof can be skipped.

The presentation of **BBSSAGA** follows the prescriptions of the **SAGA** modeling. Compared to BBS MAC presentations [Orr25], several changes are made to handle presenting group elements. Notice that **BBSSAGA** tag verification (**MACVerify**) can be re-structured as $xA = G_0 - dA + \vec{y} \cdot \vec{M}$. This gives a nicely structured equation where a user can blind $A, \vec{M}$ as $C_A, \vec{C}$ as done in the **Present** algorithm. The keyed verification **PresKeyedVf** then checks the tag on these blinded values together with the term T which cancels out the blinding

$\text{Setup}(\lambda, \ell, \mathbb{G}, p, G)$ $G_0 \xleftarrow{\$} \mathbb{G}, \vec{td} \xleftarrow{\$} \mathbb{Z}_p^\ell, \vec{G} := \vec{td}G$ $\text{pp}_{\text{SAGA}} := (\mathbb{G}, p, G, G_0, \vec{G})$ return $(\text{pp}_{\text{SAGA}}, \vec{G}, \vec{td})$	$\text{MAC}(\text{sk}_{\text{SAGA}}, \vec{M})$ parse sk_{SAGA} as $(x, \vec{y})$ $d \xleftarrow{\$} \mathbb{Z}_p, A := (x + d)^{-1}(G_0 + \vec{y} \cdot \vec{M})$ $\mathbb{X}_{\text{BBSSAGA}} := (A, d, \vec{M}), \mathbb{W}_{\text{BBSSAGA}} := (x, \vec{y})$ $\pi_{\text{BBSSAGA}} \leftarrow \text{NIP}_{\text{BBSSAGA}}.\text{Prove}(\mathbb{X}_{\text{BBSSAGA}}, \mathbb{W}_{\text{BBSSAGA}})$ return $\tau := (A, d, \pi_{\text{BBSSAGA}})$
$\text{Kg}(\text{pp}_{\text{SAGA}})$ $x \xleftarrow{\$} \mathbb{Z}_p, \vec{y} \xleftarrow{\$} \mathbb{Z}_p^\ell$ $X := xG, \vec{Y} := \vec{y} \circ \vec{G}$ $\text{sk}_{\text{SAGA}} := (x, \vec{y}), \text{pk}_{\text{SAGA}} := (X, \vec{Y})$ return $(\text{sk}_{\text{SAGA}}, \text{pk}_{\text{SAGA}})$	$\text{Present}(\text{pk}_{\text{SAGA}}, \tau, \vec{M})$ parse $(\text{pk}_{\text{SAGA}}, \tau)$ as $((X, \vec{Y}), (A, d, \pi_{\text{BBSSAGA}}))$ $\mathbb{X}_{\text{BBSSAGA}} := (A, d, \vec{M})$ require $\text{NIP}_{\text{BBSSAGA}}.\text{Vf}(\mathbb{X}_{\text{BBSSAGA}}, \pi_{\text{BBSSAGA}})$ $r \xleftarrow{\$} \mathbb{Z}_p, \vec{\xi} \xleftarrow{\$} \mathbb{Z}_p^\ell$ $\vec{C} := \vec{M} + \vec{\xi} \circ \vec{G}, C_A := A + rG$ $T := rX - dC_A + drG - \vec{\xi} \cdot \vec{Y}$ return $(\text{pres}_{\text{SAGA}} := (C_A, T), \vec{C}, \vec{\xi}, \text{wits}_{\text{SAGA}} := (r, d))$
$\text{MACVerify}(\text{sk}_{\text{SAGA}}, \tau, \vec{M})$ parse sk_{SAGA} as $(x, \vec{y})$ parse τ as $(A, d, \pi_{\text{BBSSAGA}})$ return $(x + d)A = G_0 + \vec{y} \cdot \vec{M}$	$\text{PresKeyedVf}(\text{sk}_{\text{SAGA}}, \text{pres}_{\text{SAGA}}, \vec{C})$ parse sk_{SAGA} as $(x, \vec{y})$ parse $\text{pres}_{\text{SAGA}}$ as (C_A, T) return $x C_A = G_0 + \vec{y} \cdot \vec{C} + T$
	$\text{PresWitVf}^{\text{pres}_{\text{SAGA}}, \vec{C}}(\text{wits}_{\text{SAGA}}, \vec{\xi})$ parse $(\text{pres}_{\text{SAGA}}, \text{wits}_{\text{SAGA}})$ as $((C_A, T), (r, d))$ return $T = rX - dC_A + drG - \vec{\xi} \cdot \vec{Y}$

Fig. 9. BBS-based instantiation of SAGA

terms in $C_A, \vec{C}$. Correspondingly, the witness verification algorithm PresWitVf ensures that T is well-formed with respect to the witness $\text{wits}_{\text{SAGA}}$ and $\vec{\xi}$.

Security. In the following, we state the security theorems for BBSSAGA and provide proof sketches. The full proofs are deferred to Appendix K. To prove security of this instantiation, we first establish that the MAC component of BBSSAGA is unforgeable (we formally define security of the MAC component in Appendix J). Note that while BBSSAGA bears strong resemblance to BBS signatures, which can be proven secure under the q -SDH assumption, its message space is group elements instead of scalars. This complicates the security proof. For this reason, we offer a proof in the generic group model (GGM). We further note that we do not identify immediate attacks against the unforgeability of BBSSAGA under plain chosen-message attacks, and leave a full security analysis of it as an algebraic MAC to future work.

Theorem 1. *If $\text{NIP}_{\text{BBSSAGA}}$ is zero-knowledge, then BBSSAGA (Figure 9) is MAC-unforgeable under chosen open message attacks (Definition 21) in GGM.*

Proof (Sketch). Based on the zero-knowledge property of $\text{NIP}_{\text{BBSSAGA}}$, the proofs π_{BBSSAGA} can be simulated and therefore reveal no information about the secret keys. If all d values used to answer MAC queries are distinct, then by a standard

GGM argument the only way for the adversary to produce a valid message-tag pair is to replay one of the pairs obtained from the MAC oracle. Since the probability that two MAC queries use the same d is negligible, it follows that our BBSSAGA construction is MAC-unforgeable in the GGM.

Building on the security of the MAC component, we then show that the presentation protocol part of BBSSAGA cannot be forged either.

Theorem 2. *If BBSSAGA (Figure 9) is MAC-unforgeable under chosen open message attacks (Definition 21), then BBSSAGA is unforgeable under chosen open message attack (Definition 8).*

Proof (Sketch). Both for the final forgery and the verification oracle queries, we are given the secret blinding terms $\text{wits}_{\text{SAGA}}$ and $\vec{\xi}$ and can use them to obtain a message tag pair by cancelling out the blinding terms from the presentation $\text{pres}_{\text{SAGA}}$. This gives us a straightforward reduction to MAC unforgeability.

Finally, we establish privacy of BBSSAGA.

Theorem 3. *BBSSAGA (Figure 9) satisfies the privacy property (Definition 9) if $\text{NIP}_{\text{BBSSAGA}}$ is sound.*

Proof (Sketch). Since $(\text{wits}_{\text{SAGA}}, \vec{\xi})$ are uniformly random, $(C_A, \vec{C})$ are uniformly random group elements. By the soundness property of $\text{NIP}_{\text{BBSSAGA}}$, the message-tag pairs obtained from the adversary are guaranteed to be well formed. Consequently, the value T computed by $\text{pres}_{\text{SAGA}}$ must verify correctly under the unique secret keys $(x, \vec{y})$, namely,

$$T = xC_A - G_0 - \sum_{j \in [\ell]} y_j C_j.$$

Therefore, regardless of whether $b = 0$ or $b = 1$, the adversary either receives 0 — if at least one of its message-tag pairs is not well formed — or it receives $(C_A, \vec{C})$, which are uniformly random group elements, together with a value T that is uniquely determined by the random group elements $(C_A, \vec{C})$ and the unique values $(x, \vec{y})$. Hence, the output revealed to the adversary leaks no information about the bit b .

5 Our mKVAC Construction $\text{mKVAC}_{\text{CMZ}}$

In this section, we present our mKVAC construction $\text{mKVAC}_{\text{CMZ}}$. The main challenge of our construction is to construct a CMZ-style blind issuance protocol that hides which verifier key the issued credential is valid under. We now develop our solution step-by-step.

CMZ Credentials. We start by recalling basic CMZ (keyed-verification) credentials [CMZ14, Orr25]. A credential is an algebraic MAC of the form

$$(uG, (x_0 + \vec{a} \cdot \vec{x})uG)$$

for messages $\vec{a} \in \mathbb{Z}_p^n$, random $u \xleftarrow{\$} \mathbb{Z}_p^*$, and secret keys $(x_0, \vec{x})$ (cf. Figure 2). The public key enabling blind issuance is $\vec{X} := (x_j G)_{j=1}^n$. Crucially, $x_0 G$ must remain secret, as it allows computing valid credentials for arbitrary messages.

To issue a CMZ credential (in the plain, known-verifier setting [Orr25]), the user commits to their attributes $\vec{a}$ as $C_a := \vec{a} \cdot \vec{X} + sG$ using random $s \xleftarrow{\$} \mathbb{Z}_p$ and proves in zero-knowledge that the commitment is well-formed and that the attributes fulfill the issuance policy $\phi(\vec{a})$. Note that C_a is structurally close to a valid credential that is missing the secret key x_0 term and the randomization term u . The issuer adds the missing $x_0 G$ term and applies randomization with u , i.e. the issuer returns $(uG, u(x_0 G + C_a))$. The issuer proves that they have done this correctly (preventing key parameter consistency attacks on privacy).

Removing the CMZ key from the issuer. As a first step towards our solution, we will enable the issuer to issue credentials without directly using the verifier's CMZ key. For this, the issuer generates decryption key e and publish the encryption key $E := eG$. We extend the verifier's CMZ public key with an ElGamal encryption $X_0 := x_0 G + \nu E, Z_0 := \nu G$ of the verifier's secret $x_0 G$. Hence the verifier's public key is now $(\vec{X}, X_0, Z_0)$. With this change, the user can hand the desired verifier's public key to the issuer, who can then decrypt (X_0, Z_0) and use the resulting $x_0 G$ to issue a credential as above. Essentially, e becomes the issuance key.

Authenticating the verifier key. A crucial weakness of the above protocol is that user may maliciously modify the verifier's public key before handing it to the issuer. This applies in particular to the verifier's commitment basis $\vec{X}$ for the commitment C_a . For example, the user could instead supply basis $(2X_1, \dots, 2X_n)$, commit to attributes a as $C_a := \sum_{j=1}^n a_j(2X_j) + sG$, prove that a_j fulfill the issuance policy ϕ , and then reinterpret the commitment as $C_a = \sum_{j=1}^n (2a_j)X_j + sG$ and effectively receive a credential for attributes $a' := (2a_1, \dots, 2a_n)$, which may not fulfill the issuance policy. To prevent such attacks on the integrity of verifier keys, the issuer will compute a SAGA tag τ on the verifier's key vpk and publish it alongside vpk . The user hands the desired vpk and its tag τ to the issuer, who would check τ and then run the rest of the issuance protocol assuming vpk is a valid key, preventing the attack sketched above. With this modification, the issuance protocol ensures unforgeability, but reveals the verifier key to the issuer in plain.

Hiding the Verifier. In this step, we hide all verifier-specific data from the issuer. We omit some details, which we discuss later. In order to hide all verifier-specific values from the previous steps, we need to hide from the issuer the ElGamal ciphertext (X_0, Z_0) , the commitment basis $\vec{X}$, and the tag τ on $(\vec{X}, X_0, Z_0)$. Instead of sending the tag τ to the issuer directly, the user now executes a SAGA presentation, which safely exposes a blinded authenticated version $\vec{C}$ of the verifier key. Instead of sending the ElGamal ciphertext (X_0, Z_0) , the user sends a re-randomized ciphertext $(\bar{X}_0, \bar{Z}_0)$ of a randomized plaintext $x_0 G + \bar{x}_0 G$, and proves in zero-knowledge that it is well-formed (with respect to the SAGA-blinded ciphertext C_{n+1}, C_{n+2}). Instead of sending the commitment basis $\vec{X}$, the user uses the SAGA's $C_1, \dots, C_n$ to prove in zero-knowledge that his attribute commitment C_a is well-formed. Overall, the verifier identity is now hidden from

mKVAC_{CMZ}.Setup (λ, n)	mKVAC_{CMZ}.IKg (pp)
$(\mathbb{G}, p, G) \leftarrow \text{GGen}(\lambda), H \xleftarrow{\$} \mathbb{G}^*$	$(\text{sk}_{\text{SAGA}}, \text{pk}_{\text{SAGA}}) \leftarrow \text{SAGA.Kg}(\text{pp}_{\text{SAGA}})$
$(\text{pp}_{\text{SAGA}}, \vec{G}, \cdot) \leftarrow \text{SAGA.Setup}(\lambda, n+2, \mathbb{G}, p, G)$	$e \xleftarrow{\$} \mathbb{Z}_p, E := eG$
return pp = $(\mathbb{G}, p, G, H, \text{pp}_{\text{SAGA}}, \vec{G})$	return (isk := $(\text{sk}_{\text{SAGA}}, e)$, ipk := $(\text{pk}_{\text{SAGA}}, E)$)
mKVAC_{CMZ}.VKg (isk, ipk)	
parse (isk, ipk) as $((\text{sk}_{\text{SAGA}}, e), (\text{pk}_{\text{SAGA}}, E))$	
$(\text{sk}_{\text{CMZ}} := (x_0, \vec{x}), \text{pk}_{\text{CMZ}} := \vec{X}) \leftarrow \text{CMZ.Kg}(\mathbb{G}, p, G), \nu \xleftarrow{\$} \mathbb{Z}_p, X_0 := x_0G + \nu E, Z_0 := \nu G$	
$\tau \leftarrow \text{SAGA.MAC}(\text{sk}_{\text{SAGA}}, (\vec{X}, X_0, Z_0))$	
return $(\text{vsk} := (x_0, \vec{x}, \vec{X}), \text{vpk} := (\vec{X}, X_0, Z_0, \tau))$	

Fig. 10. Setup and key generation algorithms of mKVAC_{CMZ}.

the issuer, but the issuer can still be sure that the user's request is well-formed w.r.t. a valid verifier key.

5.1 Construction of mKVAC_{CMZ}

mKVAC_{CMZ} is formally defined in Figures 10, 11, and 12. Below these procedures are explained in more detail.

Setup and Key Generation. The algorithms Setup, IKg, and VKg are described in Figure 10. Setup initializes the group, the SAGA public parameters, and a random group element H that will be used in the presentation protocol. As outlined above, the issuer's key pair consists of an ElGamal encryption key pair, used for credential issuance, and a SAGA key pair, used to compute SAGA tags on verifier keys and to validate the blinded verifier keys during issuance.

A verifier secret key is simply a CMZ secret key, syntactically extended with its corresponding CMZ public key, since verifiers need this public key when checking presentations. The verifier public key contains CMZ public key $\vec{X}$, the ElGamal encryption of x_0G to be later decrypted by the issuer, and a SAGA tag on these values, enabling the issuer to check integrity of a verifier key.

Issuance Protocol. The issuance protocol algorithms Obt₁, Iss, and Obt₂ are described in Figure 11. At its core, the issuance protocol proceeds as follows: The user commits to the desired attributes as $C_a := \vec{a} \cdot \vec{X} + \underline{s}G$ using the verifier's CMZ public key $\vec{X}$. The user furthermore sends a randomized ElGamal encryption $(\vec{X}_0, \vec{Z}_0)$ of $(x_0 + \underline{x}_0)G$ where x_0G is the real verifier's key, blinded by a random $\underline{x}_0$. The issuer decrypts $(x_0 + \underline{x}_0)G$ from the ElGamal ciphertext and uses it to complete C_a to a blinded credential with $(\vec{U}, \vec{V}) = (\vec{U}, (x_0 + \underline{x}_0 + \vec{a} \cdot \vec{x} + \underline{s})\vec{U})$. Finally, the user removes the error terms $\underline{x}_0$ and $\underline{s}$ from $(\vec{U}, \vec{V})$ to obtain the final MAC on $\vec{a}$. The rest of the protocol deals with checking that both parties properly adhere to these steps.

Well-Formedness of Credential Requests. The issuance of the blinded credential $(\vec{U}, \vec{V})$ only depends on $\vec{X}_0, \vec{Z}_0, C_a$, and the user proves the well-formedness of

$\text{mKVAC}_{\text{CMZ}}.\text{Obt}_1(\text{ipk}, \text{vpk}, \vec{a}, \phi)$ parse (ipk, vpk) as $((\text{pk}_{\text{SAGA}}, E), (\vec{X}, X_0, Z_0, \tau))$ $(\text{pres}_{\text{SAGA}}, \vec{C}, \vec{\xi}, \text{wits}_{\text{SAGA}}) \leftarrow \text{SAGA.Present}(\text{pk}_{\text{SAGA}}, \tau, (\vec{X}, X_0, Z_0))$ require SAGA.Present did not output $\perp$ $\bar{x}_0, \bar{\nu} \xleftarrow{\$} \mathbb{Z}_p, \bar{X}_0 := X_0 + \bar{x}_0 G + \bar{\nu} E, \bar{Z}_0 := Z_0 + \bar{\nu} G \parallel \text{decrypts to } \bar{X}_0 - e\bar{Z}_0 = (\bar{x}_0 + x_0)G$ $s \xleftarrow{\$} \mathbb{Z}_p, C_a := \vec{a} \cdot \vec{X} + sG \parallel \text{Commitment to attributes}$ $\pi_R \leftarrow \text{NIP}_R.\text{Prove}(\mathbb{x}_R := (\vec{C}, \bar{X}_0, \bar{Z}_0, C_a), \mathbb{w}_R := (s, \vec{a}, \vec{\xi}^T := \vec{a} \circ \vec{\xi}, \bar{x}_0, \bar{\nu}, \text{wits}_{\text{SAGA}}, \vec{\xi}))$ return $(\text{st} := (s, \bar{x}_0, \bar{X}_0, \bar{Z}_0, \vec{a}), \text{creq} := (\text{pres}_{\text{SAGA}}, \mathbb{x}_R, \pi_R))$ $\text{lss}(\text{isk}, \phi, \text{creq})$ parse $(\text{isk}, \text{ipk}, \text{creq})$ as $((\text{sk}_{\text{SAGA}}, e), (\text{pk}_{\text{SAGA}}, E), (\text{pres}_{\text{SAGA}}, \mathbb{x}_R := (\vec{C}, \bar{X}_0, \bar{Z}_0, C_a), \pi_R))$ require $\text{NIP}_R.\text{Vf}(\mathbb{x}_R, \pi_R) \wedge \text{SAGA.PresKeyedVf}(\text{isk}, \text{pres}_{\text{SAGA}}, \vec{C})$ $u \xleftarrow{\$} \mathbb{Z}_p^*, \bar{U} := uG, \bar{V} := u((\bar{X}_0 - e\bar{Z}_0) + C_a) \parallel = (x_0 + \bar{x}_0 + \vec{a} \cdot \vec{x} + \underline{s})\bar{U}$ $\pi_I \leftarrow \text{NIP}_I.\text{Prove}(\mathbb{x}_I := (E, \bar{U}, \bar{V}, \bar{X}_0, \bar{Z}_0, C_a), \mathbb{w}_I := (e, u))$ return $\text{bcred} := (\bar{U}, \bar{V}, \pi_I)$ $\text{Obt}_2(\text{st}, \text{bcred})$ parse $(\text{st}, \text{bcred})$ as $((s, \bar{x}_0, \bar{X}_0, \bar{Z}_0, \vec{a}), (\bar{U}, \bar{V}, \pi_I))$ require $\bar{U} \neq 0 \wedge \text{NIP}_I.\text{Vf}(\mathbb{x}_I := (E, \bar{U}, \bar{V}, \bar{X}_0, \bar{Z}_0, C_a), \pi_I)$ $\gamma \xleftarrow{\$} \mathbb{Z}_p^*, U := \gamma\bar{U}, V := \gamma(\bar{V} - (s + \bar{x}_0)\bar{U}) \parallel \text{Remove error terms } \bar{x}_0, \underline{s} \text{ from } \bar{V}$ return $\text{cred} := (U, V, \vec{a})$
--

Fig. 11. Credential issuance protocol of the $\text{mKVAC}_{\text{CMZ}}$.

these values according to NIP_R described below.

$$\begin{aligned}
& \text{NIP}_R := \left\{ \left(s, \vec{a}, \vec{\xi}^T := \vec{a} \circ \vec{\xi}, \bar{x}_0, \bar{\nu}, \text{wits}_{\text{SAGA}}, \vec{\xi} \right) : \right. \\
& \phi(\vec{a}) = \text{true} \parallel \text{Attributes fulfill policy} \\
& \wedge \bar{X}_0 = (C_{n+1} - \xi_{n+1}G_{n+1}) + \bar{x}_0G + \bar{\nu}E \parallel \text{Ciphertext valid} \\
& \wedge \bar{Z}_0 = (C_{n+2} - \xi_{n+2}G_{n+2}) + \bar{\nu}G \parallel \text{Ciphertext valid} \\
& \wedge C_a = sG + \left(\sum_{j \in [n]} a_j C_j \right) - \left(\sum_{j \in [n]} \xi'_j G_j \right) \parallel \text{Commitment valid} \\
& \wedge \text{PresWitVf}^{\text{pres}_{\text{SAGA}}, \vec{C}}(\text{wits}_{\text{SAGA}}, \vec{\xi}) \parallel \text{SAGA check} \left. \right\}
\end{aligned}$$

First, the user links the underlying $(\vec{X}, X_0, Z_0)$ to a legitimate vpk . This is done by relying on the SAGA presentation of tag τ from vpk . Specifically, SAGA presentation provides an authenticated vector $\vec{C} := (\vec{X}, X_0, Z_0) + \vec{\xi} \circ \vec{G}$ with blinding terms $\vec{\xi}$. The user proves that PresWitVf holds as part of NIP_R and the issuer checks that PresKeyedVf holds during the issuance, which completes the SAGA presentation verification. As $\vec{C}$ can be safely handed to the issuer, the user is

$\text{Present}(\text{ipk}, \text{vpk}, \text{cred}, \text{ctx}, \phi)$ $\text{parse}(\text{vpk}, \text{cred}) \text{ as } ((\vec{X}, X_0, Z_0, \tau), (U, V, \vec{a}))$ $\gamma \xleftarrow{\$} \mathbb{Z}_p^*, \tilde{\gamma} \xleftarrow{\$} \mathbb{Z}_p, \vec{\gamma} \xleftarrow{\$} \mathbb{Z}_p^n$ $(\tilde{U}, \tilde{V}) := (\gamma U, \gamma V), Z := \vec{\gamma} \cdot \vec{X} - \tilde{\gamma} H, C_V := \tilde{V} + \tilde{\gamma} H, \vec{C} := \vec{a} \cdot \tilde{U} + \vec{\gamma} \cdot G$ $\pi_S \leftarrow \text{NIP}_S.\text{Prove}(\mathbb{X}_S := (\vec{X}, \tilde{U}), \mathbb{W}_S := ((\tilde{\gamma}, \vec{a}, (\gamma_j)_{j=1}^n), \perp), \text{ctx}_{\text{NIP}} := (\text{vpk}, \text{ctx}))$ $\text{return pres} := (\tilde{U}, Z, C_V, \vec{C}, \pi_S)$ $\text{VfPres}(\text{vsk}, \text{ipk}, \text{pres}, \text{ctx}, \phi)$ $\text{parse}(\text{vsk}, \text{pres}) \text{ as } ((x_0, \vec{x}, \vec{X}), (\tilde{U}, Z, C_V, \vec{C}, \pi_S))$ $\text{return NIP}_S.\text{Vf}(\mathbb{X}_S := (\vec{X}, \tilde{U}), \pi_S, \text{ctx}_{\text{NIP}} := (\text{vpk}, \text{ctx}))$ $\wedge \tilde{U} \neq 0 \wedge (Z = x_0 \tilde{U} + \vec{x} \cdot \vec{C} - C_V)$
--

Fig. 12. Credential presentation and verification of $\text{mKVAC}_{\text{CMZ}}$.

then tasked to prove the well-formedness of $\vec{X}_0, \vec{Z}_0, C_a$ by relying on a linear relation with public bases and secret exponents as shown in NIP_R .

Note that NIP_R does not enforce the well-formedness of $\vec{\xi} := \vec{a} \circ \vec{\xi}$. This is a deliberate optimization: requiring $\xi_j = a_j \cdot \xi_j$ would introduce non-linear terms to be proven and impose extra complexity in NIP_R . In our security proof, we show that this omission is harmless by relying on the trapdoor $\vec{td}$ of SAGA.

Well-Formedness of Blinded Credentials. To prevent key parameter consistency attacks on user privacy, the issuer proves that NIP_I below holds. This relation shows that 1) $\tilde{U}$ and $\tilde{V}$ are consistent with their exponent u , and 2) $\tilde{V}$ is derived from the correct decryption of the ElGamal ciphertext $(\vec{X}_0, \vec{Z}_0)$.

$$\text{NIP}_I := \{(e, u) : \tilde{U} = uG \wedge E = eG \wedge \tilde{V} = u((\vec{X}_0 - e\vec{Z}_0) + C_a)\}$$

Credential Presentation. The credential presentation is shown in Figure 12. Because the issuance phase yields a valid CMZ KVAC, the presentation phase is almost identical to the CMZ KVAC's presentation [CMZ14]. The only significant difference is a modification of the signature-of-knowledge relation $\mathcal{R}_S$ to ensure deniability using the following well-known technique. Namely, the user proves the knowledge of either the vsk component x_1 such that $X_1 = x_1G$, or a valid CMZ credential for the predicate. This establishes deniability: a malicious verifier can create such a proof by only using x_1 from its secret key, without having to find valid attributes $\vec{a}$ fulfilling ϕ (which might be computationally hard).

$$\begin{aligned} \text{NIP}_S := \{ & ((\tilde{\gamma}, \vec{a}, \vec{\gamma}), x_1) : X_1 = x_1G \vee \text{ // Deniability} \\ & (Z = \vec{\gamma} \cdot \vec{X} - \tilde{\gamma} H \wedge \vec{C} = \vec{a} \cdot \tilde{U} + \vec{\gamma} \cdot G \wedge \phi(\vec{a}) = \text{true}) \} \end{aligned}$$

5.2 Security of $\text{mKVAC}_{\text{CMZ}}$

In this section, we state our theorems for the security of $\text{mKVAC}_{\text{CMZ}}$, and sketch the proofs of these theorems. Appendix G contains full proofs of these theorems.

Theorem 4. $\text{mKVAC}_{\text{CMZ}}$ scheme from Figures 10 to 12 is unforgeable if NIP_I is zero-knowledge, NIP_R is complete and straightline extractable, and NIP_S is complete, zero-knowledge, and rewinding-based simulation-extractable, DDH is hard, and SAGA and CMZ are correct and unforgeable.

Proof (Sketch). We set the presentation unforgeability extractor E to run the (straightline) extractor of NIP_R and output the extracted attributes. We then take several proof steps towards enabling a reduction to CMZ MAC unforgeability. First, the honest and corrupt credential issuance oracles are restricted to respond only to credential requests with valid vpk values by relying on SAGA unforgeability. This allows us to skip ElGamal decryption during issuance as we can look up x_0 of each extracted vpk and respond using that value. In turn, this allows us to simulate ElGamal ciphertexts in verifier keys as random values under the DDH assumption, thereby removing any involvement of the secret CMZ MAC key x_0^* from vpk^* . The proof then addresses two challenges: simulating honest presentation queries without the actual credentials, and handling error terms in blinded credentials for corrupt issuance without white-box access to vsk^* . These are resolved using the zero-knowledge properties of NIP_I and NIP_S , along with algebraic manipulations. In the final step, each corrupt issuance query is answered with a CMZ MAC computation, allowing us to build our reduction to CMZ MAC unforgeability. Notably, parts of the CMZ forgery are hidden within the π_S proof, which we extract from by carefully applying rewinding.

Theorem 5. $\text{mKVAC}_{\text{CMZ}}$ has presentation anonymity if NIP_I is sound and NIP_S is zero-knowledge.

Proof (Sketch). To simulate $\mathcal{O}\text{Chall}$ queries without using bit b : we compute π_S of pres via NIP_S zero-knowledge simulator, and simulate $(\tilde{U}, Z, C_V, \vec{C})$ of pres with random $\tilde{U}C_V, \vec{C}$ while Z is equivocated by using $(x_0G, \vec{X})$ of the designated verifier of the presentation. An obstacle arises in this strategy if the credential issuance produces an invalid credential without triggering Obt_2 to abort. Then simulating a valid presentation as above would become distinguishable. We prevent this case by relying on the soundness of NIP_I .

Theorem 6. $\text{mKVAC}_{\text{CMZ}}$ scheme from Figures 10 to 12 satisfies the issuance anonymity property if NIP_I is rewinding-based extractable, NIP_R is zero-knowledge, and the underlying SAGA scheme satisfies privacy.

Proof (Sketch). Our proof aims to simulate the issuance anonymity game of $\text{mKVAC}_{\text{CMZ}}$ by simulating the credential requests and credentials without using the bit b . To simulate credentials independently, we extract the ElGamal decryption key e and utilize it together with vpks to issue fresh credentials, which requires a rewinding-based argumentation.

Once e is extracted, we conclude the proof with the following steps: 1) the credentials are simulated by issuing fresh CMZ credentials, 2) SAGA presentations in credentials are always given in an order independent of b under the SAGA privacy property, and 3), the $C_a, \vec{X}_0, \vec{Z}_0$ in the credential requests are

simulated randomly. These changes lead to a game hop in which b is information theoretically hidden, concluding our proof.

Theorem 7. $\text{mKVAC}_{\text{CMZ}}$ is deniable for DenS in Figure 22 if NIP_I is sound, and NIP_S is zero-knowledge.

Proof (Sketch). Our deniability simulator DenS works as follows: Given $\text{vsk} := (x_0, \vec{x})$, it first simulates $(\tilde{U}, Z, C_V, \vec{C})$ of pres with random $Z, C_V, \vec{C}$ while $\tilde{U}$ is computed by using vsk . Then it computes the proof π_S of pres by using x_1 as witness for the $X_1 = x_1 G$ part of NIP_S .

6 Applications & Implementation

Applications of mKVAC. We now sketch how our multi-verifier KVACs can be used to enhance security in closed settings (such as Apple’s anonymous rate-limited credentials), and also fits within the EUDI context.

Anonymous Rate-limited Credentials (ARC). A recent internet draft [YW25] proposes *anonymous rate-limited credentials (ARC)* to improve Privacy Pass, which is an IETF standard for single-use anonymous tokens [CDVW24]. In ARC, users obtain a CMZ KVAC on a PRF key and authenticate by revealing a PRF evaluation on a random index $i \in [n]$ where n is the rate-limit, along with a proof of correctness tied to the credential. Verifiers accept only fresh PRF outputs which ensures rate-limiting. ARC improves Privacy Pass by supporting multi-show unlinkability and rate-limiting, but loses public-verifiability (if Privacy Pass is used with blind RSA signatures). The draft reasons their choice by noting that publicly verifiable credentials, such as BBS, rely on pairings which are not widely supported [YW25]. Another reason for switching from Privacy Pass to ARC is that it enables *verifier-specific rate-limiting*, i.e., each verifier can set their own upper bound. That is, ARC considers to be run with multiple verifiers — which all need to have access to the same secret key when using conventional KVACs. That hiding access information is important, even in an Apple internal setting, is demonstrated through Apple’s use of Privacy Pass in the blind authentication to the Private Cloud Compute service [App25], aiming to hide which node server the user is accessing.

Our mKVAC fits that ARC system perfectly and provides better security: Clearly, verification is supposed to be run significantly more often than issuance in this system. Thus, in a conventional KVAC the issuer’s key needs to be highly replicated to handle high-throughput authentications, which jeopardizes security of the entire system. Using mKVAC, the issuer secret key is *not* needed for every verification, and instead a more fine-grained key structure can be established. If a verifier key is compromised, it can be revoked without impacting other verifiers.

EUDI (with Intermediaries). In an eID system, the number and set of verifiers might not be that clear and well established as in the ARC context. Using our mKVAC scheme, the user might have to fetch a larger amount of credentials.

$ \vec{a} $	Credential issuance						Credential presentation		
	Obt ₁ (ms)	Iss (ms)	Obt ₂ (ms)	creq	bcred	cred	Present (ms)	VfPres (ms)	pres
4	9.35	8.57	0.69	19.9	0.2	0.1	1.09	0.80	0.6
8	12.12	10.92	0.68	29.8	0.2	0.1	1.21	0.85	1.0
32	33.02	26.93	0.69	89.0	0.2	0.1	2.59	1.96	3.2
64	62.07	47.41	0.70	168	0.2	0.1	3.70	2.97	6.2

Fig. 13. mKVAC benchmarks. Times are in milliseconds, and sizes are in KiB. (i) The three-step issuance pipeline `Obt1`, `issue_cred` (`Iss`), `Obt2`; (ii) presentation (`Present`) and verification (`VfPres`). Sizes are for request `creq`, blinded credential `bcred`, credential `cred`, and presentation `pres`.

However, for scenarios where the user wants to repeatedly authenticate to the same set of core services while remaining anonymous, our solution provides the first pairing-free and multi-show unlinkable solution. Further, the implicit “RP authentication” aspect of mKVAC (verifiers must explicitly register with the issuer), is an interesting strengthening over truly publicly verifiable solutions and can counter the risk of over-authentication, where user might get tricked into revealing identifying information to untrusted verifiers. Further, our mKVAC scheme naturally fits a deployment setting where verifiers rely on a smaller set of intermediaries that provide *verification-as-a-service*. In fact, while the EUDI envisions a fully distributed setting, the real-world deployment might end up being much more centralized. Already for simple (ECDSA-based) immunity certificates rolled out during COVID, which were the first wide deployment of user-centric credentials, the verification turned out to be too complex for services: not because of the cryptography, but because the services would have to maintain credential schemas and revocation check-ups for dozens of countries and millions of users. This led to the development of intermediaries that verify the credential presentations and translate them into simpler and standardized tokens [eHe22, GL23]. For such an intermediary-based deployment, our mKVAC fits perfectly. This is also where the deniability aspect of our solution is important, making user data collected by intermediaries unattractive to buy.

Performance evaluation. We implemented the end-to-end mKVAC prototype in pure Rust, using the Edwards25519 curve (via the Arkworks ecosystem [Ark22]) for all group operations and hash-to-scalar computations. All NIZK proofs in the request, issuance, and presentation phases are instantiated using the Fiat-Shamir transform, except NIP_R which is instantiated with the Fischlin transform (Appendix C). All experiments were conducted on a MacBook Pro with an Apple M4 processor and 24 GB RAM. We measured latency for each algorithm in the mKVAC API. All numbers are for fully hidden attributes (no selective disclosure), reflecting the default predicate from Appendix C. Our benchmark results are shown in Figure 13. We discuss details in Appendix D.

References

- AHIV17. Scott Ames, Carmit Hazay, Yuval Ishai, and Muthuramakrishnan Venkatasubramanian, *Ligero: Lightweight sublinear arguments without a trusted setup*, ACM CCS 2017 (Bhavani M. Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu, eds.), ACM Press, October / November 2017, pp. 2087–2104.
- App25. Apple, *Apple Private Cloud Compute*, <https://security.apple.com/documentation/private-cloud-compute>, 2025.
- Ark22. Arkworks Contributors, *Arkworks zkSNARK ecosystem*, <https://arkworks.rs>, 2022.
- BBC⁺24. Carsten Baum, Olivier Blazy, Jan Camenisch, Jaap-Henk Hoepman, Eysa Lee, Anja Lehmann, Anna Lysyanskaya, René Mayrhofer, Hart Montgomery, Ngoc Khanh Nguyen, et al., *Cryptographers’ feedback on the eu digital identity’s arf*.
- BBDE19. Johannes Blömer, Jan Bobolz, Denis Diemert, and Fabian Eidens, *Updatable anonymous credentials and applications to incentive systems*, ACM CCS 2019 (Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz, eds.), ACM Press, November 2019, pp. 1671–1685.
- BBDT16. Amira Barki, Solenn Brunet, Nicolas Desmoulins, and Jacques Traoré, *Improved algebraic MACs and practical keyed-verification anonymous credentials*, SAC 2016 (Roberto Avanzi and Howard M. Heys, eds.), LNCS, vol. 10532, Springer, Cham, August 2016, pp. 360–380.
- BBS04. Dan Boneh, Xavier Boyen, and Hovav Shacham, *Short group signatures*, Advances in Cryptology - CRYPTO 2004, 24th Annual International Cryptology Conference, Santa Barbara, California, USA, August 15-19, 2004, Proceedings (Matthew K. Franklin, ed.), Lecture Notes in Computer Science, vol. 3152, Springer, 2004, pp. 41–55.
- BEK⁺21. Jan Bobolz, Fabian Eidens, Stephan Krenn, Sebastian Ramacher, and Kai Samelin, *Issuer-hiding attribute-based credentials*, CANS 21 (Mauro Conti, Marc Stevens, and Stephan Krenn, eds.), LNCS, vol. 13099, Springer, Cham, December 2021, pp. 158–178.
- Bel20. Mihir Bellare, *Lectures on NIZKs* (en).
- BL13. Foteini Baldimtsi and Anna Lysyanskaya, *Anonymous credentials light*, ACM CCS 2013 (Ahmad-Reza Sadeghi, Virgil D. Gligor, and Moti Yung, eds.), ACM Press, November 2013, pp. 1087–1098.
- Bra95. Stefan Brands, *Off-line electronic cash based on secret-key certificates*, LATIN 1995 (Ricardo A. Baeza-Yates, Eric Goles Ch., and Patricio V. Poblete, eds.), LNCS, vol. 911, Springer, Berlin, Heidelberg, April 1995, pp. 131–166.
- BTZ22. Mihir Bellare, Stefano Tessaro, and Chenzhi Zhu, *Stronger security for non-interactive threshold signatures: BLS and FROST*, Cryptology ePrint Archive, Report 2022/833, 2022.
- CAHLT25. Rutchathon Chairattana-Apirom, Franklin Harding, Anna Lysyanskaya, and Stefano Tessaro, *Server-aided anonymous credentials*, CRYPTO 2025, Part VI (Yael Tauman Kalai and Seny F. Kamara, eds.), LNCS, vol. 16005, Springer, Cham, August 2025, pp. 291–324.
- CDDH19. Jan Camenisch, Manu Drijvers, Petr Dzurenda, and Jan Hajny, *Fast keyed-verification anonymous credentials on standard smart cards*, IFIP International Conference on ICT Systems Security and Privacy Protection, Springer, 2019, pp. 286–298.

- CDS94. Ronald Cramer, Ivan Damgård, and Berry Schoenmakers, *Proofs of partial knowledge and simplified design of witness hiding protocols*, CRYPTO'94 (Yvo Desmedt, ed.), LNCS, vol. 839, Springer, Berlin, Heidelberg, August 1994, pp. 174–187.
- CDVW24. Sofia Celi, Alex Davidson, Steven Valdez, and Christopher A. Wood, *Privacy Pass Issuance Protocols*, RFC 9578, June 2024.
- Cha85. David Chaum, *Security without identification: Transaction systems to make big brother obsolete*, Communications of the ACM **28** (1985), no. 10, 1030–1044.
- CKL⁺14. Jan Camenisch, Stephan Krenn, Anja Lehmann, Gert Læssøe Mikkelsen, Gregory Neven, and Michael Østergaard Pedersen, *Formal treatment of privacy-enhancing credential systems*, Cryptology ePrint Archive, Report 2014/708, 2014.
- CL01. Jan Camenisch and Anna Lysyanskaya, *An efficient system for non-transferable anonymous credentials with optional anonymity revocation*, EUROCRYPT 2001 (Birgit Pfitzmann, ed.), LNCS, vol. 2045, Springer, Berlin, Heidelberg, May 2001, pp. 93–118.
- CL02. ———, *Dynamic accumulators and application to efficient revocation of anonymous credentials*, CRYPTO 2002 (Moti Yung, ed.), LNCS, vol. 2442, Springer, Berlin, Heidelberg, August 2002, pp. 61–76.
- CL03. ———, *A signature scheme with efficient protocols*, SCN 02 (Stelvio Cimato, Clemente Galdi, and Giuseppe Persiano, eds.), LNCS, vol. 2576, Springer, Berlin, Heidelberg, September 2003, pp. 268–289.
- CL04. ———, *Signature schemes and anonymous credentials from bilinear maps*, CRYPTO 2004 (Matthew Franklin, ed.), LNCS, vol. 3152, Springer, Berlin, Heidelberg, August 2004, pp. 56–72.
- CMZ14. Melissa Chase, Sarah Meiklejohn, and Greg Zaverucha, *Algebraic MACs and Keyed-Verification Anonymous Credentials*, Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security (New York, NY, USA), CCS '14, Association for Computing Machinery, 2014, pp. 1205–1216.
- CPZ20. Melissa Chase, Trevor Perrin, and Greg Zaverucha, *The Signal private group system and anonymous credentials supporting efficient verifiable encryption*, ACM CCS 2020 (Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna, eds.), ACM Press, November 2020, pp. 1445–1459.
- CV02. Jan Camenisch and Els Van Herreweghen, *Design and implementation of the idemix anonymous credential system*, ACM CCS 2002 (Vijayalakshmi Atluri, ed.), ACM Press, November 2002, pp. 21–30.
- DG23. Heini Bergsson Debes and Thanassis Giannetsos, *RETRACT: Expressive Designated Verifier Anonymous Credentials*, Proceedings of the 18th International Conference on Availability, Reliability and Security (New York, NY, USA), ARES '23, Association for Computing Machinery, 2023, pp. 1–12.
- DGS⁺18. Alex Davidson, Ian Goldberg, Nick Sullivan, George Tankersley, and Filippo Valsorda, *Privacy pass: Bypassing internet challenges anonymously*, PoPETs **2018** (2018), no. 3, 164–180.
- DKS16. David Derler, Stephan Krenn, and Daniel Slamanig, *Signer-anonymous designated-verifier redactable signatures for cloud-based data sharing*, CANS 16 (Sara Foresti and Giuseppe Persiano, eds.), LNCS, vol. 10052, Springer, Cham, November 2016, pp. 211–227.

- eHe22. eHealth Network, *EU Digital COVID Certificate Validation Service.*, <https://github.com/eu-digital-green-certificates/dgca-validation-service/tree/2f047cd>, 2022.
- Eth19. Ethereum Foundation, *Semaphore: A zero-knowledge protocol for anonymous interactions*, <https://semaphore.pse.dev/>, 2019, Accessed: 2025-08-26.
- EU 25. EU Digital Identity Wallet, *European digital identity wallet. architecture and reference framework. version 2.4.0*, 2025.
- Eur24. European Parliament and Council, *Regulation (EU) 2024/1183 of the European Parliament and of the Council of 11 April 2024 amending Regulation (EU) No 910/2014 as regards establishing the European Digital Identity Framework*, <https://eur-lex.europa.eu/eli/reg/2024/1183/oj>, 2024, Official Journal of the European Union, L 1183, 30 April 2024, entered into force 20 May 2024.
- Eur25. European Commission/EU Digital Identity, *EU Digital Identity Wallet*, <https://ec.europa.eu/digital-building-blocks/sites/spaces/EUDIGITALIDENTITYWALLET/pages/694487738/EU+Digital+Identity+Wallet+Home>, 2025, Accessed: 2025-09-30.
- Fas25. Matteo Frigo and abhi shelat, *The longfellow zero-knowledge scheme*, IETF Internet-Draft draft-google-cfrg-libzk-01, September 2025, Work in Progress.
- FG18. Georg Fuchsbauer and Romain Gay, *Weakly secure equivalence-class signatures from standard assumptions*, PKC 2018, Part II (Michel Abdalla and Ricardo Dahab, eds.), LNCS, vol. 10770, Springer, Cham, March 2018, pp. 153–183.
- FHS19. Georg Fuchsbauer, Christian Hanser, and Daniel Slamanig, *Structure-preserving signatures on equivalence classes and constant-size anonymous credentials*, Journal of Cryptology **32** (2019), no. 2, 498–546.
- Fis05. Marc Fischlin, *Communication-efficient non-interactive proofs of knowledge with online extractors*, CRYPTO 2005 (Victor Shoup, ed.), LNCS, vol. 3621, Springer, Berlin, Heidelberg, August 2005, pp. 152–168.
- FKMV12. Sebastian Faust, Markulf Kohlweiss, Giorgia Azzurra Marson, and Daniele Venturi, *On the non-malleability of the Fiat-Shamir transform*, INDOCRYPT 2012 (Steven D. Galbraith and Mridul Nandi, eds.), LNCS, vol. 7668, Springer, Berlin, Heidelberg, December 2012, pp. 60–79.
- FS87. Amos Fiat and Adi Shamir, *How to prove yourself: Practical solutions to identification and signature problems*, CRYPTO’86 (Andrew M. Odlyzko, ed.), LNCS, vol. 263, Springer, Berlin, Heidelberg, August 1987, pp. 186–194.
- Fs24. Matteo Frigo and abhi shelat, *Anonymous credentials from ECDSA*, Cryptology ePrint Archive, Report 2024/2010, 2024.
- GKR08. Shafi Goldwasser, Yael Tauman Kalai, and Guy N. Rothblum, *Delegating computation: interactive proofs for muggles*, 40th ACM STOC (Richard E. Ladner and Cynthia Dwork, eds.), ACM Press, May 2008, pp. 113–122.
- GL23. Tarek Galal and Anja Lehmann, *Privacy-preserving outsourced certificate validation*, Proceedings on Privacy Enhancing Technologies (2023).
- Goo25. Google, *Longfellow zk*, <https://github.com/google/longfellow-zk>, 2025, Open-source zero-knowledge proof library for age assurance (accessed: 2025-09-10).

- HIP⁺24. Scott Hendrickson, Jana Iyengar, Tommy Pauly, Steven Valdez, and Christopher A. Wood, *Rate-Limited Token Issuance Protocol*, Internet-Draft draft-ietf-privacypass-rate-limit-tokens-06, Internet Engineering Task Force, April 2024, Work in Progress.
- HS21. Lucjan Hanzlik and Daniel Slamanig, *With a little help from my friends: Constructing practical anonymous credentials*, ACM CCS 2021 (Giovanni Vigna and Elaine Shi, eds.), ACM Press, November 2021, pp. 2004–2023.
- ISO21. *Personal identification — iso-compliant driving licence — part 5: Mobile driving licence (mdl) application*, 2021, FDIS / Final Draft International Standard.
- KLR23. Julia Kastner, Julian Loss, and Omar Renawi, *Concurrent security of anonymous credentials light, revisited*, ACM CCS 2023 (Weizhi Meng, Christian Damsgaard Jensen, Cas Cremers, and Engin Kirda, eds.), ACM Press, November 2023, pp. 45–59.
- Ks22. Yashvanth Kondi and abhi shelat, *Improved straight-line extraction in the random oracle model with applications to signature aggregation*, ASIACRYPT 2022, Part II (Shweta Agrawal and Dongdai Lin, eds.), LNCS, vol. 13792, Springer, Cham, December 2022, pp. 279–309.
- LR22. Anna Lysyanskaya and Leah Namisa Rosenbloom, *Universally composable Σ -protocols in the global random-oracle model*, TCC 2022, Part I (Eike Kiltz and Vinod Vaikuntanathan, eds.), LNCS, vol. 13747, Springer, Cham, November 2022, pp. 203–233.
- Mau09. Ueli M. Maurer, *Unifying zero-knowledge proofs of knowledge*, AFRICACRYPT 09 (Bart Preneel, ed.), LNCS, vol. 5580, Springer, Berlin, Heidelberg, June 2009, pp. 272–286.
- MBS⁺25. Omid Mirzamohammadi, Jan Bobolz, Mahdi Sedaghat, Emad Heydari Beni, Aysajan Abidin, Dave Singelée, and Bart Preneel, *Keyed-verification anonymous credentials with highly efficient partial disclosure*, IACR Communications in Cryptology **2** (2025), no. 3.
- Orr24. Michele Orrù, *Revisiting keyed-verification anonymous credentials*, Cryptology ePrint Archive, Report 2024/1552, 2024.
- Orr25. ———, *Revisiting keyed-verification anonymous credentials*, ACM CCS 2025 (Chun-Ying Huang, Jyh-Cheng Chen, Shiuh-Pyng Shieh, David Lie, and Véronique Cortier, eds.), ACM Press, October 2025, pp. 1188–1199.
- PZ13. Christian Paquin and Greg Zaverucha, *U-prove cryptographic specification v1.1*, Technical Report Revision 3, Microsoft Corporation, December 2013, Accessed: 2025-09-01.
- San20. Olivier Sanders, *Efficient redactable signature and application to anonymous credentials*, PKC 2020, Part II (Aggelos Kiayias, Markulf Kohlweiss, Petros Wallden, and Vassilis Zikas, eds.), LNCS, vol. 12111, Springer, Cham, May 2020, pp. 628–656.
- Sch90. Claus-Peter Schnorr, *Efficient identification and signatures for smart cards*, CRYPTO’89 (Gilles Brassard, ed.), LNCS, vol. 435, Springer, New York, August 1990, pp. 239–252.
- SKSW22. Yumi Sakemi, Tetsutaro Kobayashi, Tsunekazu Saito, and Riad S. Wahby, *Pairing-friendly curves*, Internet-Draft, draft-irtf-cfrg-pairing-friendly-curves-11, Internet Engineering Task Force, November 2022, Work in Progress.
- ST24. Olivier Sanders and Jacques Traoré, *Compact issuer-hiding authentication, application to anonymous credential*, PoPETs **2024** (2024), no. 3, 645–658.

- TG23. Lindsey Tulloch and Ian Goldberg, *Lox: Protecting the social graph in bridge distribution*, PoPETs **2023** (2023), no. 1, 494–509.
- TZ23. Stefano Tessaro and Chenzhi Zhu, *Revisiting BBS signatures*, EURO-CRYPT 2023, Part V (Carmen Hazay and Martijn Stam, eds.), LNCS, vol. 14008, Springer, Cham, April 2023, pp. 691–721.
- YW25. Cathie Yun and Christopher A. Wood, *Anonymous Rate-Limited Credentials*, Internet-Draft draft-yun-cfrg-arc-01, Internet Engineering Task Force, August 2025, Work in Progress.

A Extended Related Work

Anonymous Credentials. Over the years, a number of concrete systems have shown how ACs can be realized in practice. Microsoft’s U-Prove [PZ13] builds on blind-signature techniques, while IBM’s Idemix [CV02] relies on signature schemes tailored for efficient zero-knowledge proofs. In the blind-signature approach, privacy comes from the algebraic properties of blind Schnorr signatures. In the second approach, the issuer signs a commitment to the user’s attributes, and the user can later prove knowledge of this signature while revealing only selected information. Both approaches showed that ACs are possible in practice, but they also come with important costs and restrictions. Baldimtsi and Lysyanskaya [BL13] proposed a one-show anonymous credential, termed “AC light” (ACL). In this setting, users must obtain from the issuer as many credential copies as the number of intended showings. This design, however, entails a tradeoff between privacy and efficiency: either users acquire a sufficiently large batch of ACLs in advance — bounded by an estimate of their lifetime usage — or they request re-issuance once exhausted, thereby leaking information about their credential usage patterns.

SNARK-based Anonymous Credentials. General-purpose zero-knowledge systems have been explored as a way to bring more flexibility to credential design. For example, Frigo and Shelat [Fs24, Goo25] proposed a technique that combines Liger [AHIV17] and the GKR sum-check protocols [GKR08], allowing users to prove possession of valid ECDSA-signed credentials without revealing their device public key over mobile Driving Licence (mDL) data format [ISO21]. This design shows that zero-knowledge can address concrete privacy flaws in existing credential standards like mDL, while remaining compatible with current infrastructure and feasible on smartphones. Nonetheless, it comes with relatively higher computational costs and larger proof sizes compared to specialized credential schemes. Similarly, Semaphore [Eth19] uses zk-SNARKs to build a privacy-preserving identity system for Ethereum, enabling users to prove membership in a group without revealing their identity. While these systems offer great flexibility, they often require more computational resources and larger proofs than traditional credential schemes, making them less suitable for resource-constrained environments.

Anonymous authentication Schemes with Keyed Verification. A major step forward for pairing-free settings came with the introduction of keyed-verification anonymous credentials (KVACs), first proposed by Chase, Meiklejohn, and Zaverucha [CMZ14]. By replacing public-key signatures with algebraic message authentication codes, KVACs avoid pairings while preserving efficient proof techniques. Verification, however, requires knowledge of the issuer’s secret key, which prevents public verifiability and makes deployment difficult in scenarios with many verifiers. Orrù [Orr25] revisited the foundations of KVACs and formalized stronger notions such as extractability and one-more unforgeability. His work introduced optimized variants, notably μ CMZ and μ BBS, that achieve constant-size issuance and statistical anonymity in the algebraic group model. These results not only improve efficiency but also clarify the exact security guarantees offered by pairing-free credentials. Despite these advances, KVACs remain constrained by the requirement that issuer and verifier share a secret key, which restricts scalability and limits applicability in decentralized or large-scale identity systems.

Another important line of research builds on BBS-style credentials. BBS and BBS+ enable efficient selective disclosure proofs and are naturally publicly verifiable [BBC⁺24]. Barki, et al. [BBDT16] proposed BBS-based KVACs that reduce reliance on pairings while retaining compactness and flexibility. Orrù [Orr25] further analyzed and refined this line, proposing μ BBS as a statistically anonymous, pairing-free alternative. Very recently, Mirzamohammadi et al. [MBS⁺25] proposed two KVAC constructions with short presentation size: one based on pairing-friendly curves, and another pairing-free variant that, however, the latter comes with credentials of linear size in the number of attributes. These developments highlight an interesting trade-off: these constructions yield compact and efficient credentials, but rely on pairing-friendly curves or stronger algebraic assumptions, while their KVAC counterparts avoid pairings but remain tied to issuer-managed secret keys.

While KVACs assume a symmetric key between the issuer and the verifier, there are previous works that define a keyed verification anonymous authentication schemes where the verifier keys are distinct from the issuer keys. However, none of them fit into our setting. Furthermore, we are not aware of instantiations of them in pairing-free primed order groups. Below we list these works.

Derler et al. proposed *signer-anonymous designated-verifier redactable signatures* [DKS16]. In a nutshell, this primitive extends group signatures to the redactable and designated-verifier setting. This primitive provides anonymous authentication for its signers as they are registered by the group manager. However, this scheme does not allow group manager to certify attributes of users/signers. Furthermore, its anonymity holds only conditionally as the group manager can de-anonymize the users.

Recently, Debes et al. proposed *designated-verifier anonymous credentials* [DG23]. Their scheme allows publicly verifiable credentials which are designated to a verifier in the presentation phase. The motivation of their design is to achieve deniability for anonymous credential schemes.

Sanders and Traore recently proposed an issuer-hiding anonymous credential scheme [ST24]. Issuer hiding ACs allow predicates to set multiple possible issuers to present a credential. Then a user holding a credential from any of them can authenticate without revealing the exact issuer that they hold a credential from. The authors define a key pair to dedicate to each credential predicate. Then the verifier stores the secret key to verify presentations for this predicate. Their main motivation to define keyed verification is obtaining efficiency in the construction level.

Server-Aided Anonymous Credentials. Server-Aided Anonymous Credentials (SAAC) [CAHLT25] were introduced to overcome the tension between efficiency and public verifiability of KVACs. In this model, a helper server, operated by the issuer, transforms private KVAC presentations into publicly verifiable proofs. The helper is designed to remain oblivious to the user’s attributes, so it does not learn sensitive information. This construction allows pairing-free systems to achieve public verifiability, a property that is otherwise lost in KVACs, and thus makes them more suitable for open deployments such as the European Digital Identity Wallet. The drawback is the reliance on additional infrastructure: the helper server becomes a potential point of failure and increases operational complexity, though in return it enables verifiers to be fully decoupled from issuers. The SAAC framework includes both BBS-based instantiations, which achieve compactness and statistical anonymity under gap assumptions, and DDH-based variants, which avoid gap assumptions at some additional efficiency cost.

In parallel, blind-signature based credentials have remained an attractive option for lightweight deployments. Recent work on anonymous tokens and VOPRF-based credentials shows that such systems are highly efficient for single-use or limited-show scenarios, which explains their adoption by major platforms. However, they lack the generality of multi-show credentials and require re-issuance for repeated use, which may reintroduce linkability or usage leakage without careful system design.

B Further Preliminaries

In this section, we list additional preliminaries.

B.1 Assumptions

Definition 10 (DDH Assumption). DDH assumption holds if the following advantage is negligible for all $\lambda \in \mathbb{N}$ and $\mathcal{A}$:

$$\text{Adv}_{\mathcal{A}}^{\text{DDH}}(\lambda) := \left| \Pr \left[\begin{array}{l} (\mathbb{G}, p, G) \leftarrow \text{GGen}(\lambda) \\ b \leftarrow \{0, 1\}, a_{\text{DDH}}, b_{\text{DDH}}, c_{\text{DDH}} \leftarrow \mathbb{Z}_p \\ A := a_{\text{DDH}}G, B := b_{\text{DDH}}G \\ C := (c_{\text{DDH}} \cdot b + a_{\text{DDH}} \cdot b_{\text{DDH}})G \\ b' \leftarrow \mathcal{A}((\mathbb{G}, p, G), A, B, C) \end{array} : b = b' \right] - 1/2 \right|.$$

$\text{Exp}_{\mathcal{A}}^{\text{CMZ-Unf},n}(\lambda)$	$\mathcal{OSign}(\vec{m} \in \mathbb{Z}_p^n)$
$Q := \emptyset, \text{pp} \leftarrow \text{CMZ.Setup}(\lambda, n)$	$Q := Q \cup \{\vec{m}\}$
$(\text{sk}, \text{pk}) \leftarrow \text{CMZ.Kg}()$	return $\text{CMZ.Sign}(\text{sk}, \vec{m})$
$(\sigma^*, \vec{m}^*) \leftarrow \mathcal{A}^{\mathcal{OSign}, \mathcal{OHelp}}(\mathbb{G}, p, G, n, \text{pk})$	$\mathcal{OHelp}(U_0, \dots, U_n, V)$
return $\text{Vf}(\text{sk}, \sigma^*, \vec{m}^*) \wedge \vec{m}^* \notin Q$	return $V = \sum_{j=0}^n x_j U_j$

Fig. 14. CMZ unforgeability game [Orr25].

Definition 11 (*n*-DL Assumption). *n*-DL assumption holds if the following advantage is negligible for all $\lambda \in \mathbb{N}$ and $\mathcal{A}$:

$$\text{Adv}_{\mathcal{A}}^{n\text{-DL}}(\lambda) := \Pr \left[\begin{array}{l} (\mathbb{G}, p, G) \leftarrow \text{GGen}(\lambda), a \xleftarrow{\$} \mathbb{Z}_p \\ \text{for } i \in [n] : A_i := a^i G \\ a' \leftarrow \mathcal{A}((\mathbb{G}, p, G), (A_i)_{i \in [n]}) \end{array} : a = a' \right].$$

B.2 The CMZ MAC

The CMZ MAC scheme is described in Figure 2. We define unforgeability as follows, involving the game described in Figure 14. Note that the definition of unforgeability contains a helper oracle $\mathcal{OHelp}$ as in [Orr25].

Definition 12. We say that CMZ MAC is unforgeable if for all $n \in \mathbb{N}$ and all ppt $\mathcal{A}$, the advantage $\text{Adv}_{\mathcal{A}}^{\text{CMZ-Unf},n}(\lambda) := \Pr[\text{Exp}_{\mathcal{A}}^{\text{CMZ-Unf},n}(\lambda) = 1]$ is negligible in λ .

In the full version of the paper [Orr24], Orru proves that CMZ MAC is unforgeable in AGM under 3-DL assumption, which we recap below.

Lemma 1 (CMZ MAC Unforgeability (Orru [Orr24])). CMZ MAC from Figure 2 is unforgeable for all $n = \text{poly}(\lambda)$ against algebraic adversaries with the advantage $\text{Adv}_{\mathcal{A}}^{\text{CMZ-Unf},n}(\lambda) \leq \text{Adv}_{\mathcal{A}_{3\text{-DL}}}^{3\text{-DL}}(\lambda) + \text{Adv}_{\mathcal{A}_{1\text{-DL}}}^{1\text{-DL}}(\lambda) + 3/p$.

B.3 NIZK Definitions.

Below we present the formal definitions of NIZK properties we need throughout the paper. We start with the definition of the completeness property. Note that the previous work on Fischlin's transform considers overwhelming completeness whereas our definition is for perfect completeness. Any overwhelmingly complete NIZK proof scheme can be turned into a perfectly complete one as follows: The proving algorithm is updated such that if it fails to generate a valid proof, it reveals the witness as the proof. The verification either verifies a valid proof or checks the revealed witness is valid for the statement. This protocol achieves perfect completeness. Moreover, the zero-knowledge property of the extended protocol can be shown by relying on the zero-knowledge property and the overwhelming completeness of the initial NIZK proof scheme. The remaining properties are unaffected.

$\text{Exp}_{\mathcal{A}, \text{NIP}_{\mathcal{R}}, S}^{\text{ZK}}(\lambda)$	$\mathcal{O}\text{Prove}(\mathbb{x}, \mathbb{w}, \text{ctx}_{\text{NIP}})$
$b \xleftarrow{\$} \{0, 1\}$	require $\mathcal{R}(\mathbb{x}, \mathbb{w})$
if $b = 0$: $\text{pp}_{\text{NIP}} \leftarrow \text{Setup}^{\text{RO}}(\lambda)$	if $b = 0$: $\pi \leftarrow \text{Prove}^{\text{RO}}(\mathbb{x}, \mathbb{w}, \text{ctx}_{\text{NIP}})$
if $b = 1$: $(\text{pp}_{\text{NIP}}, td) \leftarrow \text{S.Setup}^{\text{Pr}}(\lambda)$	if $b = 1$: $\pi \leftarrow \text{S.Prove}^{\text{Pr}}(td, \mathbb{x}, \text{ctx}_{\text{NIP}})$
$b' \leftarrow \mathcal{A}^{\text{RO}, \mathcal{O}\text{Prove}}(\text{pp}_{\text{NIP}})$	return π
return $b = b'$	
$\text{Pr}(x, v)$	$\text{RO}(x)$
if $\text{ROT}[x] = \perp$: $\text{ROT}[x] := v$	if $\text{ROT}[x] = \perp$: $\text{ROT}[x] \xleftarrow{\$} \{0, 1\}^\lambda$
	$Q_{\text{RO}} := Q_{\text{RO}} \cup \{(x, \text{ROT}[x])\}$
	return $\text{ROT}[x]$

Fig. 15. Zero-knowledge game.

$\text{Exp}_{\mathcal{A}, \text{NIP}_{\mathcal{R}}, \text{Extract}}^{\text{SL}}(\lambda)$	$\text{RO}(x)$	$\mathcal{O}\text{Chall}(\mathbb{x}, \pi, \text{ctx}_{\text{NIP}})$
success := false	if $\text{ROT}[x] = \perp$: $\text{ROT}[x] \xleftarrow{\$} \{0, 1\}^\lambda$	if $\forall (\mathbb{x}, \text{ctx}_{\text{NIP}}, \pi)$:
$\text{pp}_{\text{NIP}} \leftarrow \text{Setup}^{\text{RO}}(\lambda)$	$Q_{\text{RO}} := Q_{\text{RO}} \cup \{(x, \text{ROT}[x])\}$	$\mathbb{w} \leftarrow \text{Extract}(\mathbb{x}, \text{ctx}_{\text{NIP}}, \pi, Q_{\text{RO}})$
$\mathcal{A}^{\text{RO}, \mathcal{O}\text{Chall}}(\text{pp}_{\text{NIP}})$	return $\text{ROT}[x]$	if $\neg \mathcal{R}(\mathbb{x}, \mathbb{w})$: success := true
return success		

Fig. 16. Straightline extractability game.

Definition 13 (Completeness). A NIP scheme for relation $\mathcal{R}$ is complete if for all $\lambda \in \mathbb{N}$, $(\mathbb{x}, \mathbb{w}) \in \mathcal{R}$, $\text{ctx}_{\text{NIP}} \in \{0, 1\}^*$, and $\text{pp}_{\text{NIP}} \leftarrow \text{Setup}(\lambda)$, $\pi \leftarrow \text{NIP.Prove}(\mathbb{x}, \mathbb{w}, \text{ctx}_{\text{NIP}})$, we have that $\Pr[\text{NIP.Vf}(\mathbb{x}, \text{ctx}_{\text{NIP}}, \pi)] = 1$.

Definition 14 (Zero-knowledge). A NIP scheme is zero-knowledge if there exists algorithms $S := (\text{Setup}, \text{Prove})$ such that for all PPT adversaries $\mathcal{A}$,

$$\text{Adv}_{\mathcal{A}, \text{NIP}_{\mathcal{R}}, S}^{\text{ZK}}(\lambda) := \left| \Pr[\text{Exp}_{\mathcal{A}, \text{NIP}_{\mathcal{R}}, S}^{\text{ZK}}(\lambda) = \text{true}] - 1/2 \right| \leq \text{negl}(\lambda).$$

Definition 15 (Soundness). A NIP scheme is sound if for all PPT adversaries $\mathcal{A}$, we have $\text{Adv}_{\mathcal{A}, \text{NIP}_{\mathcal{R}}}^{\text{Snd}}(\lambda) := \Pr[\forall (\mathbb{x}, \text{ctx}_{\text{NIP}}, \pi) = \text{true} \wedge \nexists \mathbb{w} : (\mathbb{x}, \mathbb{w}) \in \mathcal{R}] \leq \text{negl}(\lambda)$, where $(\mathbb{x}, \text{ctx}_{\text{NIP}}, \pi) \leftarrow \mathcal{A}^{\text{RO}}(\text{pp}_{\text{NIP}})$ and $\text{pp}_{\text{NIP}} \leftarrow \text{Setup}^{\text{RO}}(\lambda)$.

Definition 16 (Straightline extractability). A NIP scheme is straightline extractable if there exists a PPT algorithm Extract such that for all PPT adversaries $\mathcal{A}$, $\text{Adv}_{\mathcal{A}, \text{NIP}_{\mathcal{R}}, \text{Extract}}^{\text{SL}}(\lambda) := \Pr[\text{Exp}_{\mathcal{A}, \text{NIP}_{\mathcal{R}}, \text{Extract}}^{\text{SL}}(\lambda) = \text{true}] \leq \text{negl}(\lambda)$.

Definition 17 (Rewinding-based (weak) simulation-extractability). A NIP scheme is rewinding-based (weakly) simulation extractable for a simulator S if there are efficiently computable deterministic functions GetChalHashInput , GetWitness such that:

- *Simulated proofs use different hash preimages:* For all $\mathbb{X}, \text{ctx}_{\text{NIP}}, \pi, \mathbb{X}', \text{ctx}'_{\text{NIP}}, \pi'$, if $(\mathbb{X}, \text{ctx}_{\text{NIP}}) \neq (\mathbb{X}', \text{ctx}'_{\text{NIP}})$, then $\text{GetChalHashInput}(\mathbb{X}, \text{ctx}_{\text{NIP}}, \pi) \neq \text{GetChalHashInput}(\mathbb{X}', \text{ctx}'_{\text{NIP}}, \pi')$
- *Simulator only programs one hash:* Let $(\mathbb{X}, \mathbb{W}) \in \mathcal{R}$. The simulator $\text{S.Prove}^{\text{Pr}}(td, \mathbb{X}, \text{ctx}_{\text{NIP}})$ only makes a single query to $\text{Pr}(x, v)$ and when it outputs π , it fulfills $\text{GetChalHashInput}(\mathbb{X}, \text{ctx}_{\text{NIP}}, \pi) = x$.
- *2-extractability:* Let $\mathbb{X}, \text{ctx}_{\text{NIP}}, \pi, \pi', \text{RO}, \text{RO}'$ be arbitrary and let $x := \text{GetChalHashInput}(\mathbb{X}, \text{ctx}_{\text{NIP}}, \pi) = \text{GetChalHashInput}(\mathbb{X}, \text{ctx}_{\text{NIP}}, \pi')$. If $\text{RO}(x) \neq \text{RO}'(x)$ and $\text{Vf}^{\text{RO}}(\mathbb{X}, \text{ctx}_{\text{NIP}}, \pi) = \text{Vf}^{\text{RO}'}(\mathbb{X}, \text{ctx}_{\text{NIP}}, \pi') = \text{true}$, then it holds that $(\mathbb{X}, \text{GetWitness}(\mathbb{X}, \text{ctx}_{\text{NIP}}, \pi, \pi')) \in \mathcal{R}$.

This definition is non-standard, but encodes standard properties. For Fiat-Shamir Sigma protocol implementations of NIP, one can set $\text{GetChalHashInput}(\mathbb{X}, \text{ctx}_{\text{NIP}}, \pi)$ to output $(\text{ProtName}, \text{ctx}_{\text{NIP}}, \mathbb{X}, A)$, where A is the announcement (first message) of the underlying Sigma protocol. The witness extractor $\text{GetWitness}(\mathbb{X}, \text{ctx}_{\text{NIP}}, \pi, \pi')$ can be set to the output of the special soundness extractor for the underlying Sigma protocol on the two accepting transcripts encoded by π, π' . In this setting, the first and second properties together ensure that if an adversary outputs an accepting proof π for $\mathbb{X}, \text{ctx}_{\text{NIP}}$ not queried to the simulator, then the random oracle entry $\text{GetChalHashInput}(\mathbb{X}, \text{ctx}_{\text{NIP}}, \pi)$ was not programmed (and hence we can apply rewinding and change the hash image). The third property ensures that in case rewinding finds a second accepting proof with the same challenge hash preimage and different challenge hash images, we can efficiently compute a witness via GetWitness .

We also use the simpler property of rewinding-based proof of knowledge (in the absence of simulation).

Definition 18 (Rewinding-based proof of knowledge). *A NIP scheme is a rewinding-based proof of knowledge if there are efficiently computable deterministic functions $\text{GetChalHashInput}, \text{GetWitness}$ such that:*

- *2-extractability:* Let $\mathbb{X}, \text{ctx}_{\text{NIP}}, \pi, \pi', \text{RO}, \text{RO}'$ be arbitrary and let $x := \text{GetChalHashInput}(\mathbb{X}, \text{ctx}_{\text{NIP}}, \pi) = \text{GetChalHashInput}(\mathbb{X}, \text{ctx}_{\text{NIP}}, \pi')$. If $\text{RO}(x) \neq \text{RO}'(x)$ and $\text{Vf}^{\text{RO}}(\mathbb{X}, \text{ctx}_{\text{NIP}}, \pi) = \text{Vf}^{\text{RO}'}(\mathbb{X}, \text{ctx}_{\text{NIP}}, \pi') = \text{true}$, then it holds that $(\mathbb{X}, \text{GetWitness}(\mathbb{X}, \text{ctx}_{\text{NIP}}, \pi, \pi')) \in \mathcal{R}$.

For all NIP proof instantiations in this paper (which we discuss in Appendix C), we first define a Sigma protocol satisfying completeness, honest-verifier zero-knowledge, and 2-special soundness properties [Bel20]. Then the completeness, zero-knowledge and soundness properties of a Fiat-Shamir compiled NIP scheme follows from the mentioned properties of the underlying Sigma protocol in ROM [Bel20]. Furthermore, rewinding-based simulation extractability of Fiat-Shamir compiled proofs can be shown by relying on the 2-special soundness and the honest-verifier zero-knowledge properties of the underlying Sigma protocol, following Faust et al. [FKMV12]. We lead the reader to [Bel20] for the formal definition of Sigma protocols and their properties.

$\text{Fork}_{\mathcal{A}}(in)$ Pick the random coin ρ of $\mathcal{A}$ randomly $h_1, h'_1, \dots, h_q, h'_q \xleftarrow{\$} \mathcal{H}$ $(I, out) \leftarrow \mathcal{A}(in, h_1, \dots, h_q; \rho)$ if $I = \perp$: return $\perp$ $(I', out') \leftarrow \mathcal{A}(in, h_1, \dots, h_{I-1}, h'_I, \dots, h'_q; \rho)$ if $I \neq I'$: return $\perp$ return (I, out, out')

Fig. 17. Forking algorithm Fork.

Similar to the Fiat-Shamir case, completeness, zero-knowledge, and straightline-extractability properties of a Fischlin compiled NIP scheme follows from the completeness, zero-knowledge, soundness, and at least one of quasi-uniqueness or strong special soundness properties of the underlying Sigma protocol [Fis05, Ks22, LR22].

B.4 Forking Lemma

Below we recap the forking lemma described by Bellare et al. [BTZ22].

Lemma 2. *Let $q \geq 1$ be an integer and $\mathcal{H}$ be a set. Let $\mathcal{A}$ be a randomized algorithm that on input $in, h_1, \dots, h_q$ outputs a pair (I, out) , where $I \in \{\perp\} \cup [q]$ and out is a side output. Let IG be a randomized algorithm that generates in . Then for $in \xleftarrow{\$} \text{IG}$ and Fork described as in Figure 17,*

$$\Pr[\text{Fork}_{\mathcal{A}}(in) \neq \perp] \geq \frac{\Pr[I \neq \perp]^2}{q}$$

C Non-interactive zero-knowledge proofs

C.1 Fiat-Shamir Schnorr-style proofs

At the heart of our protocols lies a generalization of the Schnorr protocol [Sch90, Mau09] which is a family of Sigma protocols, applying the Fiat-Shamir transform [FS87] to it (we discuss Fischlin's transform [Fis05, Ks22, LR22] later). This protocol proves knowledge of a homomorphism preimage. For our purposes, let $\varphi : \mathbb{Z}_p^{\ell_w} \rightarrow \mathbb{G}^{\ell_i}$ be a homomorphism. Let $\text{H} : \{0, 1\}^* \rightarrow \mathbb{Z}_p$ be a hash function modeled as a random oracle.

The prover on input $(\mathfrak{x}, \mathfrak{w}, \text{ctx}_{\text{NIP}})$ first derives homomorphism image $\mathfrak{x}_{\text{Schnorr}} \in \mathbb{G}^{\ell_i}$ from $\mathfrak{x}$ (this depends on the specific protocol) and secret homomorphism preimage $\mathfrak{w}_{\text{Schnorr}} \in \mathbb{Z}_p^{\ell_w}$ from $\mathfrak{w}$ (again, depending on the protocol). The prover then proceeds as follows.

1. Generate a random $a \xleftarrow{\$} \mathbb{Z}_p^{\ell_w}$ and compute the announcement $A_{\text{Schnorr}} = \varphi(a) \in \mathbb{G}^{\ell_i}$.
2. Compute the challenge $c := H(\text{ProtName}, \text{ctx}_{\text{NIP}}, \mathbb{x}, A_{\text{Schnorr}}) \in \mathbb{Z}_p$. Here, ProtName is a protocol-specific constant identifier string to avoid potential cross-protocol attacks (e.g., $\text{ProtName} = \text{"mKVAC}_{\text{CMZ-Req}} \text{"}$).
3. Compute the response $s := a + c \cdot \mathbb{w}_{\text{Schnorr}} \in \mathbb{Z}_p^{\ell_w}$.
4. Output the proof $\pi := (c, s) \in \mathbb{Z}_p^{1+\ell_w}$.

The verifier on input $(\mathbb{x}, \pi, \text{ctx}_{\text{NIP}})$, where $\pi = (c, s) \in \mathbb{Z}_p^{1+\ell_w}$, proceeds as follows. The verifier derives $\mathbb{z}_{\text{Schnorr}}$ from $\mathbb{x}$ (like the prover; again, the specifics depend on the actual protocol). The verifier then proceeds as follows.

1. Compute the unique accepting announcement $A_{\text{Schnorr}} := \varphi(s) - c \cdot \mathbb{z}_{\text{Schnorr}} \in \mathbb{G}^{\ell_i}$.
2. Accept the proof if and only if $c = H(\text{ctx}_{\text{NIP}}, \mathbb{x}, A_{\text{Schnorr}})$.

We are going to provide the instances of the generalized Schnorr protocol to instantiate the NIP proofs that are necessary for our constructions. Maurer shows that the instances of the generalized Schnorr protocol satisfies the completeness, honest-verifier zero-knowledge, and 2-special soundness properties of a Sigma protocol [Mau09]. Thus, all needed properties from the Fiat-Shamir compiled proofs by our constructions are immediately satisfied.

C.2 Fischlin-transformed proofs

We now describe the (generalized) Schnorr protocol with randomized Fischlin's transform [Fis05, Ks22, LR22] applied to it. This enables straight-line extraction (i.e., extraction without rewinding), which we need for some of our proof systems. Let $W, R \in \mathbb{N}$ be parameters. R is the number of parallel protocols (higher R means better soundness and larger proofs), and W is the proof of work difficulty (higher W means better soundness and more prover computation cost). For soundness, we require W^{-R} , which is the chance that the adversary manages to create a valid unextractable proof in a single try, to be negligible. For concreteness, we may set $R = 128/\log_2(W)$, which gives soundness error 2^{-128} . Typical values for W are powers of two between 2^2 and 2^8 . Larger W increases prover computation cost and decreases proof size.

As above, the protocol proves knowledge of a homomorphism preimage. Let $\varphi : \mathbb{Z}_p^{\ell_w} \rightarrow \mathbb{G}^{\ell_i}$ be a homomorphism that must be collision-resistant for the prover (i.e., it must be computationally infeasible to find $a \neq 0$ such that $\varphi(a) = 0$). Let $H : \{0, 1\}^* \rightarrow \{0, \dots, W-1\}$ be a hash function modeled as a random oracle.

The prover on input $(\mathbb{x}, \mathbb{w}, \text{ctx}_{\text{NIP}})$ first derives homomorphism image $\mathbb{z}_{\text{Schnorr}} \in \mathbb{G}^{\ell_i}$ from $\mathbb{x}$ (this depends on the specific protocol) and secret homomorphism preimage $\mathbb{w}_{\text{Schnorr}} \in \mathbb{Z}_p^{\ell_w}$ from $\mathbb{w}$ (again, depending on the protocol). The prover then proceeds as follows.

1. For each $i \in [R]$, generate random $a^{(i)} \xleftarrow{\$} \mathbb{Z}_p^{\ell_w}$ and compute the announcement $A_{\text{Schnorr}}^{(i)} = \varphi(a^{(i)}) \in \mathbb{G}^{\ell_i}$.

Table 2. NIZK proof instantiations with their required soundness properties and applied transforms.

Proof	Required level of soundness	Transform
$\text{NIP}_{\text{BBSSAGA}}$	Soundness	Fiat-Shamir
NIP_R	Straight-line extraction	Fischlin
NIP_I	Extraction via rewinding	Fiat-Shamir
NIP_S	Extraction via rewinding	Fiat-Shamir

2. For each $i \in [R]$, repeat
 - Choose random challenge $c^{(i)} \xleftarrow{\$} \mathbb{Z}_p$.
 - Compute the response $s^{(i)} := a^{(i)} + c^{(i)} \cdot \mathfrak{w}_{\text{Schnorr}} \in \mathbb{Z}_p^{\ell_w}$.
 until $H(\text{ProtName}, \text{ctx}_{\text{NIP}}, \mathfrak{x}, (A_{\text{Schnorr}}^{(j)})_{j=1}^R, i, c^{(i)}, s^{(i)}) = 0$.
3. Output the proof $\pi := (c^{(i)}, s^{(i)})_{i=1}^R \in \mathbb{Z}_p^{R \cdot (1 + \ell_w)}$.

The verifier on input $(\mathfrak{x}, \pi, \text{ctx}_{\text{NIP}})$, where $\pi = (c^{(i)}, s^{(i)})_{i=1}^R \in \mathbb{Z}_p^{R \cdot (1 + \ell_w)}$, proceeds as follows. The verifier derives $\mathfrak{z}_{\text{Schnorr}}$ from $\mathfrak{x}$ (like the prover; again, the specifics depend on the actual protocol). The verifier then proceeds as follows.

1. Compute the unique accepting announcement $A_{\text{Schnorr}}^{(i)} := \varphi(s^{(i)}) - c^{(i)} \cdot \mathfrak{z}_{\text{Schnorr}} \in \mathbb{G}^{\ell_i}$.
2. Accept the proof if and only if for all $i \in [R]$, it holds that $H(\text{ProtName}, \text{ctx}_{\text{NIP}}, \mathfrak{x}, (A_{\text{Schnorr}}^{(j)})_{j=1}^R, i, c^{(i)}, s^{(i)}) = 0$.

For the straightline-extractability of Fischlin compiled proofs — only needed for NIP_R — we argue that the quasi-uniqueness is satisfied by the instance of generalized Schnorr protocol we provide, implying that the straightline-extractability is achieved [Fis05]. The remaining properties of the Fischlin compiled proofs follows from the properties of the generalized Schnorr protocol [Mau09].

C.3 Application to our protocols

Below, we apply these paradigms (both Fiat-Shamir Schnorr and Fischlin Schnorr) to the protocols in this paper. We focus on the $\text{mKVAC}_{\text{CMZ}}$ construction (Section 5) instantiated with the BBSSAGA (Section 4.2). An overview of the protocols we need to instantiate can be found in Table 2. By default, we will assume that the attribute predicate ϕ is trivial, i.e. $\phi((a_j)_{j=1}^n) = \text{true}$ for all a_j . This means that all attributes are perfectly hidden from the issuer/verifier. We discuss partial disclosure as a meaningful predicate in the relevant parts below.

BBSSAGA NIZK proving correct MAC computation. Consider $\text{NIP}_{\text{BBSSAGA}}$ (Section 4.2). This proof is a straightforward application of the Fiat-Shamir Schnorr framework above.

Statement. The prover and verifier compute $\mathfrak{z}_{\text{Schnorr}} := (X, (Y_1, \dots, Y_\ell), dA - G_0)$ (where the variables are from $\mathfrak{z}_{\text{BBSSAGA}}$ and pk_{SAGA}).

Witness. The prover sets the witness $\mathbb{w}_{\text{Schnorr}} := (x, y_1, \dots, y_\ell) := \mathbb{w}_{\text{BBSSAGA}}$.

Homomorphism. The homomorphism φ is defined as follows.

$$\begin{aligned} \varphi(x, y_1, \dots, y_\ell) = & \\ & (\\ & \quad xG, // = X \\ & \quad (y_1G_1, \dots, y_\ell G_\ell), // = (Y_1, \dots, Y_\ell) \\ & \quad - xA + \sum_{j \in [\ell]} y_j M_j // = dA - G_0 \\ &) \end{aligned}$$

With these values, the prover and verifier can compute and verify the Fiat-Shamir Schnorr-style proofs as described in Section C.1.

mKVAC_{CMZ} request NIZK. Consider NIP_R (Section 5). This proof is an application of the Fischlin Schnorr framework. For this proof, we need to handle the non-linear term $d \cdot r$ in PresWitVf . We do so by having the prover commit to d as $\text{ProdCom} = dG + \eta H$ and then prove that he can open $r \text{ProdCom}$ to $d \cdot r$. Furthermore, to make the homomorphism collision-resistant for the prover, we additionally commit to the attributes a_j as $\sum a_j G_j + \zeta G$, using that the G_j discrete logarithms are unknown to the user in when we instantiate **SAGA** with **BBSSAGA** (otherwise one would use a different base for the commitment).

The overall protocol NIP_R works as follows.

Statement. The prover computes additional commitments $\text{ProdCom} := dG + \eta H$ and $\text{Com} := \sum a_j G_j + \zeta G$ for random $\eta, \zeta \xleftarrow{\$} \mathbb{Z}_p$. The prover includes $\text{ProdCom}, \text{Com}$ in the final proof π_R and in the Fischlin hashes $\text{H}(\text{ProdCom}, \text{Com}, \dots)$. Both prover and verifier compute the homomorphism image as $\mathbb{x}_{\text{Schnorr}} := (\bar{X}_0 - C_{n+1}, \bar{Z}_0 - C_{n+2}, C_a, T, \text{ProdCom}, 0, \text{Com})$ (where T is from $\text{pres}_{\text{SAGA}}$, ProdCom is chosen by the prover and taken from the final proof by the verifier, and the other variables are from $\mathbb{x}_R$).

Witness. The prover computes the witness $\mathbb{w}_{\text{Schnorr}} := (s, (a_j, \xi'_j)_{j=1}^n, \bar{x}_0, \bar{\nu}, \text{wit}_{\text{SAGA}} = (r, d), \xi_1, \dots, \xi_{n+2}, \eta, \zeta, \text{prod}, \text{prod}')$, where $\xi'_j = a_j \xi_j$, the ξ_j being as output by the **SAGA** scheme, and $\text{prod} = d \cdot r, \text{prod}' = r \cdot \eta$.

Homomorphism. The homomorphism φ is defined as follows.

$$\begin{aligned} \varphi(s, (a_j, \xi'_j)_{j=1}^n, \bar{x}_0, \bar{\nu}, \text{wits}_{\text{SAGA}} = (r, d), \xi_1, \dots, \xi_{n+2}, \eta, \text{prod}, \text{prod}') = \\ (\\ & - \xi_{n+1}G_{n+1} + \bar{x}_0G + \bar{\nu}E, \parallel = \bar{X}_0 - C_{n+1} \\ & - \xi_{n+2}G_{n+2} + \bar{\nu}G, \parallel = \bar{Z}_0 - C_{n+2} \\ & sG + (\sum_{j \in [n]} a_j C_j) - (\sum_{j \in [n]} \xi'_j G_j), \parallel = C_a \\ & rX - dC_A + \text{prod}G - \sum_{j=1}^{\ell} \xi_j Y_j, \parallel = T \parallel \text{SAGA check} \\ & dG + \eta H \parallel = \text{ProdCom}, \parallel \text{product check} \\ & \text{prod}G + \text{prod}'H - r\text{ProdCom} \parallel = 0 \parallel \text{product check} \\ & \sum_{j \in [n]} a_j G_j + \zeta G \parallel = \text{Com} \parallel \text{commitment to make } \varphi \text{ coll. res.} \\ &) \end{aligned}$$

Note that φ is collision-resistant for the prover assuming the discrete logarithms of G_j, H, E, X, Y_j are hard to compute for the prover. With these values, the prover and verifier can compute and verify the Fischlin Schnorr-style proofs as described in Section C.2.

As mentioned above, this description assumes a trivial presentation predicate ϕ . As an important example, partial disclosure can be implemented as follows: Assume ϕ is such that a_1 is disclosed. The prover would omit a_1 from the witness w_R . The C_a in the statement $\mathfrak{x}_{\text{Schnorr}}$ is changed to $C_a - a_1 C_1$ (which both prover and verifier can compute knowing the disclosed a_1). The homomorphism φ would omit the term $a_1 C_1$ from the sum. Regarding performance, the change towards partially disclosing a_1 makes the proof π_R smaller by one $\mathbb{Z}_p$ element, corresponding to the witness a_1 . Computational costs do not materially change.

mKVAC_{CMZ} issuance NIZK. Consider NIP_I (Section 5). This proof is straightforward to implement. There is a non-linear term $u \cdot e$ in the relation, but since we have uG and eG publicly available, this is easy to handle.

Statement. Prover and verifier compute $\mathfrak{x}_{\text{Schnorr}} := (\bar{U}, E, 0, \bar{V})$ (where the variables are from ipk and $\mathfrak{x}_I$).

Witness. The prover sets the witness $w_{\text{Schnorr}} := (e, u, \text{prod})$, where $\text{prod} := e \cdot u$ and e, u are from w_I .

Homomorphism. The homomorphism φ is defined as follows.

$$\begin{aligned} \varphi(e, u, prod) = & \\ (& \\ & uG, // = \bar{U} \\ & eG, // = E \\ & prodG - uE, // = 0 \\ & u\bar{X}_0 - prod\bar{Z}_0 + uC_a // = \bar{V} \\ &) \end{aligned}$$

With these values, the prover and verifier can compute and verify the Fiat-Shamir Schnorr-style proofs as described in Section C.1.

mKVAC_{CMZ} presentation NIZK. Consider NIP_S (Section 5). This proof is a Fiat-Shamir OR proof. We first describe the main parts of the proof and then explain to combine them into an OR proof.

Statement. Prover and verifier compute $\mathfrak{x}_{\text{Schnorr}}^{(1)} := (Z, (C_j)_{j=1}^n)$ (where the variables are from $\mathfrak{x}_S$) and $\mathfrak{x}_{\text{Schnorr}}^{(2)} := X_1$ (from vpk).

Witness. The prover sets the witness $\mathfrak{w}_{\text{Schnorr}}^{(1)} := (\tilde{\gamma}, (a_j, \gamma_j)_{j=1}^n)$ from $\mathfrak{w}_S$. The witness $\mathfrak{w}_{\text{Schnorr}}^{(2)} = \perp$ is unused for honest users.

Homomorphism. The homomorphism $\varphi^{(1)}$ is defined as follows.

$$\begin{aligned} \varphi^{(1)}(\tilde{\gamma}, (a_j, \gamma_j)_{j=1}^n) = & \\ (& \\ & \sum_{j=1}^n \gamma_j X_j - \tilde{\gamma} H, // = Z \\ & (a_j \tilde{U} + \gamma_j G)_{j=1}^n // = (C_j)_{j=1}^n \\ &) \end{aligned}$$

The homomorphism $\varphi^{(2)}$ is defined as follows.

$$\varphi^{(2)}(x_1) = x_1 G // = X_1$$

With these values, we can construct the proper OR proof following [CDS94] as follows. We only describe the honest user case here, i.e., the prover knows a witness for the first statement. The prover then proceeds as follows.

1. Derive $\mathfrak{x}_{\text{Schnorr}}^{(1)}, \mathfrak{x}_{\text{Schnorr}}^{(2)}, \mathfrak{w}_{\text{Schnorr}}^{(1)}$ from $\mathfrak{x}$ as above.
2. Compute a simulated transcript for the second statement: pick random challenge $c^{(2)} \xleftarrow{\$} \mathbb{Z}_p$ and response $s^{(2)} \xleftarrow{\$} \mathbb{Z}_p$, and set the announcement $A_{\text{Schnorr}}^{(2)}$ to the unique accepting value $A_{\text{Schnorr}}^{(2)} := \varphi^{(2)}(s^{(2)}) - c^{(2)} \cdot \mathfrak{x}_{\text{Schnorr}}^{(2)}$.

3. Generate a random $a^{(1)} \xleftarrow{\$} \mathbb{Z}_p^{\ell_w}$ and compute the announcement $A_{\text{Schnorr}} = \varphi^{(1)}(a^{(1)}) \in \mathbb{G}^{\ell_i}$.
4. Compute the challenge $c := \text{H}(\text{ProtName}, \text{ctx}_{\text{NIP}}, \mathbb{X}, A_{\text{Schnorr}}^{(1)}, A_{\text{Schnorr}}^{(2)}) \in \mathbb{Z}_p$.
5. Derive the challenge for the first protocol as $c^{(1)} := c - c^{(2)}$.
6. Compute the first protocol response $s^{(1)} := a^{(1)} + c^{(1)} \cdot \mathbb{w}_{\text{Schnorr}}^{(1)} \in \mathbb{Z}_p^{\ell_w}$.
7. Output the proof $\pi := (c^{(1)}, c^{(2)}, s^{(1)}, s^{(2)}) \in \mathbb{Z}_p^{2+\ell_w+1}$.

The verifier on input $(\mathbb{X}, \pi, \text{ctx}_{\text{NIP}})$, where $\pi = (c^{(1)}, c^{(2)}, s^{(1)}, s^{(2)}) \in \mathbb{Z}_p^{2+\ell_w+1}$, proceeds as follows. The verifier derives $\mathbb{X}_{\text{Schnorr}}^{(1)}, \mathbb{X}_{\text{Schnorr}}^{(2)}$ from $\mathbb{X}$ as described above. The verifier then proceeds as follows.

1. Compute the unique accepting announcements $A_{\text{Schnorr}}^{(1)} := \varphi^{(1)}(s) - c^{(1)} \cdot \mathbb{X}_{\text{Schnorr}}^{(1)}$ and $A_{\text{Schnorr}}^{(2)} := \varphi^{(2)}(s) - c^{(2)} \cdot \mathbb{X}_{\text{Schnorr}}^{(2)}$.
2. Accept the proof if and only if $c^{(1)} + c^{(2)} = \text{H}(\text{ctx}_{\text{NIP}}, \mathbb{X}, \mathbb{X}, A_{\text{Schnorr}}^{(1)}, A_{\text{Schnorr}}^{(2)})$.

As described for NIP_R above, one can implement nontrivial predicates ϕ by adding statements to the proof. To implement partial disclosure of a_1 , the prover would omit a_1 from $\mathbb{w}_{\text{Schnorr}}$, C_1 in $\mathbb{X}_{\text{Schnorr}}$ becomes $C_1 - a_1 \tilde{U}$, and the $a_j \tilde{U}$ term in the homomorphism $\varphi^{(1)}$ is omitted.

In order to apply the randomized Fischlin transform [Ks22, LR22], the underlying sigma protocol must satisfy strong special-soundness. Strong special soundness requires that there exists an efficient extractor such that given two valid transcripts (R, c, z) and (R, c', z') such that $(c, z) \neq (c', z')$, the extractor outputs a valid witness. NIP_R 's homomorphism φ is not collision-resistant in general, because the user chooses C_j (the blinded attributes) and C_A (the blinded MAC). However, forcing the user to Pedersen-commit to the requested attributes $\vec{a}$ and extending the homomorphism φ accordingly will make φ collision-resistant assuming Pedersen commitments are binding. Hence straightforward adaptations to NIP_R are required to apply Fischlin's transform.

D Additional Evaluation Discussion

We discuss Section 6 and Figure 13 in more detail here.

Linear scaling in attribute size. The dominant operations scale *linearly* in the attribute dimension $|\vec{a}|$: **Obt₁**, **Iss**, **Present**, and **VfPres**. The communication terms that carry per-attribute commitments and proof data (**creq**, **pres**) also increase linearly: for example, **pres** grows from 0.6 KiB at $|\vec{a}|=4$ to 6.2 KiB at $|\vec{a}|=64$. This matches the design of our relations in Section 5 and the Schnorr-style instantiations in Appendix C.

Issuance vs. presentation. For moderately large attribute sets ($|\vec{a}|=32$), issuance takes on the order of $33.02+26.93 \approx 59.95$ ms end-to-end (**Obt₁** + **Iss**), while a single present/verify is ≈ 2.59 ms / 1.96 ms, respectively. For the largest tested set ($|\vec{a}|=64$), issuance is ≈ 109.48 ms end-to-end, while present/verify is ≈ 3.7 ms

/ 3.0 ms. This is consistent with mKVAC’s design goal: each verifier key requires only a single issuance (non-interactive from the user’s perspective), after which the user can perform many efficient presentations without relying on a helper server.

SAGA overhead. The consolidated request payload `creq` grows linearly with $|\vec{a}|$ (e.g., 1.1 KiB at $|\vec{a}|=4$, 8.6 KiB at $|\vec{a}|=64$), while the credential size itself is always constant (0.1 KiB). This reflects the cost of cryptographically authenticating the verifier key while keeping it hidden from the issuer during issuance, as required by our model in Section 4. The overhead remains small and predictable.

Practicality. Overall, the measurements show that mKVAC is highly practical for deployment. Even with dozens of attributes, issuance costs are only a few tens of milliseconds, and presentations are in the low millisecond range, easily fast enough for interactive or real-time use. Communication overheads are modest: even at $|\vec{a}|=64$, the largest payload is just 168 KiB, well within what modern networks can handle efficiently. This makes mKVAC suitable for privacy-preserving credential systems with realistic attribute sizes, while also showing promising scalability to larger vectors.

For completeness, in Figure 18 we also provide benchmarks where proofs are heuristically instantiated using Fiat-Shamir instead of Fischlin’s transform. This improves issuance performance at the cost of provable security.

E Comparison to Existing Presentation Anonymity Definitions.

Presentation anonymity definitions differ in three aspects. One is whether the adversary may choose the issuer public key. This is permitted in our model as done by [CAHLT25, ST24, CKL⁺14], while others assume honestly generated keys [Orr25, CMZ14, CDDH19, FHS19, San20, HS21].

$ \vec{a} $	Credential issuance						Credential presentation		
	Obt ₁ (ms)	Iss (ms)	Obt ₂ (ms)	creq	bcred	cred	Present (ms)	VfPres (ms)	pres
4	3.41	2.53	0.69	1.1	0.2	0.1	1.09	0.80	0.6
8	4.11	3.27	0.68	1.6	0.2	0.1	1.21	0.85	1.0
32	9.21	8.19	0.69	4.6	0.2	0.1	2.59	1.96	3.2
64	15.27	14.64	0.70	8.6	0.2	0.1	3.70	2.97	6.2

Fig. 18. mKVAC benchmarks: **In this benchmark, the Fiat-Shamir transform was used heuristically in place of Fischlin’s transform.** Times are in milliseconds, and payload sizes are in KiB. (i) The three-step issuance pipeline Obt₁, issue_cred (Iss), Obt₂; (ii) presentation (Present) and verification (VfPres). Communication is broken down into the consolidated request payload `creq`, the blinding payload (`bcred`), the credential (`cred`), and the presentation (`pres`). Payloads are in KiB.

A second difference is how indistinguishability is achieved. We use a left-or-right oracle, requiring that presentations of different credentials in the same anonymity set be indistinguishable [FHS19, San20, HS21, ST24]. Simulator-based definitions [Orr25, CAHLT25, CKL⁺14, CMZ14, CDDH19], by contrast, generate a simulated presentation from vpk and ϕ that is indistinguishable from a real one. These are particularly useful for certain predicates, e.g. predicates involving encrypted attributes, where the simulator can simulate presentations without learning the plaintext — something not possible by just relying on a left-or-right oracle.

The third aspect is whether presentation anonymity holds if credentials of honest users are revealed to the adversary. This is crucial in practice, since compromising a credential should not allow linking past presentations. Simulator-based models do not provide this guarantee, and some left-or-right definitions also avoid revealing credentials [FHS19, San20, HS21]. Our model always reveals issued credentials, so anonymity relies only on the randomness of the presentation algorithm, similar to Sanders and Traoré [ST24].

F Deferred Content of Section 3

Definition 19 (Correctness). *A scheme mKVAC as in Definition 2 is correct if for all security parameters $\lambda \in \mathbb{N}$, all possible groups $(\mathbb{G}, p, G) \in [\text{GGen}(1^\lambda)]$, all attribute length parameters n , and all attribute vectors $\vec{a} \in \mathbb{Z}_p^n$:*

$$\Pr \left[\begin{array}{l} \text{Setup}(\lambda, n) \rightarrow \text{pp}, \\ \text{IKg}(\text{pp}) \rightarrow (\text{isk}, \text{ipk}), \text{VKg}(\text{isk}) \rightarrow (\text{vsk}, \text{vpk}), \\ \text{Obt}_1(\text{ipk}, \text{vpk}, \vec{a}, \phi) \rightarrow (\text{creq}, \text{st}), \\ \text{Iss}(\text{isk}, \phi, \text{creq}) \rightarrow \text{bcred}, \text{Obt}_2(\text{st}, \text{bcred}) \rightarrow \text{cred}, \\ \text{Present}(\text{ipk}, \text{vpk}, \text{cred}, \text{ctx}, \phi) \rightarrow \text{pres} : \\ \text{VfPres}(\text{vsk}, \text{ipk}, \text{pres}, \text{ctx}, \phi) = \text{true} \end{array} \right] = 1.$$

G Security proofs of $\text{mKVAC}_{\text{CMZ}}$

G.1 Proof of unforgeability

Theorem 4 (restated). *$\text{mKVAC}_{\text{CMZ}}$ scheme from Figures 10 to 12 is unforgeable if NIP_I is zero-knowledge, NIP_R is complete and straightline extractable, and NIP_S is complete, zero-knowledge, and rewinding-based simulation-extractable, DDH is hard, and SAGA and CMZ are correct and unforgeable.*

Because our proof strategy involves rewinding, some game hops will not be indistinguishable up to a negligible term. Instead, every game hop will ensure that “if $\Pr[\text{Game}_{i+1}]$ is negligible, then $\Pr[\text{Game}_i]$ is negligible as well”. At the end of the proof, we show that the last game can only be won with negligible probability, which then transitively implies that the original game can only be won with negligible probability as well.

Proof (mKVAC_{CMZ} unforgeability.). We set the extractor E to run the (straight-line) knowledge extractor of NIP_R and output the extracted attributes. We prove Theorem 4 by first applying a sequence of game hops $Game_i$, then apply analysis to the last game. The event that the adversary $\mathcal{A}$ wins in $Game_i$ is denoted as $\mathbf{W}_i$. $Game_0$ is identical to the original unforgeability game.

$Game_1$: This game hop computes some NIP proofs by running their zero-knowledge simulators. Specifically,

- π_I when answering $\mathcal{OCCIssue}$ queries
- π_S when answering $\mathcal{OHCSHOW}$ queries

are simulated by their respective zero-knowledge simulators. Note that in $Game_0$, given correctness of the SAGA scheme, the game always uses valid witnesses to compute these proofs. These changes are indistinguishable by relying on the zero-knowledge properties of NIP_I and NIP_S , so $\Pr[\mathbf{W}_0] \leq \Pr[\mathbf{W}_1] + \text{Adv}_{\mathcal{A}_{ZK,I}, NIP_I}^{ZK}(\lambda) + \text{Adv}_{\mathcal{A}_{ZK,S}, NIP_S}^{ZK}(\lambda)$. In particular, if $\Pr[\mathbf{W}_1]$ is negligible, then $\Pr[\mathbf{W}_0]$ is negligible as well.

$Game_2$: This game hop aborts if E fails to extract a valid witness from a valid π_R . If $Game_2$ aborts with non-negligible probability, then we can easily build an adversary $\mathcal{A}_{SL,R}$ against straightline extractability of NIP_R . Thus, we conclude that $\Pr[\mathbf{W}_1] \leq \Pr[\mathbf{W}_2] + \text{Adv}_{\mathcal{A}_{SL,R}, NIP_R}^{SL}(\lambda)$. In particular, if $\Pr[\mathbf{W}_2]$ is negligible, then $\Pr[\mathbf{W}_1]$ is negligible as well.

$Game_3$: This game hop ensures that any adversarially chosen verifier keys vpk correspond to keys generated by the game. Intuitively, this is guaranteed by unforgeability of the SAGA scheme. To formally define this guarantee, $Game_3$ introduces a book-keeping set VPKSet which is initially empty. Then $Game_3$ updates VPKSet whenever it runs VKg algorithm. Specifically, when the challenge $\text{vpk}^* := (X_1^*, \dots, X_n^*, X_0^*, Z_0^*, \tau^*)$ is created, VPKSet is updated as $\text{VPKSet} := \text{VPKSet} \cup \{\text{vpk}^*\}$. Similarly, for each $\mathcal{OVKg}$ query, the output vpk is added to $\text{VPKSet} := \text{VPKSet} \cup \{\text{vpk}\}$.

We additionally define the following notation:

- During a $\mathcal{OCCIssue}$ query, let $(\mathbf{x}_R, \mathbf{w}_R)$ be the statement and extracted witness for the adversary's π_R .
- During a $\mathcal{OHCSHOW}(\text{vpk}, a, \phi)$ query, let $\mathbf{x}_R, \mathbf{w}_R$ be as computed by Obt_1 .

In either case, we write $\mathbf{x}_R =: (C_1, \dots, C_{n+2}, \bar{X}_0, \bar{Z}_0, C_a)$ and $\mathbf{w}_R =: (s, (a_j, \xi'_j)_{j=1}^n, \bar{x}_0, \bar{v}, \text{wits}_{\text{SAGA}}, \xi_1, \dots, \xi_{n+2})$. We further denote the effective (unblinded) SAGA message $X_j := C_j - \xi_j G_j$ for $j \in [n]$, $X_0 := C_{n+1} - \xi_{n+1} G_{n+1}$, $Z_0 := C_{n+2} - \xi_{n+2} G_{n+2}$.

$Game_3$ aborts if during a $\mathcal{OCCIssue}$ or $\mathcal{OHCSHOW}$ query, $(\mathbf{x}_R, \mathbf{w}_R) \in \mathcal{R}_R$ but there is no entry of the form $(X_1, \dots, X_n, X_0, Z_0, \cdot)$ in VPKSet .

Transition Game₂ → Game₃: We prove this transition by reduction, building a SAGA forger $\mathcal{R}_{\text{UF-CoMA}}$ which runs Game₃ identically except the following changes. $\mathcal{R}_{\text{UF-CoMA}}$ receives a public key pk_{SAGA} , which it embeds into ipk^* . In $\mathcal{OVKg}$, instead of running SAGA.MAC on $(X_1 := x_1G, \dots, X_n := x_nG, X_0 := (x_0 + e \cdot \nu)G, Z_0 := \nu G)$, $\mathcal{R}_{\text{UF-CoMA}}$ makes an $\mathcal{OMAC}((x_1, \dots, x_n, x_0 + e \cdot \nu, \nu))$ query. Similarly, in $\mathcal{OHClsue}$ and $\mathcal{OCClsue}$, instead of running $\text{SAGA.PresKeyedVf}(\text{isk}, \text{pres}_{\text{SAGA}}, \text{ctx}, C_1, \dots, C_{n+2})$, $\mathcal{R}_{\text{UF-CoMA}}$ makes an $\mathcal{OPresKeyedVf}(\text{pres}_{\text{SAGA}}, \text{wits}_{\text{SAGA}}, (C_j, \xi_j)_{j \in [n+2]})$ query, where $\text{wits}_{\text{SAGA}}$ and ξ_j are obtained from $\mathbb{w}_R$.

If any $\mathcal{OHClsue}$ or $\mathcal{OCClsue}$ query triggers the abort condition of Game₃, $\mathcal{R}_{\text{UF-CoMA}}$ outputs its final UF-CoMA forgery $(\text{pres}_{\text{SAGA}}, \text{wits}_{\text{SAGA}}, (C_j, \xi_j)_{j \in [n+2]})$. By definition, this is a valid forgery: $\text{SAGA.PresKeyedVf}(\text{pres}_{\text{SAGA}}, \text{wits}_{\text{SAGA}}, (C_j, \xi_j)_{j \in [n+2]}) = \text{true}$, as guaranteed by $(\mathbb{x}_R, \mathbb{w}_R) \in \mathcal{R}_R$, and we have not queried for a SAGA tag on $(X_1, \dots, X_n, X_0, Z_0)$, as $(X_1, \dots, X_n, X_0, Z_0) \notin \text{VPKSet}$. Thus, we conclude that $\Pr[\mathbf{W}_2] \leq \Pr[\mathbf{W}_3] + \text{Adv}_{\mathcal{R}_{\text{UF-CoMA}}, \text{SAGA}}^{\text{UF-CoMA}}(\lambda)$. In particular, if $\Pr[\mathbf{W}_3]$ is negligible, then $\Pr[\mathbf{W}_2]$ is negligible as well.

Game₄: At the start of Game₄, the game chooses a random dummy attribute vector $a^{\text{rnd}} \xleftarrow{\$} \mathbb{Z}_p^n$. This game answers $\mathcal{OHCShow}$ queries using a MAC σ^{rnd} on a^{rnd} instead of the honest user's actual credential cred_i . More specifically, Game₄ handles $\mathcal{OHClsue}(i, \text{ctx}, \phi)$ queries with the following additional behavior:

- It parses vpk_i as $(X_1, \dots, X_n, X_0, Z_0, \tau)$.
- Because Game₃ aborts otherwise, we know that $(X_1, \dots, X_n, X_0, Z_0, \cdot) \in \text{VPKSet}$ and hence $X_1, \dots, X_n, X_0, Z_0$ were generated by the game.
- Thus, Game₄ is able to look up corresponding values $\text{vsk}_i = (x_1, \dots, x_n, x_0), \nu$ such that $X_j = x_jG$ for $j \in [n]$ and $(X_0, Z_0) = (x_0G + \nu E, \nu G)$ is a ciphertext of x_0G .
- It computes a MAC $\sigma^{\text{rnd}} := (U, V) \leftarrow \text{CMZ.Sign}(\text{sk} := (x_0, \dots, x_n), a^{\text{rnd}})$ on a^{rnd} .
- It presents σ^{rnd} on a^{rnd} instead of cred_i on a_i , i.e. it computes $\text{pres} \leftarrow \text{Present}(\text{ipk}^*, \text{vpk}, \text{cred}^{\text{rnd}} := (\sigma^{\text{rnd}}, a^{\text{rnd}}), \text{ctx}, \phi)$ instead of $\text{Present}(\text{ipk}^*, \text{vpk}, \text{cred}_i, \text{ctx}, \phi)$.

This modification does not change the adversary's view. This is because the output $\text{pres} = (\tilde{U}, Z, C_V, C_1, \dots, C_n, \pi_S)$ of $\text{Present}(\text{ipk}^*, \text{vpk}, \text{cred} = (U, V, a), \text{ctx}, \phi)$ is effectively independent of (U, V, a) : $\tilde{U}, C_V, C_1, \dots, C_n$ are uniformly independently random in $\mathbb{G}$ and Z is uniquely determined by these values and vsk to satisfy the verification equation. π_S is simulated (cf. Game₁) using only $\mathbb{x}_S$, which is independent of (U, V, a) .

Additionally, since this means that honest users' cred_i are now unused, Game₄ stops computing honest users' credentials during $\mathcal{OHClsue}$ queries. Specifically, for $\mathcal{OHClsue}$ queries, Game₄ does not run Iss or Obt_2 . Instead, it just checks that SAGA.Present does not output $\perp$ and that $\text{SAGA.PresKeyedVf}(\text{sk}_{\text{SAGA}}, \text{pres}_{\text{SAGA}}, \text{ctx}, C_1, \dots, C_{n+2})$. If the checks succeed, it

adds $(\text{vpk}, \text{cred} := \perp, a)$ to $\mathcal{HC}$. It is easy to see that in the previous game (Game_3), all other $\text{Obt}_1, \text{Iss}, \text{Obt}_2$ checks are guaranteed to succeed (given that both user and issuer are honest, $\text{NIP}_R, \text{NIP}_I$ are complete, and the SAGA is correct), hence checking $\text{pres}_{\text{SAGA}}$ is sufficient to model the same behavior as Game_3 .

Overall, neither change impacts the adversary's view or the win condition, hence $\Pr[\mathbf{W}_3] = \Pr[\mathbf{W}_4]$.

Game_5 : This game hop changes how the blinded credential values $\bar{U}, \bar{V}$ are computed when answering OCCIssue queries.

For an OCCIssue query with $\text{creq} := (\text{pres}_{\text{SAGA}}, \mathbb{X}_R := (C_1, \dots, C_{n+2}, \bar{X}_0, \bar{Z}_0, C_a), \pi_R)$, we are using the notation of Game_3 , i.e. some variables below correspond to (extracted) witness values from π_R . Similar to Game_4 , the new Game_5 is able to look up secret values $\text{vsk}_i = (x_1, \dots, x_n, x_0), \nu$ corresponding to vpk such that $X_j = x_j G$ for $j \in [n]$ and $(X_0, Z_0) = (x_0 G + \nu E, \nu G)$ is a ciphertext of $x_0 G$. This is possible because Game_3 aborts if $(X_1, \dots, X_n, X_0, Z_0, \cdot) \notin \text{VPKSet}$.

In Game_4 , the (blinded) credential response $(\bar{U}, \bar{V})$ is computed as

$$\bar{U} := uG \quad \bar{V} := u((\bar{X}_0 - e\bar{Z}_0) + C_a)$$

for a random $u \in \mathbb{Z}_p$.

In Game_5 , we instead compute $(\bar{U}, \bar{V})$ as follows. Game_5 computes a fresh MAC $(\bar{U}, V') \leftarrow \text{CMZ.Sign}(\text{sk} := (x_0, \dots, x_n), (a_1, \dots, a_n))$. Then it computes $\bar{V}$ as

$$\bar{V} := V' + \left(\bar{x}_0 + s + \sum_{j \in [n]} td_j(a_j \xi_j - \xi'_j) \right) \bar{U}$$

by using the trapdoor information td_j from SAGA such that $G_j = td_j G$. The distribution of $(\bar{U}, \bar{V})$ is identical to Game_4 , so $\Pr[\mathbf{W}_5] = \Pr[\mathbf{W}_4]$. Intuitively, while Game_4 takes the *blinded request* and applies the *MAC key* (hidden in ciphertext (X_0, Z_0)) to it, Game_5 instead takes a *valid MAC* and applies the (*extracted*) *blinding terms* to it.

Game_6 : This game changes how the ElGamal ciphertext (X_0^*, Z_0^*) in the honest verifier public key vpk^* is computed. Instead of computing the ciphertext honestly as $(x_0^* G + \nu^* E, \nu^* G)$, it is computed as $(\alpha^* G, \beta^* G)$ for random $\alpha^*, \beta^* \in \mathbb{Z}_p$.

Transition $\text{Game}_5 \rightarrow \text{Game}_6$: This change is indistinguishable under the DDH assumption. Specifically, if an adversary can distinguish this change then we can build a DDH distinguisher $\mathcal{R}_{\text{DDH}}$ as follows. Given a DDH challenge $(G, A_{\text{DDH}}, B_{\text{DDH}}, C_{\text{DDH}})$, we set $E := A_{\text{DDH}}$, and $(X_0^*, Z_0^*) := (B_{\text{DDH}}, x_0^* G + C_{\text{DDH}})$. Note that the value e (the discrete logarithm of E and the issuer's credential-issuance secret) is unused in Game_5 : since Game_4 , OHCIssue does not create a credential anymore and in particular does not use e ; since Game_5 , OCCIssue uses extracted blinding terms to reply instead of using e . If C_{DDH}

is the real DH solution, then (X_0^*, Z_0^*) is set as in **Game₅** (proper ElGamal ciphertext of x_0^*G). Otherwise, (X_0^*, Z_0^*) is set as random group elements. Thus, we conclude that $\Pr[\mathbf{W}_5] \leq \Pr[\mathbf{W}_6] + \text{Adv}_{\mathcal{R}^{DDH}}^{DDH}(\lambda)$. In particular, if $\Pr[\mathbf{W}_6]$ is negligible, then $\Pr[\mathbf{W}_5]$ is negligible as well.

Game₇: This and the following games deal with extracting a witness from π_S^* via rewinding. As a first step, **Game₇** aborts if the candidate forgery presentation proof π_S^* belongs to a programmed random oracle query. More precisely, let **GetChalHashInput** be as in Definition 17. Then, after $\mathcal{A}$ outputs $(\text{pres}^*, \text{ctx}^*, \phi^*)$ as the candidate forgery, **Game₇** computes $x^* := \text{GetChalHashInput}(\mathfrak{x}^*, \text{ctx}_{\text{NIP}}^*, \text{NIP}_S^*)$, where $\mathfrak{x}^*, \text{ctx}_{\text{NIP}}^*, \text{NIP}_S^*$ are as in $\text{mKVAC}_{\text{CMZ}}.\text{VfPres}(\text{vsk}^*, \text{ipk}^*, \text{pres}^*, \text{ctx}^*, \phi^*)$. If the first random oracle query involving x^* was a programming query, **Game₇** aborts.

Transition Game₆ $\rightarrow$ Game₇: The only difference between **Game₆** and **Game₇** occurs if the adversary wins and the random oracle was programmed at x^* . This cannot happen. Assuming the winning condition holds, we know that $(\text{ctx}^*, \phi^*) \notin \mathcal{HP}$, i.e., there was no honest user presentation $\mathcal{OHCShow}$ query for $\text{ctx}^*, \phi^*, \text{vpk}^*$. As a consequence, the π_S simulator was never run with a statement $\mathfrak{x}$ containing ϕ^* and context ctx_{NIP} containing $\text{vpk}^*, \text{ctx}^*$. This together with the first property of Definition 17 implies that all simulated proofs π' for $\mathfrak{x}', \text{ctx}'_{\text{NIP}}$ have a different challenge hash preimage $\text{GetChalHashInput}(\mathfrak{x}', \text{ctx}'_{\text{NIP}}, \pi') \neq x^*$. In particular, due to the second property of Definition 17, the simulator never programmed the random oracle at x^* . Hence $\Pr[\text{Game}_7] = \Pr[\text{Game}_6]$.

Game₈: This game introduces rewinding. Specifically, let **GetChalHashInput** be as in Definition 17. During every random oracle query, **Game₈** takes a snapshot of the adversary's and its own state. Eventually, if $\mathcal{A}$ wins the game, then **Game₈**, using the hash preimage x^* (which was not programmed, see **Game₇**) as in **Game₇**, chooses a new random hash value $v^{**} \xleftarrow{\$} \mathbb{Z}_p$, sets the random oracle response on x^* to v^{**} , and executes the adversary and game starting with the state from the snapshot taken during the first random oracle query for x^* . Note that **Game₈** uses fresh randomness for the second execution starting at the forking point. The second execution ends with the adversary outputting $(\text{pres}^{**}, \text{ctx}^{**}, \phi^{**})$. If the adversary wins this execution as well and $x^{**} := \text{GetChalHashInput}(\mathfrak{x}^{**}, \text{ctx}_{\text{NIP}}^{**}, \text{NIP}_S^{**})$ is the same as x^* , then **Game₈** outputs 1, otherwise it aborts.

A straightforward application of the forking lemma (cf. Lemma 2) shows that $\Pr[\mathbf{W}_8] \geq \frac{\Pr[\mathbf{W}_7]^2}{q}$, where q is a polynomial upper bound on the number of random oracle queries pertaining to π_S . Notably, this makes $\mathbf{W}_7$ significantly more likely than $\mathbf{W}_8$, i.e., this game hop is not indistinguishable. However, it holds that if $\Pr[\mathbf{W}_8]$ is negligible, then $\Pr[\mathbf{W}_7]$ must be negligible as well. To fit our description of **Game₈**, we use the forking lemma in a way that on the fork, the algorithm uses fresh randomness (instead of changing only hash values). This is covered by Lemma 2 by setting $\mathcal{H} := \mathbb{Z}_p \times \{0, 1\}^{t(\lambda)}$ for some polynomial t , such

that each h_i contains both a hash value and a random string for the algorithm to use after the random oracle query. We note that using fresh randomness on the fork will be instrumental for the final reduction to CMZ unforgeability, because that reduction will need to consult its (MAC) oracles, which will inevitably use fresh randomness to answer, in order to simulate the game.

Game₉: This game executes witness extraction and aborts if it fails. More specifically, let **GetWitness** be as in Definition 17. Right before **Game₈** would conclude, **Game₉** computes the witness w_S^* as **GetWitness**(x_S^* , ctx_{NIP}^* , NIP_S^* , NIP_S^{**}) (using the values computed in **Game₈**). **Game₉** aborts if w_S^* is not a valid witness for x_S^* (i.e., if $(x_S^*, w_S^*) \notin \mathcal{R}_S$).

Assume $v^* \neq v^{**}$. In that case, by the third property of Definition 17, w_S^* is guaranteed to be a valid witness for x_S^* . Consequently, $\Pr[\mathbf{W}_8] \leq \Pr[\mathbf{W}_9] + \Pr[v^* = v^{**}] = \Pr[\mathbf{W}_9] + 1/p$. In particular, if $\Pr[\mathbf{W}_9]$ is negligible, then $\Pr[\mathbf{W}_8]$ is negligible as well.

Game₁₀: This game aborts if the extracted valid witness $w_S^* =: ((\tilde{\gamma}^*, (a_j^*, \gamma_j^*)_{j=1}^n), x_1^*)$ contains the random dummy attribute vector a^{rnd} chosen in **Game₄**, i.e., **Game₁₀** aborts if $\tilde{a}^* = a^{\text{rnd}}$.

It is easy to see that a^{rnd} is only used in **OHCSHOW** queries and affects the rest of the game (/adversary behavior) only through the presentation token **pres** that **OHCSHOW** outputs. As argued in **Game₄**, the distribution of **pres** is independent of a^{rnd} . Consequently, the probability for extraction to yield exactly $a^{\text{rnd}} \in \mathbb{Z}_p^n$ is at most $1/p^n$, leading us to conclude that $\Pr[\mathbf{W}_9] \leq \Pr[\mathbf{W}_{10}] + p^{-n}$. In particular, if $\Pr[\mathbf{W}_{10}]$ is negligible, then $\Pr[\mathbf{W}_9]$ is negligible as well.

Analysis of Game₁₀: We will conclude our proof by showing that due to unforgeability of the CMZ MAC, the chances for $\mathcal{A}$ winning in **Game₁₀** must be negligible. For this, we build an efficient CMZ-Unf adversary $\mathcal{A}_{\text{CMZ}}$. $\mathcal{A}_{\text{CMZ}}$ runs **Game₁₀** with the following changes. $\mathcal{A}_{\text{CMZ}}$ receives $X_1^*, \dots, X_n^*$ from its CMZ-Unf challenger and incorporates them into the verifier key vsk^* . Note that vsk^* is unknown to $\mathcal{A}_{\text{CMZ}}$. To cope with that, when simulating oracles of **Game₁₀**, $\mathcal{A}_{\text{CMZ}}$ makes the following changes.

- When answering **OCCLIssue** queries involving vsk^* , instead of running **CMZ.Sign**(vsk^*, a) (cf. **Game₅**), it makes a query **OSign**(a) to its own oracle.
- When answering **OHCSHOW** queries involving vsk^* , instead of running **CMZ.Sign**($\text{vsk}^*, a^{\text{rnd}}$) (cf. **Game₄**), it makes a query **OSign**(a^{rnd}) to its own oracle.
- When answering **OVfPres** queries, instead of checking the equation $Z = x_0 \tilde{U} + \vec{x} \cdot \vec{C} - C_V$ within **mKVAC_{CMZ}.VfPres** (cf. Figure 12) using $x_0, \vec{x}$ from vsk^* , our adversary $\mathcal{A}_{\text{CMZ}}$ queries, **OHLP**($\tilde{U}, \vec{C}, Z + C_V$).

These changes also apply in the rewinding run that **Game₁₀** performs after $\mathcal{A}$ outputs its forgery (see **Game₈**). If **Game₁₀** is won, it computes a valid witness $w_S^* := ((\tilde{\gamma}^*, (a_j^*, \gamma_j^*)_{j=1}^n), x_1^*)$. In that case, $\mathcal{A}_{\text{CMZ}}$ proceeds as follows.

If $x_1^*G = X_1^*$, then $\mathcal{A}_{\text{CMZ}}$ has learned part of the CMZ secret key and can easily output a CMZ forgery by taking any valid CMZ tag $(U, V = x_0U + \sum_j m_j x_j U)$ on $(m_1, m_2, \dots, m_n)$ and computing the valid tag $(U, V + rx_1U)$ on $(m_1 + r, m_2, \dots, m_n)$ for any $r \in \mathbb{Z}_p$. With this insight, $\mathcal{A}_{\text{CMZ}}$ can easily win its forgery game.

Otherwise, if $x_1^*G \neq X_1^*$, then the $(\tilde{\gamma}^*, (a_j^*, \gamma_j^*)_{j=1}^n)$ part of the witness w_S^* must be satisfying $\mathcal{R}_S$. In this case, $\mathcal{A}_{\text{CMZ}}$ computes its forgery as $(\sigma^* := (\tilde{U}^*, C_V^* - \tilde{\gamma}^*H), a^* = (a_1^*, \dots, a_n^*))$. Notice that σ^* is a valid CMZ tag on a^* . Furthermore, the forgery is non-trivial: $\mathcal{A}_{\text{CMZ}}$ only made $\mathcal{O}\text{Sign}$ queries for $a \in \mathcal{CC}$ (see **Game**₅) and for a^{rnd} (see **Game**₄). We know that $a^* \notin \mathcal{CC}$ from the winning condition (no attributes a with $\phi^*(a) = \text{true}$ are in $\mathcal{CC}$. This is true for both the main branch of the game and for the rewinding branch, which have different sets $\mathcal{CC}$, but in either case, $a^* \notin \mathcal{CC}$ if $\mathcal{A}$ wins both branches, which is required for **Game**₁₀ to attempt witness extraction) and we know $a^* \neq a^{\text{rnd}}$ because otherwise the game would have aborted (cf. **Game**₁₀).

Consequently, whenever **Game**₁₀ is won, then $\mathcal{A}_{\text{CMZ}}$ wins, too. Hence $\Pr[\text{Game}_{10}] \leq \text{Adv}_{\mathcal{A}_{\text{CMZ}}}^{\text{CMZ-Unf}, n}(\lambda)$. In particular, since we assume CMZ is unforgeable, $\Pr[\text{Game}_{10}]$ must be negligible.

Because $\Pr[\mathbf{W}_{10}]$ is negligible, $\Pr[\mathbf{W}_i]$ is negligible for all $i \in \{0, \dots, 9\}$ by the above analysis of the game hops. In particular, $\Pr[\mathbf{W}_0]$ is negligible, which is what we needed to show. $\square$

G.2 Proof of presentation anonymity

Theorem 5 (restated). *mKVAC_{CMZ} has presentation anonymity if NIP_I is sound and NIP_S is zero-knowledge.*

Proof. We change how the adversary's view is simulated step by step, and end up in a simulation, independent of the challenge bit. First, the NIP_R proofs in credentials are simulated via the zero-knowledge simulator. Then **Game**₂ ensures that all non- $\perp$ credentials contain a valid CMZ tag. This is done by finding the witness of the NIP_I proofs in **bcreds** which are received from the adversary. We show that the adversary can set such a non- $\perp$ credential with an invalid CMZ tag only if it is able to break the soundness of NIP_I. Finally, **Game**₃ simulates the presentation in **OLoR** without using the credentials or attributes. This is done by utilizing the **vpk** values and the ElGamal decryption key e that **Game**₂ already finds through the exhaustive search. This concludes our proof as the challenge bit b is not used at all in this game.

Game₁: This game is identical to the original game except that it simulates the NIP_S proofs that are created as part of the presentations in the **OLoR** oracle. By relying on the zero-knowledge property of NIP_I, we conclude that $|\Pr[\mathbf{W}_1] - \Pr[\mathbf{W}_0]| \leq \text{Adv}_{\text{NIP}_I}^{\text{ZK}}(\lambda)$.

Game₂: This game checks the validity of the CMZ tags in each non- $\perp$ credential by finding the witness of the corresponding NIP_I issue proof. If the CMZ is not valid, the game aborts by simulating the adversary's guess randomly. Below

we formally define the abort condition of the game. Let $\text{ipk} := (\text{pk}_{\text{SAGA}}, E)$ and for $c' \in [c]$, $\mathcal{HC}[c']$ be parsed as

$$\left(\text{cred}^{(c')} := (U^{(c')}, V^{(c')}, \vec{a}^{(c')}), \quad \vec{a}^{(c')}, \quad \text{vpk} := (\vec{X}^{(c')}, X_0^{(c')}, Z_0^{(c')}, \tau^{(c')}) \right).$$

At the end of the game, for each $\mathcal{OObt}_2$ call, this game finds $(e, u^{(c')})$ via exhaustive search such that $E = eG$ and $U^{(c')} = u^{(c')}G$, and it checks if the following holds:

$$V^{(c')} = u^{(c')} \left(X_0^{(c')} - eZ_0^{(c')} + \vec{a}^{(c')} \cdot \vec{X}^{(c')} \right) \quad (1)$$

If it does not hold for c' , the game aborts by simulating the adversary's guess b' randomly.

Transition $\text{Game}_1 \rightarrow \text{Game}_2$. Let $\mathbf{F}_{\text{snd}}$ be the event that Equation 1 does not hold for at least one $\mathcal{OObt}_2$ call. Notice that if $\mathbf{F}_{\text{snd}}$ does not occur the two games are equivalent. Moreover, if $\mathbf{F}_{\text{snd}}$ occurs, the adversary has to forge a valid NIP_I proof for an invalid statement. Thus,

$$\begin{aligned} |\Pr[\mathbf{W}_2] - \Pr[\mathbf{W}_1]| &= \Pr[\mathbf{F}_{\text{snd}}] \cdot |\Pr[\mathbf{W}_2 \mid \mathbf{F}_{\text{snd}}] - \Pr[\mathbf{W}_1 \mid \mathbf{F}_{\text{snd}}]| \\ &\leq \Pr[\mathbf{F}_{\text{snd}}]. \end{aligned}$$

In the following we describe a soundness adversary $\mathcal{R}_{\text{snd}}$ against NIP_I soundness. $\mathcal{R}_{\text{snd}}$ has winning probability $q \cdot \Pr[\mathbf{F}_{\text{snd}}]$, which will conclude our proof. $\mathcal{R}_{\text{snd}}$ chooses a random $c^* \xleftarrow{\$} [c]$ randomly, and runs Game_1 against the adversary identically. At the end of the game, it returns $\pi_I^{(c^*)}$ which corresponds to the c^* -th $\mathcal{OObt}_2$ call of the adversary. If $\mathbf{F}_{\text{snd}}$ occurs, then at least one of the NIP_I proofs presented by the adversary must be a valid forgery in the soundness game, so $|\Pr[\mathbf{W}_2] - \Pr[\mathbf{W}_1]| \leq \Pr[\mathbf{F}_{\text{snd}}]$.

Game₃: This game changes how $\tilde{U}, Z, C_V, \vec{C}$ values are set in the returned presentations of $\mathcal{OLoR}$ calls. Specifically, it randomly samples $\gamma^* \xleftarrow{\$} \mathbb{Z}_p^*$, $\tilde{\gamma}^* \xleftarrow{\$} \mathbb{Z}_p$, $\tilde{\gamma}^* \xleftarrow{\$} \mathbb{Z}_p^n$, and sets

$$\tilde{U} := \gamma^* G, \quad C_V := \tilde{\gamma}^* G, \quad \vec{C} := \tilde{\gamma}^* G, \quad Z := \tilde{\gamma}^* \cdot \vec{X} - \tilde{\gamma}^* + \gamma^* (X_0 - eZ_0).$$

Transition $\text{Game}_2 \rightarrow \text{Game}_3$. we show that these two games are identically distributed. We show that this holds for a single for the $\mathcal{OLoR}$, implying it holds for the game via a sequence of hybrids. Let $\mathcal{HC}[c^{(b)}]$ be parsed as

$$\left(\text{cred}^{(b)} := (U^{(b)}, V^{(b)}, \vec{a}^{(b)}), \quad \vec{a}^{(b)}, \quad \text{vpk} := (\vec{X}, X_0, Z_0, \tau) \right)$$

Below we consider the distribution of $\tilde{U}, Z, C_V, \vec{C}$ in Game_2 .

$$\left\{ \begin{array}{l} \gamma, \tilde{\gamma} \xleftarrow{\$} \mathbb{Z}_p^*, \quad \tilde{\gamma} \xleftarrow{\$} \mathbb{Z}_p^n \\ \tilde{U} := \gamma U^{(b)} \\ Z := \tilde{\gamma} \cdot \vec{X} - \tilde{\gamma} H \\ C_V := \gamma V^{(b)} + \tilde{\gamma} H \\ \vec{C} := \vec{a}^{(b)} \cdot \tilde{U}^{(b)} + \tilde{\gamma} \cdot G \end{array} : \tilde{U}, Z, C_V, \vec{C} \right\}$$

For $U^{(b)} := u^{(b)}G$, $\vec{X} = \vec{x}G$, $x_0G := X_0 - eZ_0$, and $H := \theta G$ we can re-write the above distribution as

$$\left\{ \begin{array}{l} \gamma \xleftarrow{\$} \mathbb{Z}_p^*, \tilde{\gamma} \xleftarrow{\$} \mathbb{Z}_p, \vec{\gamma} \xleftarrow{\$} \mathbb{Z}_p^n \\ \tilde{U} := \gamma u^{(b)}G \\ Z := (\vec{\gamma} \cdot \vec{x} - \tilde{\gamma}\theta)G \\ C_V := (\gamma u^{(b)} \cdot (x_0 + \vec{a}^{(b)} \cdot \vec{x}) + \tilde{\gamma}\theta)G \\ \vec{C} := (\vec{a}^{(b)} \cdot \gamma u^{(b)} + \vec{\gamma}) \cdot G \end{array} : \tilde{U}, Z, C_V, \vec{C} \right\}$$

We first change this distribution by setting $\gamma := \gamma^*/u^{(b)}$ for $\gamma^* \xleftarrow{\$} \mathbb{Z}_p^*$. Notice that the distribution of γ does not change.

$$\left\{ \begin{array}{l} \gamma^* \xleftarrow{\$} \mathbb{Z}_p^*, \tilde{\gamma} \xleftarrow{\$} \mathbb{Z}_p, \vec{\gamma} \xleftarrow{\$} \mathbb{Z}_p^n \\ \tilde{U} := \gamma^*G \\ Z := (\vec{\gamma} \cdot \vec{x} - \tilde{\gamma}\theta)G \\ C_V := (\gamma^* \cdot (x_0 + \vec{a}^{(b)} \cdot \vec{x}) + \tilde{\gamma}\theta)G \\ \vec{C} := (\vec{a}^{(b)} \cdot \gamma^* + \vec{\gamma}) \cdot G \end{array} : \tilde{U}, Z, C_V, \vec{C} \right\}$$

Next we change how $\vec{\gamma}$ is computed. We set $\vec{\gamma} := \vec{\gamma}^* - \gamma^* \cdot \vec{a}^{(b)}$ for $\vec{\gamma}^*$. The distribution of $\vec{\gamma}$ does not change as $\vec{\gamma}^*$ is random.

$$\left\{ \begin{array}{l} \gamma^* \xleftarrow{\$} \mathbb{Z}_p^*, \tilde{\gamma} \xleftarrow{\$} \mathbb{Z}_p, \vec{\gamma}^* \xleftarrow{\$} \mathbb{Z}_p^n \\ \tilde{U} := \gamma^*G \\ Z := ((\vec{\gamma}^* - \gamma^* \cdot \vec{a}^{(b)}) \cdot \vec{x} - \tilde{\gamma}\theta)G \\ C_V := (\gamma^* \cdot (x_0 + \vec{a}^{(b)} \cdot \vec{x}) + \tilde{\gamma}\theta)G \\ \vec{C} := \vec{\gamma}^* \cdot G \end{array} : \tilde{U}, Z, C_V, \vec{C} \right\}$$

Next we compute $\tilde{\gamma}$ as $\tilde{\gamma} := \theta^{-1}(\vec{\gamma}^* - \gamma^*(x_0 + \vec{x} \cdot \vec{a}^{(b)}))$ for a random $\vec{\gamma}^* \xleftarrow{\$} \mathbb{Z}_p^n$.

$$\left\{ \begin{array}{l} \gamma^* \xleftarrow{\$} \mathbb{Z}_p^*, \tilde{\gamma}^* \xleftarrow{\$} \mathbb{Z}_p, \vec{\gamma}^* \xleftarrow{\$} \mathbb{Z}_p^n \\ \tilde{U} := \gamma^*G \\ Z := (\vec{x} \cdot \vec{\gamma}^* - \tilde{\gamma}^* + \gamma^*x_0)G \\ C_V := \tilde{\gamma}^*G \\ \vec{C} := \vec{\gamma}^* \cdot G \end{array} : \tilde{U}, Z, C_V, \vec{C} \right\}$$

Then we can re-write this as

$$\left\{ \begin{array}{l} \gamma^* \xleftarrow{\$} \mathbb{Z}_p^*, \tilde{\gamma}^* \xleftarrow{\$} \mathbb{Z}_p, \vec{\gamma}^* \xleftarrow{\$} \mathbb{Z}_p^n \\ \tilde{U} := \gamma^*G \\ Z := \vec{\gamma}^* \cdot \vec{X} - \tilde{\gamma}^* + \gamma^*(X_0 - eZ_0) \\ C_V := \tilde{\gamma}^*G \\ \vec{C} := \vec{\gamma}^* \cdot G \end{array} : \tilde{U}, Z, C_V, \vec{C} \right\}$$

which is identical to the distribution in Game_3 .

Finally, notice that the adversary does not use the bit b when answering $\mathcal{OLoR}$ calls in Game_3 . Thus, b is information theoretically hidden and we conclude that $\Pr[\mathbf{W}_3] = 1/2$. This gives us

$$\text{Adv}_{\text{mKVAC}_{\text{CMZ}}}^{\text{PAnon}}(\lambda) = \text{Adv}_{\text{NIP}_R}^{\text{ZK}}(\lambda) + c \cdot \text{Adv}_{\text{NIP}_I}^{\text{snd}}(\lambda).$$

G.3 Proof of issuance anonymity

Theorem 6 (restated). *mKVAC_{CMZ} scheme from Figures 10 to 12 satisfies the issuance anonymity property if NIP_I is rewinding-based extractable, NIP_R is zero-knowledge, and the underlying SAGA scheme satisfies privacy.*

Proof. Our proof aims to end up in a game hop in which the challenge bit is information theoretically hidden from the adversary. For this, we should not use b when sending credentials requests and the final credentials to the adversary. As the final credentials contain CMZ tags, it is computationally hard to just simulate them without the adversary's contribution. Our strategy is to *extract* the ElGamal decryption key e of the adversary from its Fiat-Shamir proofs in the blinded credentials. If we can extract e , then we can directly compute credentials without relying on the adversary's further responses. The remaining task would be decoupling the credentials requests sent to the adversary from the challenge bit. This can be performed by relying on the randomness of the included commitments in credentials requests (as done in Game₆) and the privacy property of the underlying SAGA scheme (as done in Game₇).

Ensuring that we can extract e with non-negligible property needs more complex argumentation. First, there may be an adversary which can win with non-negligible advantage without outputting correct Fiat-Shamir proofs in their blinded credentials. Specifically, consider a contrived mKVAC scheme which reveals attributes in plain as part of its credentials requests. An adversary would not need to create valid blinded credentials to win against such a scheme. Thus, we must ensure first that the algorithm to be rewinded forces the adversary to output valid proofs. For this, Game₂ adds an additional abort condition. Namely, if the adversary's responses does not result in valid final credentials, the game simulates the adversary's final guess by sampling a random bit. We show that this change is indistinguishable under the privacy property of the underlying SAGA scheme.

The next challenge is to ensure that the rewinded adversary will use the same e as the issuance anonymity game, yet it will have an independent view of the challenge bit b of the issuance anonymity game. This is ensured as follows. We do not rewind the game itself, but we run the forking algorithm on a wrapper algorithm $\mathcal{B}$ that contains a copy of the adversary and the issuance anonymity game. To be clear,

- The issuance anonymity game runs the adversary, obtains the challenge $\text{ipk}, \text{vpk}, \bar{a}$ values, and the adversary's internal state.
- Then a wrapper algorithm $\mathcal{B}$ takes all this state, including the adversary's random oracle queries up to that point as input. $\mathcal{B}$ runs a copy of the game with an independent challenge bit b^* .
- When the adversary outputs its blinded credentials, $\mathcal{B}$ looks up the corresponding random oracle query and returns it as the point of forking.

By Game₂, we know that a winning adversary with non-negligible probability must output a valid proof, so if the adversary has a non-negligible advantage,

Game ₁	Game ₂	Game ₃	Game ₄	Game ₅
$\text{pp} \leftarrow \text{Setup}(\lambda, n), \forall m : \text{ROT}_{FS}[m] := \perp$ $(\text{ipk}, \phi, (\text{vpk}^{(i)}, \vec{a}^{(i)})_{i \in \{0,1\}}, st_{\mathcal{A},1}) \leftarrow \mathcal{A}_1(\text{pp})$ $b \xleftarrow{\$} \{0, 1\}$				
$(out, out') \leftarrow \text{Fork}_{\mathcal{B}} \left((\text{pp}, \text{ipk}, \phi, (\text{vpk}^{(i)}, \vec{a}^{(i)})_{i \in \{0,1\}}, st_{\mathcal{A},1}) \right)$ if $(out, out') = (\perp, \perp)$: return $b = b' \xleftarrow{\$} \{0, 1\}$				
Extract e by using (out, out') . If fails, return $b = b' \xleftarrow{\$} \{0, 1\}$				
for $i \in \{0, 1\}$: $(\text{creq}^{(i)}, st^{(i)}) \leftarrow \text{Obt}'_1(\text{ipk}, \text{vpk}^{(i)}, \vec{a}^{(i)}, \phi)$ if $\neg \phi(\vec{a}^{(0)}) \vee \neg \phi(\vec{a}^{(1)}) \vee \perp \in \{\text{creq}^{(0)}, \text{creq}^{(1)}\}$: $b' \xleftarrow{\$} \{0, 1\}$				
elseif there is a ROT_{FS} entry with $\text{creq}^{(0)}$, then $b' \xleftarrow{\$} \{0, 1\}$				
else : $(\text{bcred}^{(b)}, \text{bcred}^{(1-b)}, st_{\mathcal{A},2}) \leftarrow \mathcal{A}_2(st_{\mathcal{A},1}, \text{creq}^{(b)}, \text{creq}^{(1-b)})$ for $i \in \{0, 1\}$: $\text{cred}^{(i)} \leftarrow \text{Obt}_2(st^{(i)}, \text{bcred}^{(i)})$ if $\perp \in \{\text{cred}^{(0)}, \text{cred}^{(1)}\}$: $b' \leftarrow \mathcal{A}_3(st_{\mathcal{A},2}, \perp, \perp)$ $b' \xleftarrow{\$} \{0, 1\}$ else : $b' \leftarrow \mathcal{A}_3(st_{\mathcal{A},2}, \text{cred}^{(0)}, \text{cred}^{(1)})$ return $b = b'$				
$\text{RO}_{FS}(m)$ if $\text{ROT}_{FS}[m] = \perp$: $\text{ROT}_{FS}[m] \xleftarrow{\$} \mathbb{Z}_p$ return $\text{ROT}_{FS}[m]$				

Fig. 19. Games for the issuance anonymity proof.

$\mathcal{B}$ does not fail with non-negligible probability. One issue would be that the adversary could make the random oracle query that is subject to be forked prior to the execution of the algorithm $\mathcal{B}$. We eliminate this case in **Game₃** by relying on the randomness that the credential requests contain. As a result of these steps, **Game₄** runs $\text{Fork}_{\mathcal{B}}$ and tries to extract e . The remaining games perform the regular steps explained before to decouple the adversary's view from the bit b . We provide a sequence of games below. Games **Game₁** to **Game₅** are given in Figure 19.

Game₁: This game first performs some notational changes.

- It notates the adversary $\mathcal{A} := (\mathcal{A}_1, \mathcal{A}_2, \mathcal{A}_3)$ with their explicit states.
- It notates the Fiat-Shamir challenge random oracle a RO_{FS} and explicitly notates the table it uses for the lazy sampling.

Both of these changes are made to increase the readability of the further game hops and they do not change the adversary's view.

$\text{Obt}'_1(\text{ipk}, \text{vpk}, \vec{a}, \phi)$ parse (ipk, vpk) as $((\text{pk}_{\text{SAGA}}, E), (\vec{X}, X_0, Z_0, \tau))$ $(\text{pres}_{\text{SAGA}}, \vec{C}, \vec{\xi}, \text{wits}_{\text{SAGA}}) \leftarrow \text{SAGA.Present}(\text{pk}_{\text{SAGA}}, \tau, (\vec{X}, X_0, Z_0))$ require SAGA.Present did not output $\perp$ $\bar{x}_0, \bar{\nu} \xleftarrow{\\$} \mathbb{Z}_p, \bar{X}_0 := X_0 + \bar{x}_0 G + \bar{\nu} E, \bar{Z}_0 := Z_0 + \bar{\nu} G$ $s \xleftarrow{\\$} \mathbb{Z}_p, C_a := \vec{a} \cdot \vec{X} + sG$ $\pi_R \leftarrow \text{NIP}_R.\text{S.Prove}(\mathbb{x}_R := (\vec{C}, \bar{X}_0, \bar{Z}_0, C_a))$ return $(\text{st} := (s, \bar{x}_0, \bar{X}_0, \bar{Z}_0, \vec{a}), \text{creq} := (\text{pres}_{\text{SAGA}}, \mathbb{x}_R, \pi_R))$
--

Fig. 20. Obt'_1 algorithm for the issuance anonymity proof's game hops.

The non-trivial change that this game makes is that the game runs the Obt'_1 algorithm given in Figure 20. Obt'_1 is identical to Obt_1 , but it runs the NIP_R proof simulator instead of running honest proving algorithm. This change is indistinguishable under the zero-knowledge property of NIP_R , so $|\Pr[\mathbf{W}_1] - \Pr[\mathbf{W}_0]| \leq \text{Adv}_{\text{NIP}_R}^{\text{ZK}}(\lambda)$.

Game₂: This game changes the behavior if the issuance of any credential fails (if any of them is equal to $\perp$). In this case, instead of running $\mathcal{A}_3$ on $(\perp, \perp)$ as credentials, it samples b' randomly. Intuitively, this game guarantees that seeing only the credentials requests does not help the adversary to guess the bit b correctly.

Transition Game₁ $\rightarrow$ Game₂. Let $\mathbf{F}_{\perp\text{-cred}}$ be the event that $\perp \in \{\text{cred}^{(0)}, \text{cred}^{(1)}\}$. For the case that $\mathbf{F}_{\perp\text{-cred}}$ does not occur, both games behave identically. Thus, we only consider the remaining case. Namely, we prove via a sequence of hybrids that $|\Pr[\mathbf{W}_1 \wedge \mathbf{F}_{\perp\text{-cred}}] - \Pr[\mathbf{W}_2 \wedge \mathbf{F}_{\perp\text{-cred}}]| = 2 \cdot \text{Adv}_{\text{SAGA}}^{\text{Priv}}(\lambda)$. Below $\mathbf{W}_{1,i}$ denotes the event that the adversary wins in hybrid $\mathcal{H}_{1,i}$ and $\mathbf{F}_{\perp\text{-cred}}$ occurs.

$\mathcal{H}_{1,1}$: Notice that

$$\text{creq}^{(i)} := (\text{pres}_{\text{SAGA}}^{(i)}, \mathbb{x}_R^{(i)} := (\vec{C}^{(i)}, \bar{X}_0^{(i)}, \bar{Z}_0^{(i)}, C_a^{(i)}), \pi_R^{(i)}).$$

This hybrid is identical to **Game₁**, except that it samples $\bar{X}_0^{(i)}, \bar{Z}_0^{(i)}, C_a^{(i)} \xleftarrow{\$} \mathbb{Z}_p$ randomly. This change is perfectly indistinguishable as these elements contain re-randomizer factors already. Furthermore, as we only analyze the case that $\mathbf{F}_{\perp\text{-cred}}$ occurs, we do not need to simulate valid credentials. Thus, $\Pr[\mathbf{W}_1 \wedge \mathbf{F}_{\perp\text{-cred}}] = \Pr[\mathbf{W}_{1,1}]$.

$\mathcal{H}_{1,2}$: This hybrid always sends credential requests to the adversary in the correct order $(\text{creq}^{(0)}, \text{creq}^{(1)})$ without using bit b .

When $b = 0$, the two hybrids are identical, so we only consider the case when $b = 1$. Notice that the only difference between $\text{creq}^{(0)}$ and $\text{creq}^{(1)}$ in $\mathcal{H}_{1,1}$ is which $\text{vpk}^{(i)}$ is used when building them. We prove that if the adversary can

distinguish these hybrids, then it can break the privacy of the underlying SAGA. Below we describe the reduction $\mathcal{R}_{\text{Priv}}$.

$\mathcal{R}_{\text{Priv}}$ receives pp_{SAGA} from the challenger, performs $\text{mKVAC}_{\text{CMZ}}$ setup with that, and sends pp to $\mathcal{A}_1$. When $\mathcal{A}_1$ responds with

$$\text{ipk} := (\text{pk}_{\text{SAGA}}, E), \quad \phi, \quad \text{vpk}^{(i)} := (\vec{X}^{(i)}, X_0^{(i)}, Z_0^{(i)}, \tau^{(i)}), \quad \vec{a}^{(i)},$$

$\mathcal{R}_{\text{Priv}}$ runs $\mathcal{H}_{1.1}$ identically except the following changes when computing the credential requests $\text{creq}^{(i)}$. $\mathcal{R}_{\text{Priv}}$ makes a query to the SAGA Priv challenger as $(\text{pk}_{\text{SAGA}}, (\vec{X}^{(0)}, X_0^{(0)}, Z_0^{(0)}), \tau^{(0)}, (\vec{X}^{(1)}, X_0^{(1)}, Z_0^{(1)}), \tau^{(1)})$. Then $\mathcal{R}_{\text{Priv}}$ receives $(\text{pres}_{\text{SAGA}}^0, \vec{C}^{(0)}, \text{pres}_{\text{SAGA}}^{(1)}, \vec{C}^{(1)})$ and uses them to complete the computation of credentials requests. If the challenge bit of SAGA Priv challenger is 0, then $\mathcal{R}_{\text{Priv}}$ simulates $\mathcal{H}_{1.2}$. Otherwise, it simulates $\mathcal{H}_{1.1}$. Thus, we conclude that

$$\begin{aligned} & |\Pr[\mathbf{W}_{1.1}] - \Pr[\mathbf{W}_{1.2}]| \\ &= |\Pr[\mathbf{W}_{1.1} \wedge b = 0] + \Pr[\mathbf{W}_{1.1} \wedge b = 1] - \Pr[\mathbf{W}_{1.2} \wedge b = 0] - \Pr[\mathbf{W}_{1.2} \wedge b = 1]| \\ &= |\Pr[\mathbf{W}_{1.1} \wedge b = 1] - \Pr[\mathbf{W}_{1.2} \wedge b = 1]| = \text{Adv}_{\text{SAGA}}^{\text{Priv}}(\lambda). \end{aligned}$$

$\mathcal{H}_{1.3}$: This hybrid changes the behavior when $\mathbf{F}_{\perp\text{-cred}}$ occurs: it samples b' randomly, just as in Game_2 . This change is perfectly indistinguishable. Specifically, when $\mathbf{F}_{\perp\text{-cred}}$ occurs, no information about b is leaked through the credentials. Furthermore, $\mathcal{H}_{1.2}$ does not use b when sending the credential requests to the adversary either. Thus, b is information theoretically hidden and $\Pr[\mathbf{W}_{1.2}] = \Pr[\mathbf{W}_{1.3}]$.

We succeeded to add the abort condition of Game_2 in $\mathcal{H}_{1.3}$. The next two hybrids just take back the changes of the prior hybrids on the credential requests. With that, we end up in an identical hybrid to Game_2 .

$\mathcal{H}_{1.4}$: This hybrid again uses the bit b when sending credential requests to the adversary. To show that this change is indistinguishable, one can build a very similar reduction to $\mathcal{R}_{\text{Priv}}$, so $|\Pr[\mathbf{W}_{1.3}] - \Pr[\mathbf{W}_{1.4}]| = \text{Adv}_{\text{SAGA}}^{\text{Priv}}(\lambda)$.

$\mathcal{H}_{1.5}$: This hybrid again computes $\vec{X}_0^{(i)}, \vec{Z}_0^{(i)}, C_a^{(i)}$ as in Game_1 and Game_2 . As argued in $\mathcal{H}_{1.1}$, this change is perfectly indistinguishable. Furthermore, $\mathcal{H}_{1.5}$ is identical to Game_2 . Thus, we conclude that

$$|\Pr[\mathbf{W}_2] - \Pr[\mathbf{W}_1]| = |\Pr[\mathbf{W}_2 \wedge \mathbf{F}_{\perp\text{-cred}}] - \Pr[\mathbf{W}_1 \wedge \mathbf{F}_{\perp\text{-cred}}]| = 2 \cdot \text{Adv}_{\text{SAGA}}^{\text{Priv}}(\lambda).$$

Game_3 : This game adds an abort condition. Namely, if there is a random oracle query with the challenge $\text{creq}^{(0)}$ before it was sent to $\mathcal{A}$, then the game aborts. As $\text{creq}^{(0)}$ contains a perfectly randomized C_a value, the abort condition of this game occurs with probability at most q/p , so $|\Pr[\mathbf{W}_3] - \Pr[\mathbf{W}_2]| \leq q/p$.

Game_4 : This game runs the forking algorithm on the algorithm $\mathcal{B}$ which takes the whole state of the game after the adversary chooses its keys and the challenge attributes. It further takes the state of the game which contains the random oracle table and an independent random bit b^* which simulates the challenge bit of the forked copy. Game_4 aborts if the forking algorithm fails to extract a valid e .

$\mathcal{B} \left(\text{pp}, \text{ipk}, \phi, (\text{vpk}^{(i)}, \tilde{a}^{(i)})_{i \in \{0,1\}}, st_{\mathcal{A},1}, b^*, \text{ROT}_{FS}, h_1, \dots, h_q \right)$
$\text{ctr} := 0$ for $i \in \{0, 1\}$: $(\text{creq}^{(i)}, \text{st}^{(i)}) \leftarrow \text{Obt}'_1(\text{ipk}, \text{vpk}^{(i)}, \tilde{a}^{(i)}, \phi)$ if $\neg\phi(\tilde{a}^{(0)}) \vee \neg\phi(\tilde{a}^{(1)}) \vee \perp \in \{\text{creq}^{(0)}, \text{creq}^{(1)}\}$: return $(\perp, \perp)$ elseif there is a ROT_{FS} entry with $\text{creq}^{(0)}$, then return $(\perp, \perp)$ $(\text{bcred}^{(b^*)}, \text{bcred}^{(1-b^*)}, st_{\mathcal{A},2}) \leftarrow \mathcal{A}_2(st_{\mathcal{A},1}, \text{creq}^{(b^*)}, \text{creq}^{(1-b^*)})$ parse $(\text{st}^{(0)}, \text{bcred}^{(0)})$ as $((s^{(0)}, \tilde{x}_0^{(0)}, \tilde{X}_0^{(0)}, \tilde{Z}_0^{(0)}, \tilde{a}^{(0)}), (\bar{U}^{(0)}, \bar{V}^{(0)}, \pi_I^{(0)}))$ if $\text{NIP}_I.\text{Vf}(x_I^{(0)} := (E, \bar{U}^{(0)}, \bar{V}^{(0)}, \tilde{X}_0^{(0)}, \tilde{Z}_0^{(0)}, C_a^{(0)}, \pi_I^{(0)})$: Find the RO query index $i^* \in [q]$ corresponding to $\pi_I^{(0)}$ and return $(i^*, \pi_I^{(0)})$. return $(\perp, \perp)$ $\text{RO}_{FS}(m)$ if $\text{ROT}_{FS}[m] = \perp$: $\text{ctr} := \text{ctr} + 1$ $\text{ROT}_{FS}[m] := h_{\text{ctr}}$ return $\text{ROT}_{FS}[m]$

Fig. 21. Algorithm $\mathcal{B}$ to be run with the forking algorithm.

Transition Game₃ → Game₄. Notice that the additional copy that the forking algorithm runs is independent of the actual adversary that sees Game₃'s challenge bit related values. Thus, the only change in Game₃'s winning probability is due to the added abort condition on whether the forking algorithm fails. Notice further that $\mathcal{B}$ fails if the adversary does not output a valid NIP_I proof. However, if the adversary has a non-negligible advantage in Game₃, then the adversary must output a valid proof with this probability.

Our formal transition does not rely on indistinguishability of games Game₃ and Game₄. Instead we argue that the adversary's advantages in two games are related to each other. We first analyze the adversary's winning probability in Game₄ (not the advantage but winning probability only). Let the event $\mathbf{F}_{ext}$ be the event that the extraction of e failed. Observe that when $\mathbf{F}_{ext}$ does not occur, we know that Game₄ is identical to Game₃, so $\Pr[\mathbf{W}_4 \mid \neg\mathbf{F}_{ext}] = \Pr[\mathbf{W}_3]$. Furthermore, when $\mathbf{F}_{ext}$ occurs, Game₄ chooses b' randomly, so $\Pr[\mathbf{W}_4 \mid \mathbf{F}_{ext}] = 1/2$. Thus,

$$\begin{aligned}
 \Pr[\mathbf{W}_4] &= \Pr[\neg\mathbf{F}_{ext}] \cdot \Pr[\mathbf{W}_4 \mid \neg\mathbf{F}_{ext}] + \Pr[\mathbf{F}_{ext}] \cdot \Pr[\mathbf{W}_4 \mid \mathbf{F}_{ext}] \\
 &= \Pr[\neg\mathbf{F}_{ext}] \cdot \Pr[\mathbf{W}_3] + (1 - \Pr[\neg\mathbf{F}_{ext}]) \cdot \frac{1}{2} \\
 &= \frac{1}{2} + \Pr[\neg\mathbf{F}_{ext}] \cdot \left(\Pr[\mathbf{W}_3] - \frac{1}{2} \right).
 \end{aligned}$$

Next we analyze the advantage of the adversary in Game_4 . As we know that $\Pr[\neg \mathbf{F}_{ext}] \geq |\Pr[\mathbf{W}_3] - 1/2|^2/q$:

$$\begin{aligned} \left| \Pr[\mathbf{W}_4] - \frac{1}{2} \right| &= \left| \Pr[\neg \mathbf{F}_{ext}] \cdot \left(\Pr[\mathbf{W}_3] - \frac{1}{2} \right) \right| \\ &\geq \left| \frac{|\Pr[\mathbf{W}_3] - 1/2|^2}{q} \cdot \left(\Pr[\mathbf{W}_3] - \frac{1}{2} \right) \right| \\ &\geq \frac{|\Pr[\mathbf{W}_3] - 1/2|^3}{q}. \end{aligned}$$

Thus, we conclude that $\sqrt[3]{q \cdot |\Pr[\mathbf{W}_4] - 1/2|} \geq |\Pr[\mathbf{W}_3] - 1/2|$.

Game₅: This game tries to extract e from (out, out') by relying on the 2-extractability of NIP_I (see Definition 18). If fails, it aborts by simulating b' randomly. The extraction fails only if $in_I = in'_I$. This can be the case only with probability $1/p$ for each $i \in [q]$ in the Fork algorithm. Thus, $|\Pr[\mathbf{W}_5] - \Pr[\mathbf{W}_4]| = 1/p$.

Game₆: This game computes the CMZ tags in final credentials by using the ElGamal decryption key e and vpks , so the credentials do not depend on creqs or bcreds . Formally, given $\text{vpk}^{(i)} := (\vec{X}^{(i)}, X_0^{(i)}, Z_0^{(i)}, \cdot)$ for $i \in \{0, 1\}$, $(U^{(i)}, V^{(i)})$ of $\text{cred}^{(i)}$ is computed as

$$u^{(i)} \xleftarrow{\$} \mathbb{Z}_p^*, \quad U^{(i)} := u^{(i)}G, \quad V^{(i)} := u^{(i)} \left((X_0^{(i)} - eZ_0^{(i)}) + \vec{a}^{(i)} \cdot \vec{X}^{(i)} \right)$$

This change is perfectly indistinguishable, if the adversary did not forge NIP_I proofs in its bcred values as $(U^{(i)}, V^{(i)})$ is a randomly distributed valid CMZ tag on $\vec{a}^{(i)}$. Under the soundness of NIP_I (which implied by the 2-special soundness of the underlying Sigma protocol), we get $|\Pr[\mathbf{W}_6] - \Pr[\mathbf{W}_5]| = \text{Adv}_{\text{NIP}_I}^{\text{Snd}}(\lambda)$.

Game₇: This game samples $\bar{X}_0, \bar{Z}_0, C_a$ randomly for both credential requests, which is a perfectly indistinguishable change. Specifically, $\bar{X}_0, \bar{Z}_0, C_a$ are distributed randomly with the contributions of the terms $\bar{x}_0, \bar{v}, s$, so $\Pr[\mathbf{W}_7] = \Pr[\mathbf{W}_6]$.

Game₈: This game always sends $\text{creq}^{(0)}, \text{creq}^{(1)}$ in the correct order, independently of b , to the adversary. This change is indistinguishable under the privacy property of SAGA. Particularly, one can build a reduction $\mathcal{R}_{\text{Priv}}$ similar to Game_2 if there is an efficient distinguisher, so $|\Pr[\mathbf{W}_8] - \Pr[\mathbf{W}_7]| \leq \text{Adv}_{\text{SAGA}}^{\text{Priv}}(\lambda)$.

Finally, $\Pr[\mathbf{W}_8] = 1/2$ since the challenge bit b is information theoretically hidden in this game. Thus, we conclude that

$$\begin{aligned} \text{Adv}_{\mathcal{A}, \text{mKVAC}}^{\text{IAnon}}(\lambda) &= \left| \Pr \left[\text{Exp}_{\mathcal{A}, \text{mKVAC}}^{\text{IAnon}}(\lambda) = \text{true} \right] - \frac{1}{2} \right| = |\Pr[\mathbf{W}_0] - \frac{1}{2}| \\ &\leq \sqrt[3]{q \cdot \left(\text{Adv}_{\text{SAGA}}^{\text{Priv}}(\lambda) + \text{Adv}_{\text{NIP}_I}^{\text{Snd}}(\lambda) + \frac{1}{p} \right)} \\ &\quad + \frac{q}{p} + 2 \cdot \text{Adv}_{\text{SAGA}}^{\text{Priv}}(\lambda) + \text{Adv}_{\text{NIP}_R}^{\text{ZK}}(\lambda). \end{aligned}$$

□

$\text{DenS}(\text{ipk}, \text{vsk}, \text{ctx}, \phi)$ parse vsk as $(x_0, \dots, x_n, x_r, X_0, \dots, X_n, X_r)$ $\tilde{U}, C_V, C_1, \dots, C_n \xleftarrow{\$} \mathbb{G}, Z := x_0 \tilde{U} + \sum_{j \in [n]} x_j C_j - C_V$ $\mathbb{x}_S := (X_1, \dots, X_n, \tilde{U}), \mathbb{w}_S := (\perp, x_1)$ $\pi_S \leftarrow \text{NIP}_S.\text{Prove}(\mathbb{x}_S, \mathbb{w}_S, (\text{vpk}, \text{ctx}))$ return pres $:= (\tilde{U}, Z, C_V, C_1, \dots, C_n, \pi_S)$
--

Fig. 22. Description of the deniability simulator DenS for $\text{mKVAC}_{\text{CMZ}}$.

G.4 Proof of deniability

Theorem 7 (restated). $\text{mKVAC}_{\text{CMZ}}$ is deniable for DenS in Figure 22 if NIP_I is sound, and NIP_S is zero-knowledge.

Proof. Game₁: This game ensures that cred is a valid credential on a so that its presentations will not be trivially distinguishable from valid presentations of DenS . Namely, additional to checking that π_I holds, Game₁ also checks if $(x_0 + \sum_{j \in [n]} x_j a_j)U = V$. If this does not hold, Game₁ sets $\text{cred} = \perp$. Notice that if π_I is valid but the extra condition introduced in Game₁ does not hold, then the adversary breaks the soundness of NIP_I and we can build a soundness adversary $\mathcal{A}_{\text{Snd}}$, so $|\Pr[\mathbf{W}_1] - \Pr[\mathbf{W}_0]| = \text{Adv}_{\mathcal{A}_{\text{Snd}}, \text{NIP}_I}^{\text{Snd}}(\lambda)$.

Game₂: This game changes how π_S is computed in $\mathcal{O}\text{Chall}$. For both $b = 0$ and $b = 1$, when $\text{NIP}_S.\text{Prove}$, Game₂ simulates π_S . We can build a zero-knowledge adversary $\mathcal{A}_{\text{ZK}}$ against NIP_S if an efficient adversary can distinguish Game₁ and Game₂, so $|\Pr[\mathbf{W}_2] - \Pr[\mathbf{W}_1]| = \text{Adv}_{\mathcal{A}_{\text{ZK}}, \text{NIP}_S, S}^{\text{ZK}}(\lambda)$.

Game₃: This game changes how $\tilde{U}, C_V, C_1, \dots, C_n, Z$ are computed when $b = 0$. Game₃ computes these value identically to DenS . The proof π_S is simulated as before. This change is perfectly indistinguishable as the distribution of output pres is unchanged, $\Pr[\mathbf{W}_3] = \Pr[\mathbf{W}_2]$.

In Game₃, the behavior of $\mathcal{O}\text{Chall}$ is independent of the challenge bit b . Thus, Game₃ leaks no information about b to $\mathcal{A}$ and $\mathcal{A}$ can guess it only randomly: $\Pr[\mathbf{W}_3] \leq 1/2$. As a result, we conclude that

$$\text{Adv}_{\mathcal{A}, \text{mKVAC}_{\text{CMZ}}, \text{DenS}}^{\text{Den}}(\lambda) \leq \text{Adv}_{\mathcal{A}_{\text{Snd}}, \text{NIP}_I}^{\text{Snd}}(\lambda) + \text{Adv}_{\mathcal{A}_{\text{ZK}}, \text{NIP}_S, S}^{\text{ZK}}(\lambda).$$

□

H Deferred Content of Section 4

Definition 20 (Correctness). A scheme SAGA as in Definition 7 is correct if for all security parameters $\lambda \in \mathbb{N}$, all possible groups $(\mathbb{G}, p, G) \in [\text{GGen}(1^\lambda)]$,

all message length parameters ℓ , and all message vectors $\vec{M} \in \mathbb{G}^\ell$:

$$\Pr \left[\begin{array}{l} \text{Setup}(\lambda, \ell, \mathbb{G}, p, G) \rightarrow (\text{pp}_{\text{SAGA}}, \vec{G}, \vec{td}), \\ \text{Kg}(\text{pp}_{\text{SAGA}}) \rightarrow (\text{sk}_{\text{SAGA}}, \text{pk}_{\text{SAGA}}), \text{MAC}(\text{sk}_{\text{SAGA}}, \vec{M}) \rightarrow \tau, \\ \text{Present}(\text{pk}_{\text{SAGA}}, \tau, \vec{M}) \rightarrow (\text{pres}_{\text{SAGA}}, \vec{C}, \vec{\xi}, \text{wits}_{\text{SAGA}}) : \\ \forall j : C_j = M_j + \xi_j G_j \wedge \text{MACVerify}(\text{sk}_{\text{SAGA}}, \tau, \vec{M}) \\ \wedge \text{PresWitVf}^{\text{pres}_{\text{SAGA}}, \vec{C}}(\text{wits}_{\text{SAGA}}, \vec{\xi}) \wedge \text{PresKeyedVf}(\text{sk}_{\text{SAGA}}, \text{pres}_{\text{SAGA}}, \vec{C}) \end{array} \right] = 1.$$

I Failed attempts at BBS-based SAGA

Recall that BBS signatures on a message $m_1 \in \mathbb{Z}_p$ are in the form of $\sigma := (A := (x + d)^{-1}(G_0 + m_1 Y_1), d)$ for a key pair $(\text{sk} := x, \text{pk} := (X := xG, Y_1))$ and $d \xleftarrow{\$} \mathbb{Z}_p$.

First Attempt to Sign Group Elements. Consider our scheme to sign a single group element for simplicity. Given $Y_1 = y_1 G$, the original scheme computes $A := (x + d)^{-1}(G_0 C)$ where $C := m_1 y_1 G$. However, when we wish to create and verify tags on $M_1 := m_1 G$, C must be computed without knowing m_1 . Based on this intuition, we extend the BBS secret key as $\text{sk} := (x, y_1)$, and when signing a group element M_1 — or when verifying a tag on M_1 — C is computed as $C := y_1 M_1$.

Unfortunately this first version turns out to be insecure: Let an adversary obtain a tag on an arbitrary M_1 as $\sigma := (A := (x + d)^{-1}(G_0 + y_1 M_1), d)$. Then the adversary can forge a tag on $M^* := M_1 + X + dG$ as $A^* := (A + Y_1)$ for the same d . This tag is valid since

$$\begin{aligned} (x + d)A^* &= (x + d)A + (x + d)Y_1 \\ &= G_0 + y_1 M_1 + (x + d)Y_1 \\ &= G_0 + y_1 M_1 + y_1(X + dG) \\ &= G_0 + y_1(M_1 + X + dG) = G_0 + y_1 M^* \end{aligned}$$

Fixing the Scheme. The attack above occurs due to the switch in the message space. Notice that the same attack strategy would not work in BBS MAC on field elements as one has to know the discrete logarithm $m^* = m_1 + x + d$ of M^* which requires to compute the secret key x of the signer.

Intuitively, we prevent the above attack by changing the structure of the public key Y_1 . Instead of computing Y_1 with base G , it is computed with a random generator G_1 as $Y_1 := y_1 G_1$. This prevents the above attack as now the forgery message requires to compute $M^* := M_1 + g_1 X + dG$ where g_1 is the discrete logarithm of $G_1 := g_1 G$.

$\text{Exp}_{\text{SAGA}, \mathcal{A}}^{\text{UF-CoMA-MAC}}(\lambda)$	$\mathcal{OMAC}(\alpha_1, \dots, \alpha_\ell)$
$(\mathbb{G}, p, G) \leftarrow \text{GGen}(1^\lambda)$	for $j \in [\ell] : M_j := \alpha_j \cdot G$
$(\text{pp}_{\text{SAGA}}, (G_j, td_j)_{j=1}^\ell) \leftarrow \text{Setup}(1^\lambda, 1^\ell, \mathbb{G}, p, G)$	$\vec{M} := (M_1, \dots, M_\ell)$
$(\text{sk}_{\text{SAGA}}, \text{pk}_{\text{SAGA}}) \leftarrow \text{Kg}(\text{pp}_{\text{SAGA}})$	$\mathcal{Q} \leftarrow \mathcal{Q} \cup \{\vec{M}\}$
$(\tau, \vec{M}) \leftarrow \mathcal{A}^{\mathcal{OMAC}, \mathcal{OMACVerify}}(\text{pp}_{\text{SAGA}}, \text{pk}_{\text{SAGA}})$	return $\text{MAC}(\text{sk}_{\text{SAGA}}, \vec{M})$
$b := \text{MACVerify}(\text{sk}_{\text{SAGA}}, \tau, \vec{M})$	$\mathcal{OMACVerify}(\tau', \vec{M}')$
return $b = 1 \wedge \vec{M} \notin \mathcal{Q}$	return $\text{MACVerify}(\text{sk}_{\text{SAGA}}, \tau', \vec{M}')$

Fig. 23. MAC unforgeability game for SAGA. The adversary has access to $\mathcal{OMAC}$ and $\mathcal{OMACVerify}$ oracles.

J MAC unforgeability definition for SAGA

In this section, we define unforgeability with respect to the MAC component of the SAGA scheme, i.e. it is hard to produce a MAC accepted by MACVerify except by querying an open message MAC oracle $\mathcal{OMAC}$. This property will be used to as an intermediate step in the proof of unforgeability of a full SAGA scheme (see Theorems 1 and 2). The MAC unforgeability game (UF-CoMA-MAC) for SAGA is shown in Figure 23 and it deals with unforgeability of *MAC tags*. Note that this differs from UF-CoMA in Figure 7, which deals with unforgeability of *presentations* of SAGA.

Definition 21 (MAC unforgeability under Chosen open Message Attack). A SAGA scheme satisfies MAC unforgeability under chosen open message attack (UF-CoMA) if for all PPT adversaries $\mathcal{A}$, the advantage

$$\text{Adv}_{\text{SAGA}, \mathcal{A}}^{\text{UF-CoMA-MAC}}(\lambda) := \Pr[\text{Exp}_{\text{SAGA}, \mathcal{A}}^{\text{UF-CoMA-MAC}}(\lambda) = 1]$$

is negligible in λ .

K Proofs for BBSSAGA

Theorem 1 (restated). If $\text{NIP}_{\text{BBSSAGA}}$ is zero-knowledge, then BBSSAGA (Figure 9) is MAC-unforgeable under chosen open message attacks (Definition 21) in GGM.

*Proof.*¹

¹ The presence or absence of oblivious sampling in the GGM proof does not materially affect the security proof in the setting where there is no external extractor (in contrast to settings like proving a proof of knowledge secure in the GGM, where oblivious sampling is crucial to rule out spurious assumptions and proofs). Therefore, we ignore oblivious sampling in our proof. Discussions on this can be found in the literature (e.g., Appendix C of [MBS⁺25]).

We define a sequence of game hops. Let $\mathcal{A}$ be a (generic) adversary against the MAC unforgeability of BBSSAGA as defined in Definition 21 in the generic group model (i.e. both $\mathcal{A}$ and the game receive random encodings of group elements and use oracles to execute group operations).

Before carrying out the generic group model (GGM) analysis, we introduce a sequence of game hops. Let $\mathbf{W}_i$ denote the event that the adversary $\mathcal{A}$ wins in Game_i . Game_0 is identical to the original MAC unforgeability game.

Game_1 : The tag consists of three components $(A, d, \pi_{\text{BBSSAGA}})$. We begin by switching to a modified game in which the adversary receives simulated NIZK proofs π_{BBSSAGA} for its MAC oracle queries. By the zero-knowledge property of the NIZK proof system, these simulated proofs do not leak any information about the secret keys $(x, \vec{y})$. Hence, the adversary's distinguishing advantage between the modified game and the original game is bounded by $\text{Adv}_{\mathcal{A}, \text{NIP}_{\text{BBSSAGA}}}^{\text{ZK}}(\lambda)$. Consequently, the adversary accepts the MAC oracle tags and does not gain any useful information from the NIZK proofs, since they do not rely on the secret keys $(x, \vec{y})$.

$$|\Pr[\mathbf{W}_1] - \Pr[\mathbf{W}_0]| \leq \text{Adv}_{\mathcal{A}, \text{NIP}_{\text{BBSSAGA}}}^{\text{ZK}}(\lambda)$$

Game_2 : This game is identical to the previous one, except that we abort if the responses to at least two distinct MAC queries contain the same value d , since such a collision would allow the adversary to forge a MAC easily, or if $d = -x$. The probability of this event is negligible: for q MAC queries, the collision probability is

$$1 - \frac{p!}{p^q(p-q)!} \approx \frac{q^2}{2p},$$

which is negligible in the security parameter λ , as $p \gg q$. We have,

$$|\Pr[\mathbf{W}_2] - \Pr[\mathbf{W}_1]| \leq \frac{q^2}{2p} + \frac{q}{p}.$$

Game_3 : This game is identical to the previous one, except that the game chooses all q values d_i ($i \in [q]$) at the start of the game. We replace the generator G with $f(x)G$. Let H denote the generator in Game_2 . In Game_3 , we set $G := f(x)H$, where $f(x) := \prod_{i=1}^q (x + d_i)$. We further set $G_0 := g_0G$ (for random $g_0 \xleftarrow{\$} \mathbb{Z}_p$), and all other computations referencing the generator H in Game_2 are now using the new generator G .

In particular, the i -th MAC query is answered as

$$A_i := \frac{1}{(x + d_i)} \left(g_0G + \sum_{j \in [\ell]} y_j m_{i,j} G \right)$$

(where $m_{i,1}, \dots, m_{i,\ell} \in \mathbb{Z}_p$ is the message for the i th query). The public key is $\vec{G} := t\vec{d}G$, meaning $\vec{G} := f(x)t\vec{d}H$. This change prepares for the next game hop by removing divisions $1/(x + d_i)$: A_i can be computed without division. The

adversary's winning probability does not change because $f(x) \neq 0$ (as enforced in Game_2). Hence

$$\Pr[\mathbf{W}_3] = \Pr[\mathbf{W}_2].$$

Game₄: This game is identical to the previous one, except that we replace the generic group model with a polynomial-based execution, as usual in GGM proofs. This means that instead of randomly encoded group elements, the GGM is now based on randomly encoded polynomials in the ring

$$\mathbb{Z}_p[g_0, (td_j, y_j)_{j=1}^\ell, x].$$

As a result, **Game₄** now treats $G, g_0, (td_j, y_j)_{j=1}^\ell, x$ as polynomial variables instead of choosing random scalar values in $\mathbb{Z}_p$ for them. The **Game₂** generator H in **Game₃** corresponds to the polynomial 1, the **Game₃** generator G corresponds to the polynomial $f(x)$ (we simply write $G := f(x)$). The group operation $B + B'$ corresponds to polynomial addition $b + b'$, where b is the polynomial corresponding to B and b' is the polynomial corresponding to B' . Similarly, scalar multiplication aB in the group corresponds to scalar multiplication $a \cdot B$ in the polynomial ring.

The $\text{MACVerify}(\tau' = (A', d'), \vec{M}')$ oracle stops computing group operations and instead directly checks the polynomial equation $(x + d')A' = g_0 + \vec{y} \cdot \vec{M}'$ over the ring $\mathbb{Z}_p[g_0, (td_j, y_j)_{j=1}^\ell, x]$. (note that we are interpreting A', M'_j as polynomials corresponding to the randomly encoded group elements supplied by the adversary to the oracle).

Game₄ produces the same distribution of encoded group elements as **Game₃** unless there are two different encoded polynomials $g \neq g' \in \mathbb{Z}_p[g_0, (td_j, y_j)_{j=1}^\ell, x]$ that evaluate to the same scalar element when $g_0, (td_j, y_j)_{j=1}^\ell, x$ are instantiated with random $\mathbb{Z}_p$ elements. By the Schwartz-Zippel lemma, for any pair $g \neq g'$, such a collision happens with probability at most t/p , where t is the degree of $g - g'$. If the game overall makes at most q_{GGM} group oracle queries, then applying the union bound over all pairs g, g' gives us

$$|\Pr[\mathbf{W}_4] - \Pr[\mathbf{W}_3]| \leq \frac{(q_{\text{GGM}})^2 t}{p},$$

with $t := q + 1$. This is because MAC responses A_i have degree at most q , since f is degree q . MACVerify multiplies arbitrary polynomials with y_j , resulting in the maximum degree $q + 1$.

In the remainder of this proof, we show that the probability to create a forgery in **Game₄** is zero.

Suppose the adversary outputs a forgery consisting of a tag-message pair (A^*, d^*) and $(M_1^*, \dots, M_\ell^*)$ fulfilling the verification equation

$$(x + d^*)A^* = g_0$$

We argue that such a forgery can only be valid if the forged pair corresponds to one of the messages previously queried to the oracle together with its returned

tag. Since the definition of the game requires the forgery to be on a fresh message (i.e., one not in the query set), this case is ruled out. Therefore, the adversary cannot produce a valid forgery.

Now consider an execution of **Game**₄. Suppose the adversary makes at most q queries. For the i -th queried message $(m_{i,1}, \dots, m_{i,\ell})$, where $i \in [q]$, the MAC oracle returns the pair (A_i, d_i) , where

$$A_i = \frac{1}{(x + d_i)} \left(g_0 G + \sum_{j \in [\ell]} y_j m_{i,j} G \right) = \frac{f(x)}{(x + d_i)} \left(g_0 + \sum_{j \in [\ell]} y_j m_{i,j} \right)$$

due to $G = f(x)$. The list of group elements the adversary sees prior to the forgery is (in terms of polynomials)

$$(g_0 G, (td_j G, y_j td_j G)_{j \in [\ell]}, xG, (A_i)_{i \in [q]}).$$

where $g_0 G$ corresponds to G_0 , the polynomial $td_j G$ corresponds to $G_j := td_j$, the polynomial $y_j td_j G$ corresponds to $Y_j := y_j G_j$, and xG corresponds to $X := xG$. Note that there are no constant terms because we do not give H to the adversary. (Additionally, the adversary gets simulated proofs π_{BBSAGA} , which we omit because they do not play a role in this proof; the adversary can compute these proofs itself using the ZK simulator).

A forged MAC (A^*, d^*) on a message $(M_1^*, \dots, M_\ell^*)$ can be expressed as a linear combination of these elements, using coefficients from a vector $\vec{\alpha}$:

$$A^* = \alpha_a G + \alpha_{a,g_0} g_0 G + \sum_{j \in [\ell]} (\alpha_{a,td_j} td_j G + \alpha_{a,y_j} y_j td_j G) + \alpha_{a,x} xG + \sum_{i \in [q]} (\alpha_{a,a_i} A_i)$$

$$\begin{aligned} \forall k \in [\ell] : M_k^* = \alpha_{m_k} G + \alpha_{m_k,g_0} g_0 G + \sum_{j \in [\ell]} (\alpha_{m_k,td_j} td_j G + \alpha_{m_k,y_j} y_j td_j G) + \alpha_{m_k,x} xG \\ + \sum_{i \in [q]} (\alpha_{m_k,a_i} A_i) \end{aligned}$$

Let

$$f(x) := \prod_{i \in [q]} (x + d_i) \quad \text{and} \quad f_k(x) := \frac{f(x)}{x + d_k},$$

for each $k \in [q]$. Then:

$$\begin{aligned} A^* = \alpha_a G + \alpha_{a,g_0} g_0 G + \sum_{j \in [\ell]} (\alpha_{a,td_j} td_j G + \alpha_{a,y_j} y_j td_j G) + \\ \alpha_{a,x} xG + \sum_{i \in [q]} (\alpha_{a,a_i} f_i(x) (g_0 G + \sum_{j \in [\ell]} y_j m_{i,j} G)) \end{aligned}$$

$$\begin{aligned}
\forall k \in [\ell] : M_k^* &= \alpha_{m_k} G + \alpha_{m_k, g_0} g_0 G \\
&+ \sum_{j \in [\ell]} (\alpha_{m_k, td_j} td_j G + \alpha_{m_k, y_j} y_j td_j G) + \alpha_{m_k, x} x G \\
&+ \sum_{i \in [q]} (\alpha_{m_k, a_i} f_i(x) (g_0 G + \sum_{j \in [\ell]} y_j m_{i,j} G))
\end{aligned}$$

A successful forgery must satisfy the following equation:

$$(x + d^*) A^* = g_0 G + \sum_{k \in [\ell]} y_k M_k^*$$

In the right-hand side, there appear monomials of the form $y_k y_j td_j$, $y_k y_j$, $y_k g_0$, and $y_k td_j$ for $j \neq k$, while no such terms occur in the left-hand side. Therefore, it must hold that

$$\alpha_{m_k, a_i} = \alpha_{m_k, y_j} = \alpha_{m_k, g_0} = \alpha_{m_k, td_j} = 0$$

for all $j, k \in [\ell]$ with $j \neq k$, and for all $i \in [q]$. Moreover, every monomial on the right-hand side contains either y_k or g_0 . Hence, no term of the form td_j , G , or xG appears, which implies that

$$\alpha_{a, td_j} = \alpha_a = \alpha_{a, x} = 0 \quad \text{for all } j \in [\ell].$$

Thus, we obtain:

$$\begin{aligned}
&\alpha_{a, g_0} (x + d^*) f(x) g_0 + \sum_{j \in [\ell]} (\alpha_{a, y_j} (x + d^*) f(x) y_j td_j) \\
&+ \sum_{i \in [q]} (\alpha_{a, a_i} (x + d^*) f_i(x) (g_0 + \sum_{j \in [\ell]} y_j m_{i,j})) \\
&= f(x) g_0 + \sum_{k \in [\ell]} y_k (\alpha_{m_k, g_k} f(x) g_k + \alpha_{m_k} f(x) + \alpha_{m_k, x} f(x) x) \quad (2)
\end{aligned}$$

using $G = f(x)$. Coefficients of g_0 must be equal on both sides, so:

$$\alpha_{a, g_0} (x + d^*) f(x) + \sum_{i \in [q]} (\alpha_{a, a_i} (x + d^*) f_i(x)) = f(x)$$

Since the left-hand side is divisible by $(x + d^*)$ and the right-hand side contains $f(x) = \prod_{i \in [q]} (x + d_i)$, we must have $d^* \in \{d_1, \dots, d_q\}$. Without loss of generality, assume $d^* = d_1$. Comparing degrees, the left-hand side contains the monomial $\alpha_{a, g_0} x^{q+1}$, whereas the right-hand side does not; hence

$$\alpha_{a, g_0} = 0.$$

Therefore, we have:

$$\alpha_{a, a_1} f(x) + \sum_{i \in [q] \setminus \{1\}} (\alpha_{a, a_i} (x + d_1) f_i(x)) = f(x)$$

$$\rightarrow \sum_{i \in [q] \setminus \{1\}} (\alpha_{a,a_i}(x + d_1)f_i(x)) = (1 - \alpha_{a,a_1})f(x)$$

Observe that the left-hand side has a double root at d_1 , whereas the right-hand side has only a single root at d_1 . This forces $\alpha_{a,a_1} = 1$ and, consequently, $\alpha_{a,a_i} = 0$ for all $i \in [q] \setminus \{1\}$. Returning to Equation (2), we obtain:

$$\begin{aligned} & \sum_{j \in [\ell]} (\alpha_{a,y_j}(x + d_1)f(x)y_jtd_j) + f(x)(g_0 + \sum_{j \in [\ell]} y_jm_{1,j}) \\ &= f(x)g_0 + \sum_{k \in [\ell]} y_k(\alpha_{m_k,g_k}f(x)g_k + \alpha_{m_k}f(x) + \alpha_{m_k,x}f(x)x) \end{aligned}$$

By replacing k with j on the right-hand side and cancelling the term $f(x)g_0$ present on both sides, we obtain:

$$\begin{aligned} & \rightarrow \sum_{j \in [\ell]} \alpha_{a,y_j}(x + d_1)f(x)y_jtd_j + f(x)y_jm_{1,j} \\ &= \sum_{j \in [\ell]} \alpha_{m_j,td_j}f(x)y_jtd_j + \alpha_{m_j}f(x)y_j + \alpha_{m_j,x}f(x)xy_j \quad (3) \end{aligned}$$

The coefficients of y_jtd_j must match on both sides, so for each $j \in [\ell]$ we have:

$$\alpha_{a,y_j}(x + d_1)f(x) = \alpha_{m_j,td_j}f(x).$$

Since the left-hand side has a double root at d_1 while the right-hand side has only a single root, we conclude that $\alpha_{a,y_j} = \alpha_{m_j,td_j} = 0$ for all $j \in [\ell]$. Returning to Equation (3), we obtain:

$$\rightarrow \sum_{j \in [\ell]} f(x)y_jm_{1,j} = \sum_{j \in [\ell]} \alpha_{m_j}f(x)y_j + \alpha_{m_j,x}f(x)xy_j.$$

Hence, for each $j \in [\ell]$, we have

$$\rightarrow m_{1,j} = \alpha_{m_j} + \alpha_{m_j,x}x.$$

Therefore, we have

$$\alpha_{m_j} = m_{1,j} \quad \text{and} \quad \alpha_{m_j,x} = 0.$$

As a result, the only possible forgery is

$$A^* = A_1, \quad \forall k \in [\ell] : M_k^* = m_{1,k}G = M_{1,k}.$$

This is not a valid forgery because the message has been queried before. Therefore, the advantage of the adversary in the original MAC unforgeability game is bounded as follows:

$$\text{Adv}_{\text{SAGA}, \mathcal{A}}^{\text{UF-CoMA-MAC}}(\lambda) \leq \text{Adv}_{\mathcal{A}, \text{NIP}_{\text{BSSAGA}}}^{\text{ZK}}(\lambda) + \frac{q^2}{2p} + \frac{q}{p} + \frac{(q_{\text{GGM}})^2(q+1)}{p},$$

Theorem 2 (restated). *If BBSSAGA (Figure 9) is MAC-unforgeable under chosen open message attacks (Definition 21), then BBSSAGA is unforgeable under chosen open message attack (Definition 8).*

Proof. We reduce the SAGA unforgeability game to the MAC unforgeability game. To simulate the MAC oracle, we answer each MAC query using the oracle from the MAC unforgeability game.

To simulate a query $\mathcal{OPresKeyedVf}(\text{pres}'_{\text{SAGA}}, \text{wit}'_{\text{SAGA}}, \vec{C}', \vec{\xi}')$, we first parse (r', d') from $\text{wit}'_{\text{SAGA}}$ and (C'_A, T') from $\text{pres}'_{\text{SAGA}}$, and check whether

$$T' = r'X - d'C'_A + d'r'G - \vec{\xi}' \cdot \vec{Y}.$$

If the check fails, we return 0 and abort. Otherwise, we continue by computing

$$\tau' = C'_A - r'G, \quad M'_j = C'_j - \xi'_j G_j \quad \text{for } j \in [\ell].$$

We then query $\mathcal{OMACVerify}(\tau', M'_1, \dots, M'_\ell)$ and use its output as the answer to $\mathcal{OPresKeyedVf}$.

When the adversary outputs $(\text{pres}_{\text{SAGA}}, \text{wit}_{\text{SAGA}}, (C_j, \xi_j)_{j=1}^\ell)$, we first parse (r, d) from wit_{SAGA} and (C_A, T) from $\text{pres}_{\text{SAGA}}$, and check whether

$$T = rX - dC_A + drG - \vec{\xi} \cdot \vec{Y}.$$

If the check fails, we return 0 and abort. Otherwise, we continue by computing

$$\tau = C_A - rG, \quad M_j = C_j - \xi G_j \quad \text{for } j \in [\ell].$$

We then submit $(\tau, M_1, \dots, M_\ell)$ as the adversary's forgery in the MAC unforgeability game and output the bit b returned by the MAC game. Therefore, the adversary's advantage in this game is exactly the same as in the MAC unforgeability game, i.e.,

$$\text{Adv}_{\text{SAGA}, \mathcal{A}}^{\text{UF-CoMA}}(\lambda) = \text{Adv}_{\text{SAGA}, \mathcal{A}}^{\text{UF-CoMA-MAC}}(\lambda),$$

which is negligible. $\square$

Theorem 3 (restated). *BBSSAGA (Figure 9) satisfies the privacy property (Definition 9) if $\text{NIP}_{\text{BBSSAGA}}$ is sound.*

Proof. We define new experiments $\text{Exp}_{\text{SAGA}, \mathcal{A}}^{\text{PRIV}'-b}(\lambda)$, which work exactly like the original experiments $\text{Exp}_{\text{SAGA}, \mathcal{A}}^{\text{PRIV}-b}(\lambda)$ in Figure 8, but with one additional step. The new (inefficient) experiments $\text{Exp}_{\text{SAGA}, \mathcal{A}}^{\text{PRIV}'-b}(\lambda)$ additionally check whether there exists a unique pair $(x, \vec{y})$ that ensures the well formedness of

$$\tau^{(0)} := (A^{(0)}, d^{(0)}, \pi_{\text{BBSSAGA}}^{(0)}) \quad \text{and} \quad \tau^{(1)} := (A^{(1)}, d^{(1)}, \pi_{\text{BBSSAGA}}^{(1)}).$$

That is, there exists a unique $(x, \vec{y})$ such that the following holds:

$$\forall i \in \{0, 1\} : X = xG \wedge \vec{Y} = \vec{y} \circ \vec{G} \wedge (x + d^{(i)})A^{(i)} = G_0 + \vec{y} \cdot \vec{M}^{(i)}.$$

If no such $(x, \vec{y})$ exists, the experiment returns 0.

We now claim that $\text{Exp}_{\text{SAGA}, \mathcal{A}}^{\text{PRIV}'-b}$ is indistinguishable from the original $\text{Exp}_{\text{SAGA}, \mathcal{A}}^{\text{PRIV}-b}$. The only difference between the two experiments is the additional check. The check succeeds if and only if there exist witnesses $w_{\text{BBSSAGA}}^{(0)}, w_{\text{BBSSAGA}}^{(1)}$ for the statements $x_{\text{BBSSAGA}}^{(0)}, x_{\text{BBSSAGA}}^{(1)}$ in BBSSAGA.Present such that $(x_{\text{BBSSAGA}}^{(i)}, w_{\text{BBSSAGA}}^{(i)}) \in \mathcal{R}_{\text{BBSSAGA}}$. As such, the only differentiating event between the two experiments is if $x_{\text{BBSSAGA}}^{(i)}$ is not in the language of $\mathcal{R}_{\text{BBSSAGA}}$ but the proof $\pi_{\text{BBSSAGA}}^{(i)}$ is accepted by BBSSAGA.Present . Through a straightforward reduction to the soundness property of $\text{NIP}_{\text{BBSSAGA}}$, which runs $\text{Exp}_{\text{SAGA}, \mathcal{A}}^{\text{PRIV}-b}$, chooses a random $i \xleftarrow{\$} \{0, 1\}$ and outputs $(x_{\text{BBSSAGA}}^{(i)}, \text{ctx}_{\text{NIP}}^{(i)} := (), \pi_{\text{BBSSAGA}}^{(i)})$ as a potentially unsound proof, one can show that this event must happen with negligible probability. Concretely,

$$\left| \Pr \left[\text{Exp}_{\text{SAGA}, \mathcal{A}}^{\text{PRIV}-b} \right] - \Pr \left[\text{Exp}_{\text{SAGA}, \mathcal{A}}^{\text{PRIV}'-b} \right] \right| \leq 2\text{Adv}_{\mathcal{A}_{\text{SE}}, \text{NIP}_{\text{BBSSAGA}}}^{\text{SE}}(\lambda).$$

We now show that the adversary has no advantage in distinguishing $\text{Exp}_{\text{SAGA}, \mathcal{A}}^{\text{PRIV}'-0}$ from $\text{Exp}_{\text{SAGA}, \mathcal{A}}^{\text{PRIV}'-1}$.

If $(A^{(i)}, d^{(i)})$ is not well formed for at least one $i \in \{0, 1\}$, then the adversary receives 0 for both choices of b . Therefore, in this case, it cannot distinguish between the experiments for different values of b . Otherwise, we need to show that the outputs of $\text{Present}(\text{pk}_{\text{SAGA}}, \tau^{(0)}, \vec{M}^{(0)})$ and $\text{Present}(\text{pk}_{\text{SAGA}}, \tau^{(1)}, \vec{M}^{(1)})$ are indistinguishable. After running $\text{Present}(\text{pk}_{\text{SAGA}}, \tau^{(i)}, \vec{M}^{(i)})$, the tuple $(C_A^{(i)}, T^{(i)})$ in $\text{pres}_{\text{SAGA}}^{(b)}$ and the elements $(C_1^{(i)}, \dots, C_\ell^{(i)})$ are returned.

Due to the random values $r^{(i)}$ in $\text{wit}_{\text{SAGA}}^{(i)}$ and $(\xi_1^{(i)}, \dots, \xi_\ell^{(i)})$, each of $C_A^{(i)}$ and $C_1^{(i)}, \dots, C_\ell^{(i)}$ is distributed uniformly at random over the group:

$$C_A^{(i)} = A^{(i)} + r^{(i)}G,$$

$$\vec{C}^{(i)} = \vec{M}^{(i)} + \xi^{(i)} \circ \vec{G}.$$

Moreover, since $(A^{(i)}, d^{(i)})$ is well formed for both $i \in \{0, 1\}$, we have

$$(x + d^{(i)})A^{(i)} = G_0 + \sum_{j \in [\ell]} y_j M_j^{(i)},$$

and

$$\begin{aligned} T^{(i)} &:= r^{(i)}X - d^{(i)}C_A^{(i)} + d^{(i)}r^{(i)}G - \xi^{(i)} \cdot \vec{Y} = r^{(i)}X - d^{(i)}A^{(i)} - \xi^{(i)} \cdot \vec{Y} \\ &\rightarrow T^{(i)} = r^{(i)}X + xA^{(i)} - xA^{(i)} - d^{(i)}A^{(i)} + \sum_{j \in [\ell]} y_j M_j^{(i)} - \sum_{j \in [\ell]} y_j M_j^{(i)} - \xi^{(i)} \cdot \vec{Y} \\ &\rightarrow T^{(i)} = xC_A^{(i)} - G_0 - \sum_{j \in [\ell]} y_j M_j^{(i)} + \sum_{j \in [\ell]} y_j M_j^{(i)} - \vec{Y} \cdot (\vec{M}^{(i)} + \xi^{(i)} \circ \vec{G}) \end{aligned}$$

Then, we obtain

$$T^{(i)} = xC_A^{(i)} - G_0 - \sum_{j \in [\ell]} y_j C_j^{(i)},$$

which is exactly the condition checked by the verification algorithm.

Therefore, regardless of i , when running $\text{Present}(\text{pk}_{\text{SAGA}}, \tau^{(i)}, \vec{M}^{(i)})$, the output consists of uniformly random group elements $(C_A^{(i)}, \vec{C}^{(i)})$ and a unique group element determined by the random group elements $(C_A^{(i)}, \vec{C}^{(i)})$ and the unique values $(x, \vec{y})$. This implies that the outputs of $\text{Present}(\text{pk}_{\text{SAGA}}, \tau^{(0)}, \vec{M}^{(0)})$ and $\text{Present}(\text{pk}_{\text{SAGA}}, \tau^{(1)}, \vec{M}^{(1)})$ are indistinguishable.

As a result, $\Pr[\text{Exp}_{\text{SAGA}, \mathcal{A}}^{\text{PRIV}'-0}(\lambda) = 1] = \Pr[\text{Exp}_{\text{SAGA}, \mathcal{A}}^{\text{PRIV}'-1}(\lambda) = 1]$. Therefore,

$$\begin{aligned} & \text{Adv}_{\text{SAGA}, \mathcal{A}}^{\text{PRIV}}(\lambda) \\ &= \left| \Pr[\text{Exp}_{\text{SAGA}, \mathcal{A}}^{\text{PRIV}-0}(\lambda) = 1] - \Pr[\text{Exp}_{\text{SAGA}, \mathcal{A}}^{\text{PRIV}-1}(\lambda) = 1] \right| \\ &= \left| \Pr[\text{Exp}_{\text{SAGA}, \mathcal{A}}^{\text{PRIV}-0}(\lambda) = 1] - \Pr[\text{Exp}_{\text{SAGA}, \mathcal{A}}^{\text{PRIV}'-0}(\lambda) = 1] \right| \\ &\quad - \left| \Pr[\text{Exp}_{\text{SAGA}, \mathcal{A}}^{\text{PRIV}-1}(\lambda) = 1] - \Pr[\text{Exp}_{\text{SAGA}, \mathcal{A}}^{\text{PRIV}'-1}(\lambda) = 1] \right| \\ &\leq \left| \Pr[\text{Exp}_{\text{SAGA}, \mathcal{A}}^{\text{PRIV}-0}(\lambda) = 1] - \Pr[\text{Exp}_{\text{SAGA}, \mathcal{A}}^{\text{PRIV}'-0}(\lambda) = 1] \right| \\ &\quad + \left| \Pr[\text{Exp}_{\text{SAGA}, \mathcal{A}}^{\text{PRIV}-1}(\lambda) = 1] - \Pr[\text{Exp}_{\text{SAGA}, \mathcal{A}}^{\text{PRIV}'-1}(\lambda) = 1] \right| \\ &\leq 4\text{Adv}_{\text{SE}, \text{NIP}_{\text{BBSAGA}}}^{\text{SE}}(\lambda), \end{aligned}$$

which is negligible. □

L A SAGA based on CPZ

In this section, we describe how to instantiate SAGA using CPZ KVACs [CPZ20]. CPZ is a full-fledged KVAC on group elements already. Thus, it is actually more powerful than needed to instantiate SAGA. Namely, its unforgeability holds even without knowing the discrete logarithms of the signed group elements. Furthermore, the presentation of a CPZ-based SAGA is shorter than our BBS-based instantiation. However, the efficiency of CPZ KVACs come with a caveat: they only provide computational anonymity under DDH assumption. This means that the issuance anonymity of $\text{mKVAC}_{\text{CMZ}}$ instantiated with CPZ-based SAGA would only hold under the DDH assumption. This leaves room for *harvest now, deanonymize later* attacks when DDH problem becomes easy to solve e.g. with quantum computers.

In Figure 24, we describe CPZSAGA, a SAGA instantiation based on CPZ KVACs [CPZ20]. When signing a CPZSAGA tag, additional to computing a CPZ tag (A, B, d) , a CPZSAGA signer also issues a NIP proof π_{CPZ} which shows the

well-formedness of the issued tag according to the relation

$$\begin{aligned} \mathcal{R}_{\text{CPZ}} &:= \left\{ (\omega, \tilde{\omega}, \alpha_0, \alpha_1, \beta_1, \dots, \beta_\ell) : \right. \\ C_W &= \omega G_\omega + \tilde{\omega} G_{\tilde{\omega}} \wedge \\ I &= G_B - (\alpha_0 G_{\alpha_0} + \alpha_1 G_{\alpha_1} + \beta_1 G_1 + \dots + \beta_\ell G_\ell) \wedge \\ B &= \omega G_\omega + \alpha_0 A + \alpha_1 dA + \sum_{j \in [\ell]} \beta_j M_j \left. \right\}. \end{aligned}$$

This proof ensures us that a malicious signer cannot link the presentations of a user by issuing malformed credentials. The presentation of **CPZSAGA** and **PresWitVf** are identical the **CPZ KVAC** presentation in which all attributes are hidden.

$\text{Setup}(1^\lambda, 1^\ell, \mathbb{G}, p, G)$ $G_\omega, G_{\tilde{\omega}}, G_{\alpha_0}, G_{\alpha_1}, G_B \xleftarrow{\$} \mathbb{G}$ for $j \in [\ell] : td_j \xleftarrow{\$} \mathbb{Z}_p, G_j := td_j G$ return $(\text{pp}_{\text{SAGA}} := (\mathbb{G}, p, G, G_\omega, G_{\tilde{\omega}}, G_{\alpha_0}, G_{\alpha_1}, G_B), (G_j, td_j)_{j=1}^\ell)$	
$\text{Kg}(\text{pp}_{\text{SAGA}}, \mathbb{G}, p, G)$ $(\omega, \tilde{\omega}, \alpha_0, \alpha_1, \beta_1, \dots, \beta_\ell) \xleftarrow{\$} \mathbb{Z}_p^{\ell+4}$ $C_W := \omega G_\omega + \tilde{\omega} G_{\tilde{\omega}}, I := G_B - (\alpha_0 G_{\alpha_0} + \alpha_1 G_{\alpha_1} + \beta_1 G_1 + \dots + \beta_\ell G_\ell)$ return $(\text{sk}_{\text{SAGA}} := (\omega, \tilde{\omega}, \alpha_0, \alpha_1, \beta_1, \dots, \beta_\ell), \text{pk}_{\text{SAGA}} := (C_W, I))$	
$\text{MAC}(\text{sk}_{\text{SAGA}}, M_1, \dots, M_\ell)$ parse sk_{SAGA} as $(\omega, \tilde{\omega}, \alpha_0, \alpha_1, \beta_1, \dots, \beta_\ell)$ $A \xleftarrow{\$} \mathbb{G}, d \xleftarrow{\$} \mathbb{Z}_p$ $B := \omega G_\omega + (\alpha_0 + \alpha_1 d)A + \sum_{i \in [\ell]} \beta_i M_i$ $\mathbb{X}_{\text{CPZ}} := (A, B, d, M_1, \dots, M_\ell)$ $\mathbb{W}_{\text{CPZ}} := (\omega, \tilde{\omega}, \alpha_0, \alpha_1, \beta_1, \dots, \beta_\ell)$ $\pi_{\text{CPZ}} \leftarrow \text{NIP}_{\text{CPZ}}.\text{Prove}(\mathbb{X}_{\text{CPZ}}, \mathbb{W}_{\text{CPZ}})$ return $\tau := (A, B, d, \pi_{\text{CPZ}})$	$\text{Present}(\text{pk}_{\text{SAGA}}, \tau, M_1, \dots, M_\ell)$ parse τ as $(A, B, d, \pi_{\text{CPZ}})$ parse pk_{SAGA} as (C_W, I) $\mathbb{X}_{\text{CPZ}} := (A, B, d, M_1, \dots, M_\ell)$ require $\text{NIP}_{\text{CPZ}}.\text{Vf}(\mathbb{X}_{\text{CPZ}}, \pi_{\text{CPZ}})$ $\xi \leftarrow \mathbb{Z}_p, \xi_0 := -d\xi, J := \xi I$ $C_{\alpha_0} := \xi G_{\alpha_0} + A, C_{\alpha_1} := \xi G_{\alpha_1} + dA$ $C_B := \xi G_B + B$ for $j \in [\ell] : C_j := \xi G_j + M_j$ return $(\text{pres}_{\text{SAGA}} := (J, C_{\alpha_0}, C_{\alpha_1}, C_B), (C_j, \xi)_{j=1}^\ell, \text{wits}_{\text{SAGA}} := (\xi_0, d))$
$\text{PresWitVf}^{\text{pres}_{\text{SAGA}} = (J, C_{\alpha_0}, C_{\alpha_1}, C_B), C_1, \dots, C_\ell}(\text{wits}_{\text{SAGA}} = (\xi_0, d), \xi, \dots, \xi)$ return $J = \xi I \wedge C_{\alpha_1} = dC_{\alpha_0} + \xi_0 G_{\alpha_0} + \xi G_{\alpha_1}$	
$\text{PresKeyedVf}(\text{sk}_{\text{SAGA}}, \text{pres}_{\text{SAGA}} = (J, C_{\alpha_0}, C_{\alpha_1}, C_B), C_1, \dots, C_\ell)$ return $C_B = \omega G_\omega + \alpha_0 C_{\alpha_0} + \alpha_1 C_{\alpha_1} + \sum_{j \in [\ell]} \beta_j C_j$	
$\text{MACVerify}(\text{sk}_{\text{SAGA}}, \tau = (A, B, d, \pi_{\text{CPZ}}), M_1, \dots, M_\ell)$ return $B = \omega G_\omega + (\alpha_0 + \alpha_1 d)A + \sum_{j \in [\ell]} \beta_j M_j$	

Fig. 24. Description of CPZSAGA.